

# Boolean Algebra and Logic Simplification

## CHAPTER OUTLINE

- 4-1 Boolean Operations and Expressions
- 4-2 Laws and Rules of Boolean Algebra
- 4-3 DeMorgan's Theorems
- 4-4 Boolean Analysis of Logic Circuits
- 4-5 Logic Simplification Using Boolean Algebra
- 4-6 Standard Forms of Boolean Expressions
- 4-7 Boolean Expressions and Truth Tables
- 4-8 The Karnaugh Map
- 4-9 Karnaugh Map SOP Minimization
- 4-10 Karnaugh Map POS Minimization
- 4-11 The Quine-McCluskey Method
- 4-12 Boolean Expressions with VHDL  
Applied Logic

## CHAPTER OBJECTIVES

- Apply the basic laws and rules of Boolean algebra
- Apply DeMorgan's theorems to Boolean expressions
- Describe gate combinations with Boolean expressions
- Evaluate Boolean expressions
- Simplify expressions by using the laws and rules of Boolean algebra
- Convert any Boolean expression into a sum-of-products (SOP) form
- Convert any Boolean expression into a product-of-sums (POS) form
- Relate a Boolean expression to a truth table
- Use a Karnaugh map to simplify Boolean expressions
- Use a Karnaugh map to simplify truth table functions
- Utilize "don't care" conditions to simplify logic functions
- Use the Quine-McCluskey method to simplify Boolean expressions
- Write a VHDL program for simple logic

- Apply Boolean algebra and the Karnaugh map method in an application

## KEY TERMS

Key terms are in order of appearance in the chapter.

- |                         |                         |
|-------------------------|-------------------------|
| ■ Variable              | ■ Product-of-sums (POS) |
| ■ Complement            | ■ Karnaugh map          |
| ■ Sum term              | ■ Minimization          |
| ■ Product term          | ■ "Don't care"          |
| ■ Sum-of-products (SOP) |                         |

## VISIT THE WEBSITE

Study aids for this chapter are available at <http://www.pearsonglobaleditions.com/floyd>

## INTRODUCTION

In 1854, George Boole published a work titled *An Investigation of the Laws of Thought, on Which Are Founded the Mathematical Theories of Logic and Probabilities*. It was in this publication that a "logical algebra," known today as Boolean algebra, was formulated. Boolean algebra is a convenient and systematic way of expressing and analyzing the operation of logic circuits. Claude Shannon was the first to apply Boole's work to the analysis and design of logic circuits. In 1938, Shannon wrote a thesis at MIT titled *A Symbolic Analysis of Relay and Switching Circuits*.

This chapter covers the laws, rules, and theorems of Boolean algebra and their application to digital circuits. You will learn how to define a given circuit with a Boolean expression and then evaluate its operation. You will also learn how to simplify logic circuits using the methods of Boolean algebra, Karnaugh maps, and the Quine-McCluskey method.

Boolean expressions using the hardware description language VHDL are also covered.

## 4-1 Boolean Operations and Expressions

Boolean algebra is the mathematics of digital logic. A basic knowledge of Boolean algebra is indispensable to the study and analysis of logic circuits. In the last chapter, Boolean operations and expressions in terms of their relationship to NOT, AND, OR, NAND, and NOR gates were introduced.

After completing this section, you should be able to

- ◆ Define *variable*
- ◆ Define *literal*
- ◆ Identify a sum term
- ◆ Evaluate a sum term
- ◆ Identify a product term
- ◆ Evaluate a product term
- ◆ Explain Boolean addition
- ◆ Explain Boolean multiplication

### InfoNote

In a microprocessor, the arithmetic logic unit (ALU) performs arithmetic and Boolean logic operations on digital data as directed by program instructions. Logical operations are equivalent to the basic gate operations that you are familiar with but deal with a minimum of 8 bits at a time. Examples of Boolean logic instructions are AND, OR, NOT, and XOR, which are called *mnemonics*. An assembly language program uses the mnemonics to specify an operation. Another program called an *assembler* translates the mnemonics into a binary code that can be understood by the microprocessor.

*Variable*, *complement*, and *literal* are terms used in Boolean algebra. A **variable** is a symbol (usually an italic uppercase letter or word) used to represent an action, a condition, or data. Any single variable can have only a 1 or a 0 value. The **complement** is the inverse of a variable and is indicated by a bar over the variable (overbar). For example, the complement of the variable  $A$  is  $\bar{A}$ . If  $A = 1$ , then  $\bar{A} = 0$ . If  $A = 0$ , then  $\bar{A} = 1$ . The complement of the variable  $A$  is read as “not  $A$ ” or “ $A$  bar.” Sometimes a prime symbol rather than an overbar is used to denote the complement of a variable; for example,  $B'$  indicates the complement of  $B$ . In this book, only the overbar is used. A **literal** is a variable or the complement of a variable.

### Boolean Addition

Recall from Chapter 3 that **Boolean addition** is equivalent to the OR operation. The basic rules are illustrated with their relation to the OR gate in Figure 4-1.

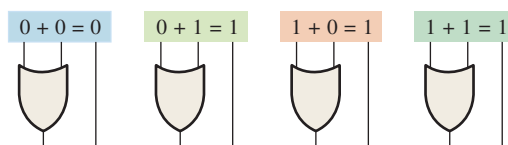


FIGURE 4-1

In Boolean algebra, a **sum term** is a sum of literals. In logic circuits, a sum term is produced by an OR operation with no AND operations involved. Some examples of sum terms are  $A + B$ ,  $A + \bar{B}$ ,  $A + B + \bar{C}$ , and  $\bar{A} + B + C + \bar{D}$ .

A sum term is equal to 1 when one or more of the literals in the term are 1. A sum term is equal to 0 only if each of the literals is 0.

### EXAMPLE 4-1

Determine the values of  $A$ ,  $B$ ,  $C$ , and  $D$  that make the sum term  $A + \bar{B} + C + \bar{D}$  equal to 0.

#### Solution

For the sum term to be 0, each of the literals in the term must be 0. Therefore,  $A = 0$ ,  $B = 1$  so that  $\bar{B} = 0$ ,  $C = 0$ , and  $D = 1$  so that  $\bar{D} = 0$ .

$$A + \bar{B} + C + \bar{D} = 0 + \bar{1} + 0 + \bar{1} = 0 + 0 + 0 + 0 = 0$$

The OR operation is the Boolean equivalent of addition.

**Related Problem\***

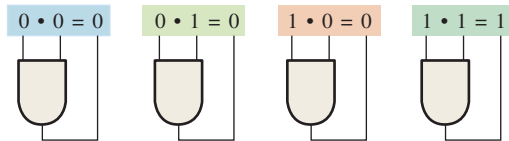
Determine the values of  $A$  and  $B$  that make the sum term  $\bar{A} + B$  equal to 0.

\*Answers are at the end of the chapter.

**Boolean Multiplication**

Also recall from Chapter 3 that **Boolean multiplication** is equivalent to the AND operation. The basic rules are illustrated with their relation to the AND gate in Figure 4–2.

The AND operation is the Boolean equivalent of multiplication.



**FIGURE 4-2**

In Boolean algebra, a **product term** is the product of literals. In logic circuits, a product term is produced by an AND operation with no OR operations involved. Some examples of product terms are  $AB$ ,  $A\bar{B}$ ,  $ABC$ , and  $\bar{A}\bar{B}\bar{C}\bar{D}$ .

A product term is equal to 1 only if each of the literals in the term is 1. A product term is equal to 0 when one or more of the literals are 0.

**EXAMPLE 4-2**

Determine the values of  $A$ ,  $B$ ,  $C$ , and  $D$  that make the product term  $\bar{A}\bar{B}C\bar{D}$  equal to 1.

**Solution**

For the product term to be 1, each of the literals in the term must be 1. Therefore,  $A = 1$ ,  $B = 0$  so that  $\bar{B} = 1$ ,  $C = 1$ , and  $D = 0$  so that  $\bar{D} = 1$ .

$$\bar{A}\bar{B}C\bar{D} = 1 \cdot \bar{0} \cdot 1 \cdot \bar{0} = 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

**Related Problem**

Determine the values of  $A$  and  $B$  that make the product term  $\bar{A}\bar{B}$  equal to 1.

**SECTION 4-1 CHECKUP**

Answers are at the end of the chapter.

1. If  $A = 0$ , what does  $\bar{A}$  equal?
2. Determine the values of  $A$ ,  $B$ , and  $C$  that make the sum term  $\bar{A} + \bar{B} + C$  equal to 0.
3. Determine the values of  $A$ ,  $B$ , and  $C$  that make the product term  $\bar{A}\bar{B}C$  equal to 1.

**4-2 Laws and Rules of Boolean Algebra**

As in other areas of mathematics, there are certain well-developed rules and laws that must be followed in order to properly apply Boolean algebra. The most important of these are presented in this section.

After completing this section, you should be able to

- ♦ Apply the commutative laws of addition and multiplication
- ♦ Apply the associative laws of addition and multiplication
- ♦ Apply the distributive law
- ♦ Apply twelve basic rules of Boolean algebra

## Laws of Boolean Algebra

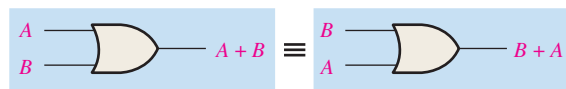
The basic laws of Boolean algebra—the **commutative laws** for addition and multiplication, the **associative laws** for addition and multiplication, and the **distributive law**—are the same as in ordinary algebra. Each of the laws is illustrated with two or three variables, but the number of variables is not limited to this.

### Commutative Laws

The *commutative law of addition* for two variables is written as

$$A + B = B + A \quad \text{Equation 4-1}$$

This law states that the order in which the variables are ORed makes no difference. Remember, in Boolean algebra as applied to logic circuits, addition and the OR operation are the same. Figure 4-3 illustrates the commutative law as applied to the OR gate and shows that it doesn't matter to which input each variable is applied. (The symbol  $\equiv$  means “equivalent to.”)

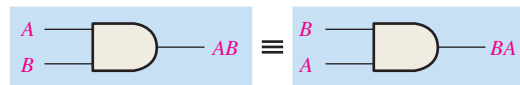


**FIGURE 4-3** Application of commutative law of addition.

The *commutative law of multiplication* for two variables is

$$AB = BA \quad \text{Equation 4-2}$$

This law states that the order in which the variables are ANDed makes no difference. Remember, in Boolean algebra as applied to logic circuits, multiplication and the AND function are the same. Figure 4-4 illustrates this law as applied to the AND gate.



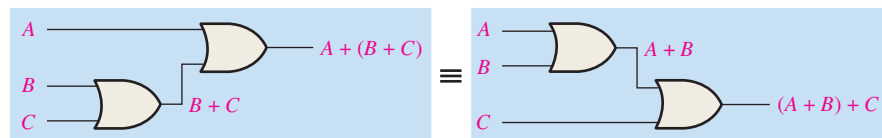
**FIGURE 4-4** Application of commutative law of multiplication.

### Associative Laws

The *associative law of addition* is written as follows for three variables:

$$A + (B + C) = (A + B) + C \quad \text{Equation 4-3}$$

This law states that when ORing more than two variables, the result is the same regardless of the grouping of the variables. Figure 4-5 illustrates this law as applied to 2-input OR gates.

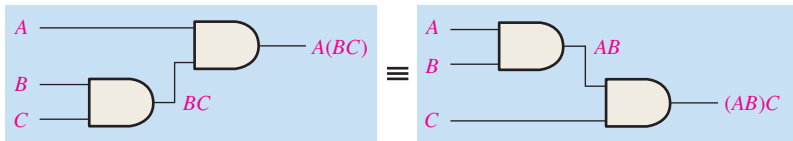


**FIGURE 4-5** Application of associative law of addition. Open file F04-05 to verify. A Multisim tutorial is available on the website.

The *associative law of multiplication* is written as follows for three variables:

$$A(BC) = (AB)C \quad \text{Equation 4-4}$$

This law states that it makes no difference in what order the variables are grouped when AND-ing more than two variables. Figure 4-6 illustrates this law as applied to 2-input AND gates.



**FIGURE 4-6** Application of associative law of multiplication. Open file F04-06 to verify.

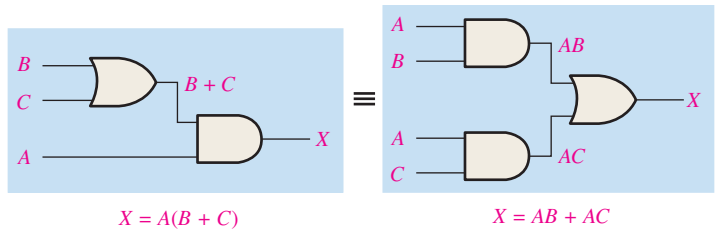


### Distributive Law

The distributive law is written for three variables as follows:

$$A(B + C) = AB + AC \quad \text{Equation 4-5}$$

This law states that ORing two or more variables and then ANDing the result with a single variable is equivalent to ANDing the single variable with each of the two or more variables and then ORing the products. The distributive law also expresses the process of *factoring* in which the common variable  $A$  is factored out of the product terms, for example,  $AB + AC = A(B + C)$ . Figure 4-7 illustrates the distributive law in terms of gate implementation.



**FIGURE 4-7** Application of distributive law. Open file F04-07 to verify.



### Rules of Boolean Algebra

Table 4-1 lists 12 basic rules that are useful in manipulating and simplifying **Boolean expressions**. Rules 1 through 9 will be viewed in terms of their application to logic gates. Rules 10 through 12 will be derived in terms of the simpler rules and the laws previously discussed.

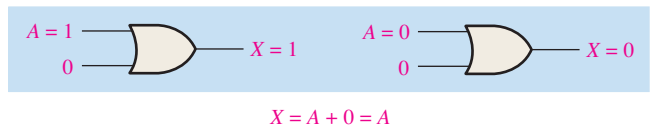
**TABLE 4-1**

Basic rules of Boolean algebra.

|                      |                               |
|----------------------|-------------------------------|
| 1. $A + 0 = A$       | 7. $A \cdot A = A$            |
| 2. $A + 1 = 1$       | 8. $A \cdot \bar{A} = 0$      |
| 3. $A \cdot 0 = 0$   | 9. $\bar{\bar{A}} = A$        |
| 4. $A \cdot 1 = A$   | 10. $A + AB = A$              |
| 5. $A + A = A$       | 11. $A + \bar{A}B = A + B$    |
| 6. $A + \bar{A} = 1$ | 12. $(A + B)(A + C) = A + BC$ |

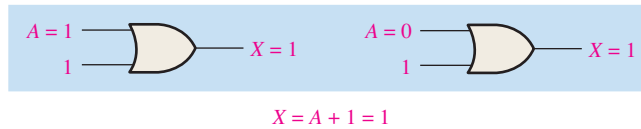
$A$ ,  $B$ , or  $C$  can represent a single variable or a combination of variables.

**Rule 1:  $A + 0 = A$**  A variable ORed with 0 is always equal to the variable. If the input variable  $A$  is 1, the output variable  $X$  is 1, which is equal to  $A$ . If  $A$  is 0, the output is 0, which is also equal to  $A$ . This rule is illustrated in Figure 4-8, where the lower input is fixed at 0.



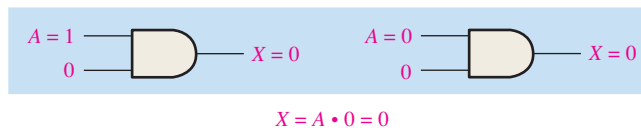
**FIGURE 4-8**

**Rule 2:  $A + 1 = 1$**  A variable ORed with 1 is always equal to 1. A 1 on an input to an OR gate produces a 1 on the output, regardless of the value of the variable on the other input. This rule is illustrated in Figure 4-9, where the lower input is fixed at 1.



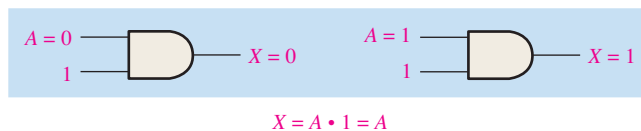
**FIGURE 4-9**

**Rule 3:  $A \cdot 0 = 0$**  A variable ANDed with 0 is always equal to 0. Any time one input to an AND gate is 0, the output is 0, regardless of the value of the variable on the other input. This rule is illustrated in Figure 4-10, where the lower input is fixed at 0.



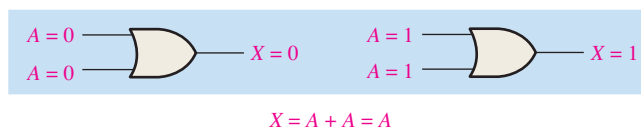
**FIGURE 4-10**

**Rule 4:  $A \cdot 1 = A$**  A variable ANDed with 1 is always equal to the variable. If A is 0, the output of the AND gate is 0. If A is 1, the output of the AND gate is 1 because both inputs are now 1s. This rule is shown in Figure 4-11, where the lower input is fixed at 1.



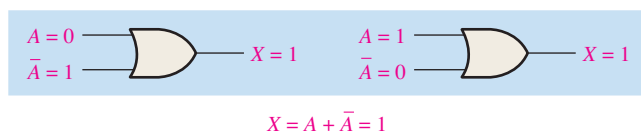
**FIGURE 4-11**

**Rule 5:  $A + A = A$**  A variable ORed with itself is always equal to the variable. If A is 0, then  $0 + 0 = 0$ ; and if A is 1, then  $1 + 1 = 1$ . This is shown in Figure 4-12, where both inputs are the same variable.



**FIGURE 4-12**

**Rule 6:  $A + \bar{A} = 1$**  A variable ORed with its complement is always equal to 1. If A is 0, then  $0 + \bar{0} = 0 + 1 = 1$ . If A is 1, then  $1 + \bar{1} = 1 + 0 = 1$ . See Figure 4-13, where one input is the complement of the other.



**FIGURE 4-13**

**Rule 7:  $A \cdot A = A$**  A variable ANDed with itself is always equal to the variable. If  $A = 0$ , then  $0 \cdot 0 = 0$ ; and if  $A = 1$ , then  $1 \cdot 1 = 1$ . Figure 4-14 illustrates this rule.

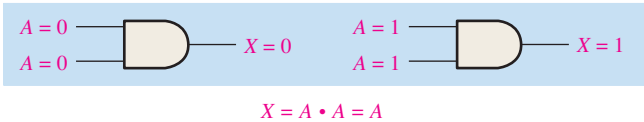


FIGURE 4-14

**Rule 8:  $A \cdot \bar{A} = 0$**  A variable ANDed with its complement is always equal to 0. Either  $A$  or  $\bar{A}$  will always be 0; and when a 0 is applied to the input of an AND gate, the output will be 0 also. Figure 4-15 illustrates this rule.

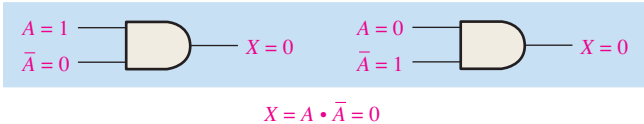


FIGURE 4-15

**Rule 9:  $\bar{\bar{A}} = A$**  The double complement of a variable is always equal to the variable. If you start with the variable  $A$  and complement (invert) it once, you get  $\bar{A}$ . If you then take  $\bar{A}$  and complement (invert) it, you get  $A$ , which is the original variable. This rule is shown in Figure 4-16 using inverters.

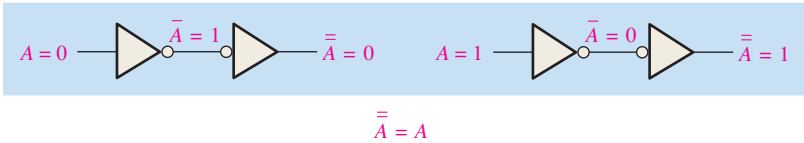


FIGURE 4-16

**Rule 10:  $A + AB = A$**  This rule can be proved by applying the distributive law, rule 2, and rule 4 as follows:

$$\begin{aligned} A + AB &= A \cdot 1 + AB = A(1 + B) && \text{Factoring (distributive law)} \\ &= A \cdot 1 && \text{Rule 2: } (1 + B) = 1 \\ &= A && \text{Rule 4: } A \cdot 1 = A \end{aligned}$$

The proof is shown in Table 4-2, which shows the truth table and the resulting logic circuit simplification.

TABLE 4-2

Rule 10:  $A + AB = A$ . Open file T04-02 to verify.

| A | B | AB | A + AB |
|---|---|----|--------|
| 0 | 0 | 0  | 0      |
| 0 | 1 | 0  | 0      |
| 1 | 0 | 0  | 1      |
| 1 | 1 | 1  | 1      |

↑

equal

↑

straight connection



**Rule 11:  $A + \bar{A}B = A + B$**  This rule can be proved as follows:

$$\begin{aligned}
 A + \bar{A}B &= (A + AB) + \bar{A}B && \text{Rule 10: } A = A + AB \\
 &= (AA + AB) + \bar{A}B && \text{Rule 7: } A = AA \\
 &= AA + AB + \bar{A}A + \bar{A}B && \text{Rule 8: adding } A\bar{A} = 0 \\
 &= (A + \bar{A})(A + B) && \text{Factoring} \\
 &= 1 \cdot (A + B) && \text{Rule 6: } A + \bar{A} = 1 \\
 &= A + B && \text{Rule 4: drop the 1}
 \end{aligned}$$

The proof is shown in Table 4-3, which shows the truth table and the resulting logic circuit simplification.

MultiSim

**TABLE 4-3**

Rule 11:  $A + \bar{A}B = A + B$ . Open file T04-03 to verify.

| A | B | $\bar{A}B$ | $A + \bar{A}B$ | $A + B$ |
|---|---|------------|----------------|---------|
| 0 | 0 | 0          | 0              | 0       |
| 0 | 1 | 1          | 1              | 1       |
| 1 | 0 | 0          | 1              | 1       |
| 1 | 1 | 0          | 1              | 1       |

↑ equal ↑

**Rule 12:  $(A + B)(A + C) = A + BC$**  This rule can be proved as follows:

$$\begin{aligned}
 (A + B)(A + C) &= AA + AC + AB + BC && \text{Distributive law} \\
 &= A + AC + AB + BC && \text{Rule 7: } AA = A \\
 &= A(1 + C) + AB + BC && \text{Factoring (distributive law)} \\
 &= A \cdot 1 + AB + BC && \text{Rule 2: } 1 + C = 1 \\
 &= A(1 + B) + BC && \text{Factoring (distributive law)} \\
 &= A \cdot 1 + BC && \text{Rule 2: } 1 + B = 1 \\
 &= A + BC && \text{Rule 4: } A \cdot 1 = A
 \end{aligned}$$

The proof is shown in Table 4-4, which shows the truth table and the resulting logic circuit simplification.

MultiSim

**TABLE 4-4**

Rule 12:  $(A + B)(A + C) = A + BC$ . Open file T04-04 to verify.

| A | B | C | $A + B$ | $A + C$ | $(A + B)(A + C)$ | $BC$ | $A + BC$ |
|---|---|---|---------|---------|------------------|------|----------|
| 0 | 0 | 0 | 0       | 0       | 0                | 0    | 0        |
| 0 | 0 | 1 | 0       | 1       | 0                | 0    | 0        |
| 0 | 1 | 0 | 1       | 0       | 0                | 0    | 0        |
| 0 | 1 | 1 | 1       | 1       | 1                | 1    | 1        |
| 1 | 0 | 0 | 1       | 1       | 1                | 0    | 1        |
| 1 | 0 | 1 | 1       | 1       | 1                | 0    | 1        |
| 1 | 1 | 0 | 1       | 1       | 1                | 0    | 1        |
| 1 | 1 | 1 | 1       | 1       | 1                | 1    | 1        |

↑ equal ↑

**SECTION 4-2 CHECKUP**

1. Apply the associative law of addition to the expression  $A + (B + C + D)$ .
2. Apply the distributive law to the expression  $A(B + C + D)$ .

**4-3 DeMorgan's Theorems**

DeMorgan, a mathematician who knew Boole, proposed two theorems that are an important part of Boolean algebra. In practical terms, DeMorgan's theorems provide mathematical verification of the equivalency of the NAND and negative-OR gates and the equivalency of the NOR and negative-AND gates, which were discussed in Chapter 3.

After completing this section, you should be able to

- ♦ State DeMorgan's theorems
- ♦ Relate DeMorgan's theorems to the equivalency of the NAND and negative-OR gates and to the equivalency of the NOR and negative-AND gates
- ♦ Apply DeMorgan's theorems to the simplification of Boolean expressions

DeMorgan's first theorem is stated as follows:

**The complement of a product of variables is equal to the sum of the complements of the variables.**

To apply DeMorgan's theorem, break the bar over the product of variables and change the sign from AND to OR.

Stated another way,

**The complement of two or more ANDed variables is equivalent to the OR of the complements of the individual variables.**

The formula for expressing this theorem for two variables is

$$\overline{XY} = \bar{X} + \bar{Y} \quad \text{Equation 4-6}$$

DeMorgan's second theorem is stated as follows:

**The complement of a sum of variables is equal to the product of the complements of the variables.**

Stated another way,

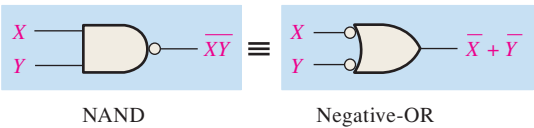
**The complement of two or more ORed variables is equivalent to the AND of the complements of the individual variables.**

The formula for expressing this theorem for two variables is

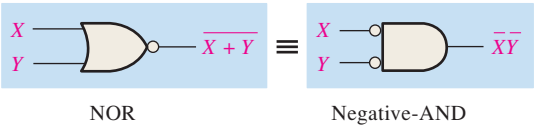
$$\overline{\bar{X} + \bar{Y}} = \bar{X}\bar{Y} \quad \text{Equation 4-7}$$

Figure 4-17 shows the gate equivalencies and truth tables for Equations 4-6 and 4-7.

As stated, DeMorgan's theorems also apply to expressions in which there are more than two variables. The following examples illustrate the application of DeMorgan's theorems to 3-variable and 4-variable expressions.



| Inputs |   | Output          |                    |
|--------|---|-----------------|--------------------|
| X      | Y | $\overline{XY}$ | $\overline{X + Y}$ |
| 0      | 0 | 1               | 1                  |
| 0      | 1 | 1               | 1                  |
| 1      | 0 | 1               | 1                  |
| 1      | 1 | 0               | 0                  |



| Inputs |   | Output             |                 |
|--------|---|--------------------|-----------------|
| X      | Y | $\overline{X + Y}$ | $\overline{XY}$ |
| 0      | 0 | 1                  | 1               |
| 0      | 1 | 0                  | 0               |
| 1      | 0 | 0                  | 0               |
| 1      | 1 | 0                  | 0               |

**FIGURE 4-17** Gate equivalencies and the corresponding truth tables that illustrate DeMorgan's theorems. Notice the equality of the two output columns in each table. This shows that the equivalent gates perform the same logic function.

**EXAMPLE 4-3**

Apply DeMorgan's theorems to the expressions  $\overline{XYZ}$  and  $\overline{X + Y + Z}$ .

**Solution**

$$\begin{aligned}\overline{XYZ} &= \overline{X} + \overline{Y} + \overline{Z} \\ \overline{X + Y + Z} &= \overline{X} \overline{Y} \overline{Z}\end{aligned}$$

**Related Problem**

Apply DeMorgan's theorem to the expression  $\overline{\overline{X} + \overline{Y} + \overline{Z}}$ .

**EXAMPLE 4-4**

Apply DeMorgan's theorems to the expressions  $\overline{WXYZ}$  and  $\overline{W + X + Y + Z}$ .

**Solution**

$$\begin{aligned}\overline{WXYZ} &= \overline{W} + \overline{X} + \overline{Y} + \overline{Z} \\ \overline{W + X + Y + Z} &= \overline{W} \overline{X} \overline{Y} \overline{Z}\end{aligned}$$

**Related Problem**

Apply DeMorgan's theorem to the expression  $\overline{\overline{W} \overline{X} \overline{Y} \overline{Z}}$ .

Each variable in DeMorgan's theorems as stated in Equations 4-6 and 4-7 can also represent a combination of other variables. For example,  $X$  can be equal to the term  $AB + C$ , and  $Y$  can be equal to the term  $A + BC$ . So if you can apply DeMorgan's theorem for two variables as stated by  $\overline{XY} = \overline{X} + \overline{Y}$  to the expression  $\overline{(AB + C)(A + BC)}$ , you get the following result:

$$\overline{(AB + C)(A + BC)} = \overline{(AB + C)} + \overline{(A + BC)}$$

Notice that in the preceding result you have two terms,  $\overline{AB + C}$  and  $\overline{A + BC}$ , to each of which you can again apply DeMorgan's theorem  $\overline{X + Y} = \overline{X} \overline{Y}$  individually, as follows:

$$\overline{(AB + C)} + \overline{(A + BC)} = (\overline{AB})\overline{C} + \overline{A}(\overline{BC})$$

Notice that you still have two terms in the expression to which DeMorgan's theorem can again be applied. These terms are  $\overline{AB}$  and  $\overline{BC}$ . A final application of DeMorgan's theorem gives the following result:

$$(\overline{AB})\overline{C} + \overline{A}(\overline{BC}) = (\overline{A} + \overline{B})\overline{C} + \overline{A}(\overline{B} + \overline{C})$$

Although this result can be simplified further by the use of Boolean rules and laws, DeMorgan's theorems cannot be used any more.

## Applying DeMorgan's Theorems

The following procedure illustrates the application of DeMorgan's theorems and Boolean algebra to the specific expression

$$\overline{\overline{A + BC} + D(E + \overline{F})}$$

**Step 1:** Identify the terms to which you can apply DeMorgan's theorems, and think of each term as a single variable. Let  $\overline{A + BC} = X$  and  $\overline{D(E + \overline{F})} = Y$ .

**Step 2:** Since  $\overline{X + Y} = \overline{X}\overline{Y}$ ,

$$\overline{(\overline{A + BC}) + (\overline{D(E + \overline{F})})} = \overline{\overline{A + BC}} \cdot \overline{\overline{D(E + \overline{F})}}$$

**Step 3:** Use rule 9 ( $\overline{\overline{A}} = A$ ) to cancel the double bars over the left term (this is not part of DeMorgan's theorem).

$$\overline{(\overline{A + BC})(\overline{D(E + \overline{F})})} = (A + BC)(\overline{D(E + \overline{F})})$$

**Step 4:** Apply DeMorgan's theorem to the second term.

$$(A + BC)(\overline{D(E + \overline{F})}) = (A + BC)(\overline{D} + \overline{E + \overline{F}})$$

**Step 5:** Use rule 9 ( $\overline{\overline{A}} = A$ ) to cancel the double bars over the  $E + \overline{F}$  part of the term.

$$(A + BC)(\overline{D} + \overline{E + \overline{F}}) = (A + BC)(\overline{D} + E + \overline{F})$$

The following three examples will further illustrate how to use DeMorgan's theorems.

### EXAMPLE 4-5

Apply DeMorgan's theorems to each of the following expressions:

(a)  $\overline{(A + B + C)D}$

(b)  $\overline{ABC + DEF}$

(c)  $\overline{AB + \overline{CD} + EF}$

#### Solution

(a) Let  $A + B + C = X$  and  $D = Y$ . The expression  $\overline{(A + B + C)D}$  is of the form  $\overline{XY} = \overline{X} + \overline{Y}$  and can be rewritten as

$$\overline{(A + B + C)D} = \overline{A + B + C} + \overline{D}$$

Next, apply DeMorgan's theorem to the term  $\overline{A + B + C}$ .

$$\overline{A + B + C} + \overline{D} = \overline{A}\overline{B}\overline{C} + \overline{D}$$

(b) Let  $ABC = X$  and  $DEF = Y$ . The expression  $\overline{ABC + DEF}$  is of the form  $\overline{X + Y} = \overline{X}\overline{Y}$  and can be rewritten as

$$\overline{ABC + DEF} = (\overline{ABC})(\overline{DEF})$$

Next, apply DeMorgan's theorem to each of the terms  $\overline{ABC}$  and  $\overline{DEF}$ .

$$(\overline{ABC})(\overline{DEF}) = (\overline{A} + \overline{B} + \overline{C})(\overline{D} + \overline{E} + \overline{F})$$

- (c) Let  $\overline{AB} = X$ ,  $\overline{CD} = Y$ , and  $EF = Z$ . The expression  $\overline{\overline{AB} + \overline{CD} + EF}$  is of the form  $\overline{X + Y + Z} = \overline{XYZ}$  and can be rewritten as

$$\overline{\overline{AB} + \overline{CD} + EF} = (\overline{\overline{AB}})(\overline{\overline{CD}})(\overline{EF})$$

Next, apply DeMorgan's theorem to each of the terms  $\overline{\overline{AB}}$ ,  $\overline{\overline{CD}}$ , and  $\overline{EF}$ .

$$(\overline{\overline{AB}})(\overline{\overline{CD}})(\overline{EF}) = (\overline{A} + B)(C + \overline{D})(\overline{E} + \overline{F})$$

#### Related Problem

Apply DeMorgan's theorems to the expression  $\overline{\overline{ABC} + D + E}$ .

#### EXAMPLE 4-6

Apply DeMorgan's theorems to each expression:

- (a)  $\overline{(A + B) + \overline{C}}$   
 (b)  $\overline{(\overline{A} + B) + CD}$   
 (c)  $\overline{(A + B)\overline{CD} + E + \overline{F}}$

#### Solution

- (a)  $\overline{(A + B) + \overline{C}} = \overline{(A + B)}\overline{\overline{C}} = (A + B)C$   
 (b)  $\overline{(\overline{A} + B) + CD} = \overline{(\overline{A} + B)}\overline{CD} = (\overline{\overline{A}}\overline{B})(\overline{C} + \overline{D}) = A\overline{B}(\overline{C} + \overline{D})$   
 (c)  $\overline{(A + B)\overline{CD} + E + \overline{F}} = \overline{((A + B)\overline{CD})(E + \overline{F})} = (\overline{A + B} + C + D)\overline{EF}$

#### Related Problem

Apply DeMorgan's theorems to the expression  $\overline{\overline{AB}(C + \overline{D}) + E}$ .

#### EXAMPLE 4-7

The Boolean expression for an exclusive-OR gate is  $A\overline{B} + \overline{A}B$ . With this as a starting point, use DeMorgan's theorems and any other rules or laws that are applicable to develop an expression for the exclusive-NOR gate.

#### Solution

Start by complementing the exclusive-OR expression and then applying DeMorgan's theorems as follows:

$$\overline{A\overline{B} + \overline{A}B} = (\overline{A\overline{B}})(\overline{\overline{A}B}) = (\overline{A} + \overline{\overline{B}})(\overline{\overline{A}} + \overline{B}) = (\overline{A} + B)(A + \overline{B})$$

Next, apply the distributive law and rule 8 ( $A \cdot \overline{A} = 0$ ).

$$(\overline{A} + B)(A + \overline{B}) = \overline{A}A + \overline{A}\overline{B} + AB + B\overline{B} = \overline{A}\overline{B} + AB$$

The final expression for the XNOR is  $\overline{A}\overline{B} + AB$ . Note that this expression equals 1 any time both variables are 0s or both variables are 1s.

#### Related Problem

Starting with the expression for a 4-input NAND gate, use DeMorgan's theorems to develop an expression for a 4-input negative-OR gate.

**SECTION 4-3 CHECKUP**

1. Apply DeMorgan's theorems to the following expressions:

(a)  $\overline{ABC} + (\overline{D} + E)$       (b)  $\overline{(A + B)C}$       (c)  $\overline{A + B + C} + \overline{DE}$

**4-4 Boolean Analysis of Logic Circuits**

Boolean algebra provides a concise way to express the operation of a logic circuit formed by a combination of logic gates so that the output can be determined for various combinations of input values.

After completing this section, you should be able to

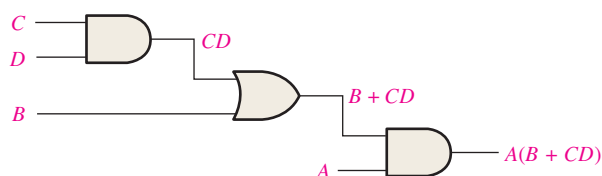
- ♦ Determine the Boolean expression for a combination of gates
- ♦ Evaluate the logic operation of a circuit from the Boolean expression
- ♦ Construct a truth table

**Boolean Expression for a Logic Circuit**

To derive the Boolean expression for a given combinational logic circuit, begin at the left-most inputs and work toward the final output, writing the expression for each gate. For the example circuit in Figure 4-18, the Boolean expression is determined in the following three steps:

A combinational logic circuit can be described by a Boolean equation.

1. The expression for the left-most AND gate with inputs  $C$  and  $D$  is  $CD$ .
2. The output of the left-most AND gate is one of the inputs to the OR gate and  $B$  is the other input. Therefore, the expression for the OR gate is  $B + CD$ .
3. The output of the OR gate is one of the inputs to the right-most AND gate and  $A$  is the other input. Therefore, the expression for this AND gate is  $A(B + CD)$ , which is the final output expression for the entire circuit.



**FIGURE 4-18** A combinational logic circuit showing the development of the Boolean expression for the output.

**Constructing a Truth Table for a Logic Circuit**

Once the Boolean expression for a given logic circuit has been determined, a truth table that shows the output for all possible values of the input variables can be developed. The procedure requires that you evaluate the Boolean expression for all possible combinations of values for the input variables. In the case of the circuit in Figure 4-18, there are four input variables ( $A$ ,  $B$ ,  $C$ , and  $D$ ) and therefore sixteen ( $2^4 = 16$ ) combinations of values are possible.

A combinational logic circuit can be described by a truth table.

**Evaluating the Expression**

To evaluate the expression  $A(B + CD)$ , first find the values of the variables that make the expression equal to 1, using the rules for Boolean addition and multiplication. In this case, the expression equals 1 only if  $A = 1$  and  $B + CD = 1$  because

$$A(B + CD) = 1 \cdot 1 = 1$$

Now determine when the  $B + CD$  term equals 1. The term  $B + CD = 1$  if either  $B = 1$  or  $CD = 1$  or if both  $B$  and  $CD$  equal 1 because

$$B + CD = 1 + 0 = 1$$

$$B + CD = 0 + 1 = 1$$

$$B + CD = 1 + 1 = 1$$

The term  $CD = 1$  only if  $C = 1$  and  $D = 1$ .

To summarize, the expression  $A(B + CD) = 1$  when  $A = 1$  and  $B = 1$  regardless of the values of  $C$  and  $D$  or when  $A = 1$  and  $C = 1$  and  $D = 1$  regardless of the value of  $B$ . The expression  $A(B + CD) = 0$  for all other value combinations of the variables.

### Putting the Results in Truth Table Format

The first step is to list the sixteen input variable combinations of 1s and 0s in a binary sequence as shown in Table 4-5. Next, place a 1 in the output column for each combination of input variables that was determined in the evaluation. Finally, place a 0 in the output column for all other combinations of input variables. These results are shown in the truth table in Table 4-5.

**TABLE 4-5**

Truth table for the logic circuit in Figure 4-18.

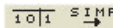
| Inputs |     |     |     | Output      |
|--------|-----|-----|-----|-------------|
| $A$    | $B$ | $C$ | $D$ | $A(B + CD)$ |
| 0      | 0   | 0   | 0   | 0           |
| 0      | 0   | 0   | 1   | 0           |
| 0      | 0   | 1   | 0   | 0           |
| 0      | 0   | 1   | 1   | 0           |
| 0      | 1   | 0   | 0   | 0           |
| 0      | 1   | 0   | 1   | 0           |
| 0      | 1   | 1   | 0   | 0           |
| 0      | 1   | 1   | 1   | 0           |
| 1      | 0   | 0   | 0   | 0           |
| 1      | 0   | 0   | 1   | 0           |
| 1      | 0   | 1   | 0   | 0           |
| 1      | 0   | 1   | 1   | 1           |
| 1      | 1   | 0   | 0   | 1           |
| 1      | 1   | 0   | 1   | 1           |
| 1      | 1   | 1   | 0   | 1           |
| 1      | 1   | 1   | 1   | 1           |

### EXAMPLE 4-8

Use Multisim to generate the truth table for the logic circuit in Figure 4-18.

#### Solution

Construct the circuit in Multisim and connect the Multisim Logic Converter to the inputs and output, as shown in Figure 4-19. Click on the  conversion bar, and the truth table appears in the display as shown.

You can also generate the simplified Boolean expression from the truth table by clicking on   $A|B$ .

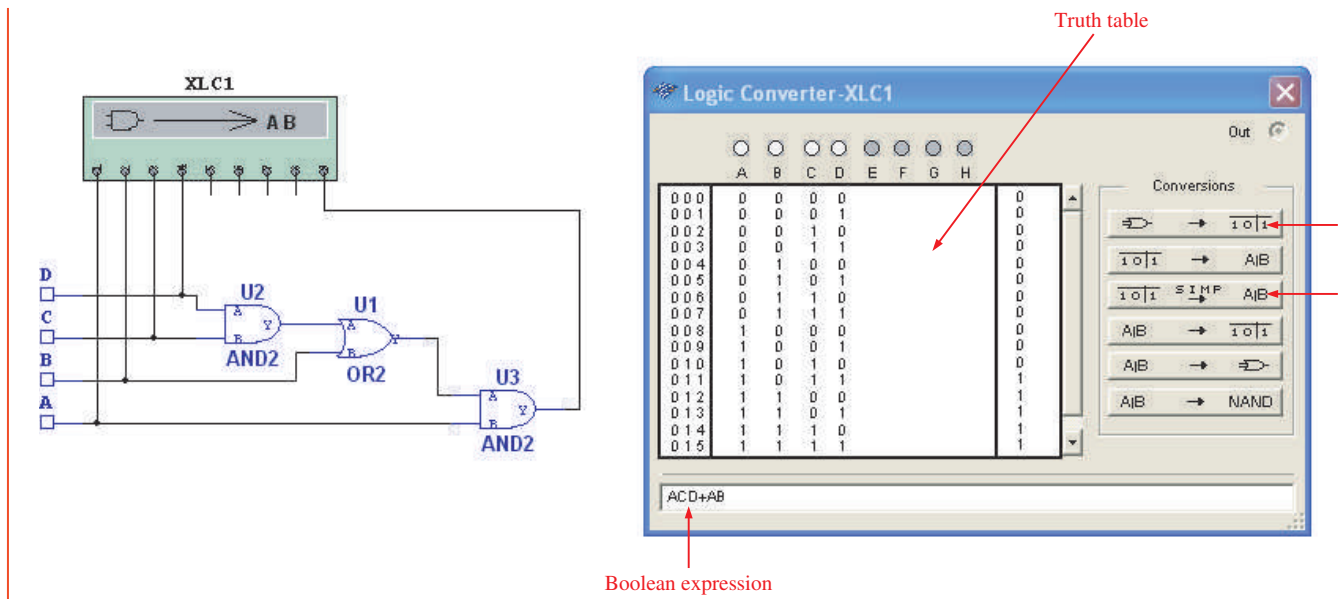


FIGURE 4-19

### Related Problem

Open Multisim. Create the setup and do the conversions shown in this example.



## SECTION 4-4 CHECKUP

1. Replace the AND gates with OR gates and the OR gate with an AND gate in Figure 4-18. Determine the Boolean expression for the output.
2. Construct a truth table for the circuit in Question 1.

## 4-5 Logic Simplification Using Boolean Algebra

A logic expression can be reduced to its simplest form or changed to a more convenient form to implement the expression most efficiently using Boolean algebra. The approach taken in this section is to use the basic laws, rules, and theorems of Boolean algebra to manipulate and simplify an expression. This method depends on a thorough knowledge of Boolean algebra and considerable practice in its application, not to mention a little ingenuity and cleverness.

After completing this section, you should be able to

- ♦ Apply the laws, rules, and theorems of Boolean algebra to simplify general expressions

A simplified Boolean expression uses the fewest gates possible to implement a given expression. Examples 4-9 through 4-12 illustrate Boolean simplification.

### EXAMPLE 4-9

Using Boolean algebra techniques, simplify this expression:

$$AB + A(B + C) + B(B + C)$$

### Solution

The following is not necessarily the only approach.

**Step 1:** Apply the distributive law to the second and third terms in the expression, as follows:

$$AB + AB + AC + BB + BC$$

**Step 2:** Apply rule 7 ( $BB = B$ ) to the fourth term.

$$AB + AB + AC + B + BC$$

**Step 3:** Apply rule 5 ( $AB + AB = AB$ ) to the first two terms.

$$AB + AC + B + BC$$

**Step 4:** Apply rule 10 ( $B + BC = B$ ) to the last two terms.

$$AB + AC + B$$

**Step 5:** Apply rule 10 ( $AB + B = B$ ) to the first and third terms.

$$B + AC$$

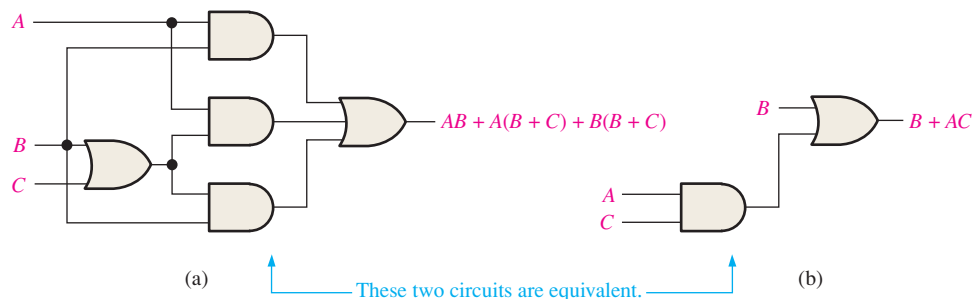
At this point the expression is simplified as much as possible. Once you gain experience in applying Boolean algebra, you can often combine many individual steps.

### Related Problem

Simplify the Boolean expression  $A\bar{B} + A(\bar{B} + \bar{C}) + B(\bar{B} + \bar{C})$ .

Simplification means fewer gates for the same function.

Figure 4–20 shows that the simplification process in Example 4–9 has significantly reduced the number of logic gates required to implement the expression. Part (a) shows that five gates are required to implement the expression in its original form; however, only two gates are needed for the simplified expression, shown in part (b). It is important to realize that these two gate circuits are equivalent. That is, for any combination of levels on the  $A$ ,  $B$ , and  $C$  inputs, you get the same output from either circuit.



**FIGURE 4-20** Gate circuits for Example 4–9. Open file F04-20 to verify equivalency.

### EXAMPLE 4-10

Simplify the following Boolean expression:

$$[A\bar{B}(C + BD) + \bar{A}\bar{B}]C$$

Note that brackets and parentheses mean the same thing: the term inside is multiplied (ANDed) with the term outside.

**Solution**

**Step 1:** Apply the distributive law to the terms within the brackets.

$$(\overline{A}\overline{B}C + \overline{A}B\overline{B}D + \overline{A}\overline{B})C$$

**Step 2:** Apply rule 8 ( $\overline{B}B = 0$ ) to the second term within the parentheses.

$$(\overline{A}\overline{B}C + A \cdot 0 \cdot D + \overline{A}\overline{B})C$$

**Step 3:** Apply rule 3 ( $A \cdot 0 \cdot D = 0$ ) to the second term within the parentheses.

$$(\overline{A}\overline{B}C + 0 + \overline{A}\overline{B})C$$

**Step 4:** Apply rule 1 (drop the 0) within the parentheses.

$$(\overline{A}\overline{B}C + \overline{A}\overline{B})C$$

**Step 5:** Apply the distributive law.

$$\overline{A}\overline{B}CC + \overline{A}\overline{B}C$$

**Step 6:** Apply rule 7 ( $CC = C$ ) to the first term.

$$\overline{A}\overline{B}C + \overline{A}\overline{B}C$$

**Step 7:** Factor out  $\overline{B}C$ .

$$\overline{B}C(A + \overline{A})$$

**Step 8:** Apply rule 6 ( $A + \overline{A} = 1$ ).

$$\overline{B}C \cdot 1$$

**Step 9:** Apply rule 4 (drop the 1).

$$\overline{B}C$$

**Related Problem**

Simplify the Boolean expression  $[AB(C + \overline{B}\overline{D}) + \overline{A}\overline{B}]CD$ .

**EXAMPLE 4-11**

Simplify the following Boolean expression:

$$\overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + ABC$$

**Solution**

**Step 1:** Factor  $BC$  out of the first and last terms.

$$BC(\overline{A} + A) + \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C$$

**Step 2:** Apply rule 6 ( $\overline{A} + A = 1$ ) to the term in parentheses, and factor  $\overline{A}\overline{B}$  from the second and last terms.

$$BC \cdot 1 + \overline{A}\overline{B}(\overline{C} + C) + \overline{A}\overline{B}\overline{C}$$

**Step 3:** Apply rule 4 (drop the 1) to the first term and rule 6 ( $\overline{C} + C = 1$ ) to the term in parentheses.

$$BC + \overline{A}\overline{B} \cdot 1 + \overline{A}\overline{B}\overline{C}$$

**Step 4:** Apply rule 4 (drop the 1) to the second term.

$$BC + \overline{A}\overline{B} + \overline{A}\overline{B}\overline{C}$$

**Step 5:** Factor  $\bar{B}$  from the second and third terms.

$$BC + \bar{B}(A + \bar{A}\bar{C})$$

**Step 6:** Apply rule 11 ( $A + \bar{A}\bar{C} = A + \bar{C}$ ) to the term in parentheses.

$$BC + \bar{B}(A + \bar{C})$$

**Step 7:** Use the distributive and commutative laws to get the following expression:

$$BC + A\bar{B} + \bar{B}\bar{C}$$

#### Related Problem

Simplify the Boolean expression  $ABC + \bar{A}\bar{B}C + \bar{A}BC + \bar{A}\bar{B}\bar{C}$ .

#### EXAMPLE 4-12

Simplify the following Boolean expression:

$$\overline{AB} + \overline{AC} + \bar{A}\bar{B}C$$

#### Solution

**Step 1:** Apply DeMorgan's theorem to the first term.

$$(\bar{A}\bar{B})(\bar{A}\bar{C}) + \bar{A}\bar{B}C$$

**Step 2:** Apply DeMorgan's theorem to each term in parentheses.

$$(\bar{A} + \bar{B})(\bar{A} + \bar{C}) + \bar{A}\bar{B}C$$

**Step 3:** Apply the distributive law to the two terms in parentheses.

$$\bar{A}\bar{A} + \bar{A}\bar{C} + \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{A}\bar{B}C$$

**Step 4:** Apply rule 7 ( $\bar{A}\bar{A} = \bar{A}$ ) to the first term, and apply rule 10 [ $\bar{A}\bar{B} + \bar{A}\bar{B}C = \bar{A}\bar{B}(1 + C) = \bar{A}\bar{B}$ ] to the third and last terms.

$$\bar{A} + \bar{A}\bar{C} + \bar{A}\bar{B} + \bar{B}\bar{C}$$

**Step 5:** Apply rule 10 [ $\bar{A} + \bar{A}\bar{C} = \bar{A}(1 + \bar{C}) = \bar{A}$ ] to the first and second terms.

$$\bar{A} + \bar{A}\bar{B} + \bar{B}\bar{C}$$

**Step 6:** Apply rule 10 [ $\bar{A} + \bar{A}\bar{B} = \bar{A}(1 + \bar{B}) = \bar{A}$ ] to the first and second terms.

$$\bar{A} + \bar{B}\bar{C}$$

#### Related Problem

Simplify the Boolean expression  $\bar{A}\bar{B} + \bar{A}\bar{C} + \bar{A}\bar{B}\bar{C}$ .

#### EXAMPLE 4-13

Use Multisim to perform the logic simplification shown in Figure 4-20.

#### Solution

**Step 1:** Connect the Multisim Logic Converter to the circuit as shown in Figure 4-21.

**Step 2:** Generate the truth table by clicking on  .

**Step 3:** Generate the simplified Boolean expression by clicking on   .

**Step 4:** Generate the simplified logic circuit by clicking on  .

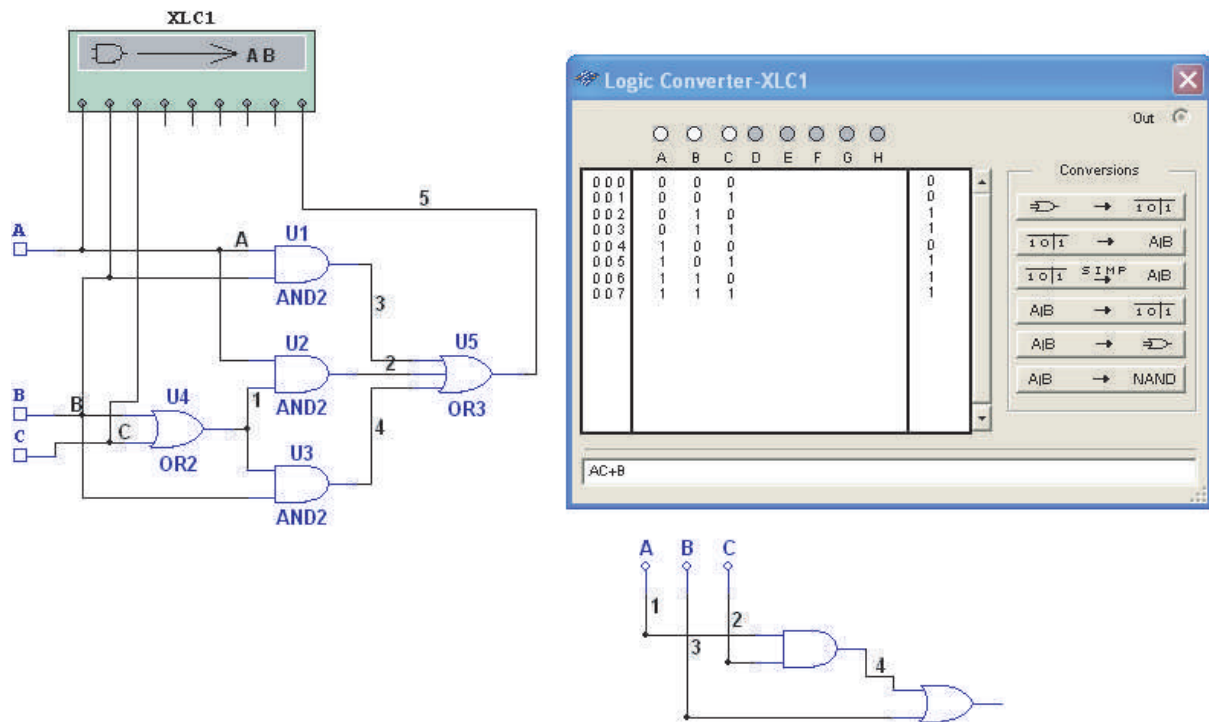


FIGURE 4-21

**Related Problem**

Open Multisim. Create the setup and perform the logic simplification illustrated in this example.

**SECTION 4-5 CHECKUP**

- Simplify the following Boolean expressions:
  - $A + AB + A\bar{B}C$
  - $(\bar{A} + B)C + ABC$
  - $A\bar{B}C(BD + CDE) + A\bar{C}$
- Implement each expression in Question 1 as originally stated with the appropriate logic gates. Then implement the simplified expression, and compare the number of gates.

**4-6 Standard Forms of Boolean Expressions**

All Boolean expressions, regardless of their form, can be converted into either of two standard forms: the sum-of-products form or the product-of-sums form. Standardization makes the evaluation, simplification, and implementation of Boolean expressions much more systematic and easier.

After completing this section, you should be able to

- ◆ Identify a sum-of-products expression
- ◆ Determine the domain of a Boolean expression
- ◆ Convert any sum-of-products expression to a standard form
- ◆ Evaluate a standard sum-of-products expression in terms of binary values
- ◆ Identify a product-of-sums expression

- ◆ Convert any product-of-sums expression to a standard form
- ◆ Evaluate a standard product-of-sums expression in terms of binary values
- ◆ Convert from one standard form to the other

## The Sum-of-Products (SOP) Form

An SOP expression can be implemented with one OR gate and two or more AND gates.

A product term was defined in Section 4–1 as a term consisting of the product (Boolean multiplication) of literals (variables or their complements). When two or more product terms are summed by Boolean addition, the resulting expression is a **sum-of-products (SOP)**. Some examples are

$$\begin{aligned} AB + ABC \\ ABC + CDE + \overline{BCD} \\ \overline{AB} + \overline{ABC} + AC \end{aligned}$$

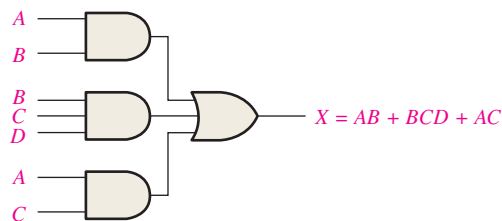
Also, an SOP expression can contain a single-variable term, as in  $A + \overline{A}BC + BCD$ . Refer to the simplification examples in the last section, and you will see that each of the final expressions was either a single product term or in SOP form. In an SOP expression, a single overbar cannot extend over more than one variable; however, more than one variable in a term can have an overbar. For example, an SOP expression can have the term  $\overline{A}\overline{B}\overline{C}$  but not  $\overline{ABC}$ .

## Domain of a Boolean Expression

The **domain** of a general Boolean expression is the set of variables contained in the expression in either complemented or uncomplemented form. For example, the domain of the expression  $\overline{AB} + A\overline{B}C$  is the set of variables  $A, B, C$  and the domain of the expression  $ABC + \overline{CDE} + \overline{BCD}$  is the set of variables  $A, B, C, D, E$ .

## AND/OR Implementation of an SOP Expression

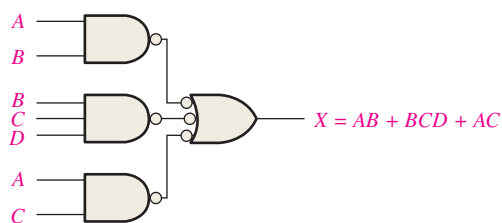
Implementing an SOP expression simply requires ORing the outputs of two or more AND gates. A product term is produced by an AND operation, and the sum (addition) of two or more product terms is produced by an OR operation. Therefore, an SOP expression can be implemented by AND-OR logic in which the outputs of a number (equal to the number of product terms in the expression) of AND gates connect to the inputs of an OR gate, as shown in Figure 4–22 for the expression  $AB + BCD + AC$ . The output  $X$  of the OR gate equals the SOP expression.



**FIGURE 4–22** Implementation of the SOP expression  $AB + BCD + AC$ .

## NAND/NAND Implementation of an SOP Expression

NAND gates can be used to implement an SOP expression. By using only NAND gates, an AND/OR function can be accomplished, as illustrated in Figure 4–23. The first level of NAND gates feed into a NAND gate that acts as a negative-OR gate. The NAND and negative-OR inversions cancel and the result is effectively an AND/OR circuit.



**FIGURE 4-23** This NAND/NAND implementation is equivalent to the AND/OR in Figure 4-22.

## Conversion of a General Expression to SOP Form

Any logic expression can be changed into SOP form by applying Boolean algebra techniques. For example, the expression  $A(B + CD)$  can be converted to SOP form by applying the distributive law:

$$A(B + CD) = AB + ACD$$

### EXAMPLE 4-14

Convert each of the following Boolean expressions to SOP form:

- (a)  $AB + B(CD + EF)$       (b)  $(A + B)(B + C + D)$       (c)  $\overline{\overline{(A + B)} + C}$

#### Solution

(a)  $AB + B(CD + EF) = AB + BCD + BEF$

(b)  $(A + B)(B + C + D) = AB + AC + AD + BB + BC + BD$

(c)  $\overline{\overline{(A + B)} + C} = \overline{\overline{(A + B)}}\overline{C} = (A + B)\overline{C} = A\overline{C} + B\overline{C}$

#### Related Problem

Convert  $\overline{A}B\overline{C} + (A + \overline{B})(B + \overline{C} + A\overline{B})$  to SOP form.

## The Standard SOP Form

So far, you have seen SOP expressions in which some of the product terms do not contain all of the variables in the domain of the expression. For example, the expression  $\overline{A}B\overline{C} + \overline{A}BD + \overline{A}B\overline{C}D$  has a domain made up of the variables  $A$ ,  $B$ ,  $C$ , and  $D$ . However, notice that the complete set of variables in the domain is not represented in the first two terms of the expression; that is,  $D$  or  $\overline{D}$  is missing from the first term and  $C$  or  $\overline{C}$  is missing from the second term.

A *standard SOP expression* is one in which *all* the variables in the domain appear in each product term in the expression. For example,  $\overline{A}BCD + \overline{A}B\overline{C}D + \overline{A}B\overline{C}\overline{D}$  is a standard SOP expression. Standard SOP expressions are important in constructing truth tables, covered in Section 4-7, and in the Karnaugh map simplification method, which is covered in Section 4-8. Any nonstandard SOP expression (referred to simply as SOP) can be converted to the standard form using Boolean algebra.

## Converting Product Terms to Standard SOP

Each product term in an SOP expression that does not contain all the variables in the domain can be expanded to standard form to include all variables in the domain and their complements. As stated in the following steps, a nonstandard SOP expression is converted into standard form using Boolean algebra rule 6 ( $A + \overline{A} = 1$ ) from Table 4-1: A variable added to its complement equals 1.

**Step 1:** Multiply each nonstandard product term by a term made up of the sum of a missing variable and its complement. This results in two product terms. As you know, you can multiply anything by 1 without changing its value.

**Step 2:** Repeat Step 1 until all resulting product terms contain all variables in the domain in either complemented or uncomplemented form. In converting a product term to standard form, the number of product terms is doubled for each missing variable, as Example 4–15 shows.

#### EXAMPLE 4-15

Convert the following Boolean expression into standard SOP form:

$$\overline{A}\overline{B}C + \overline{A}\overline{B} + AB\overline{C}D$$

#### Solution

The domain of this SOP expression is  $A, B, C, D$ . Take one term at a time. The first term,  $\overline{A}\overline{B}C$ , is missing variable  $D$  or  $\overline{D}$ , so multiply the first term by  $D + \overline{D}$  as follows:

$$\overline{A}\overline{B}C = \overline{A}\overline{B}C(D + \overline{D}) = \overline{A}\overline{B}CD + \overline{A}\overline{B}C\overline{D}$$

In this case, two standard product terms are the result.

The second term,  $\overline{A}\overline{B}$ , is missing variables  $C$  or  $\overline{C}$  and  $D$  or  $\overline{D}$ , so first multiply the second term by  $C + \overline{C}$  as follows:

$$\overline{A}\overline{B} = \overline{A}\overline{B}(C + \overline{C}) = \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C}$$

The two resulting terms are missing variable  $D$  or  $\overline{D}$ , so multiply both terms by  $D + \overline{D}$  as follows:

$$\begin{aligned}\overline{A}\overline{B} &= \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} = \overline{A}\overline{B}C(D + \overline{D}) + \overline{A}\overline{B}\overline{C}(D + \overline{D}) \\ &= \overline{A}\overline{B}CD + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}\overline{C}\overline{D}\end{aligned}$$

In this case, four standard product terms are the result.

The third term,  $AB\overline{C}D$ , is already in standard form. The complete standard SOP form of the original expression is as follows:

$$\overline{A}\overline{B}C + \overline{A}\overline{B} + AB\overline{C}D = \overline{A}\overline{B}CD + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}\overline{C}\overline{D} + AB\overline{C}D$$

#### Related Problem

Convert the expression  $W\overline{X}Y + \overline{X}YZ + WXY$  to standard SOP form.

### Binary Representation of a Standard Product Term

A standard product term is equal to 1 for only one combination of variable values. For example, the product term  $\overline{A}\overline{B}C\overline{D}$  is equal to 1 when  $A = 1, B = 0, C = 1, D = 0$ , as shown below, and is 0 for all other combinations of values for the variables.

$$\overline{A}\overline{B}C\overline{D} = 1 \cdot \overline{0} \cdot 1 \cdot \overline{0} = 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

In this case, the product term has a binary value of 1010 (decimal ten).

Remember, a product term is implemented with an AND gate whose output is 1 only if each of its inputs is 1. Inverters are used to produce the complements of the variables as required.

**An SOP expression is equal to 1 only if one or more of the product terms in the expression is equal to 1.**

#### EXAMPLE 4-16

Determine the binary values for which the following standard SOP expression is equal to 1:

$$ABCD + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}C\overline{D}$$

#### Solution

The term  $ABCD$  is equal to 1 when  $A = 1, B = 1, C = 1$ , and  $D = 1$ .

$$ABCD = 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

The term  $A\bar{B}\bar{C}D$  is equal to 1 when  $A = 1$ ,  $B = 0$ ,  $C = 0$ , and  $D = 1$ .

$$A\bar{B}\bar{C}D = 1 \cdot \bar{0} \cdot \bar{0} \cdot 1 = 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

The term  $\bar{A}\bar{B}\bar{C}\bar{D}$  is equal to 1 when  $A = 0$ ,  $B = 0$ ,  $C = 0$ , and  $D = 0$ .

$$\bar{A}\bar{B}\bar{C}\bar{D} = \bar{0} \cdot \bar{0} \cdot \bar{0} \cdot \bar{0} = 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

The SOP expression equals 1 when any or all of the three product terms is 1.

### Related Problem

Determine the binary values for which the following SOP expression is equal to 1:

$$\bar{X}YZ + X\bar{Y}Z + XY\bar{Z} + \bar{X}\bar{Y}\bar{Z} + XYZ$$

Is this a standard SOP expression?

## The Product-of-Sums (POS) Form

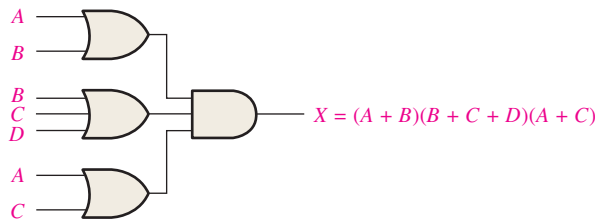
A sum term was defined in Section 4-1 as a term consisting of the sum (Boolean addition) of literals (variables or their complements). When two or more sum terms are multiplied, the resulting expression is a **product-of-sums (POS)**. Some examples are

$$\begin{aligned} &(\bar{A} + B)(A + \bar{B} + C) \\ &(\bar{A} + \bar{B} + \bar{C})(C + \bar{D} + E)(\bar{B} + C + D) \\ &(A + B)(A + \bar{B} + C)(\bar{A} + C) \end{aligned}$$

A POS expression can contain a single-variable term, as in  $\bar{A}(A + \bar{B} + C)(\bar{B} + \bar{C} + D)$ . In a POS expression, a single overbar cannot extend over more than one variable; however, more than one variable in a term can have an overbar. For example, a POS expression can have the term  $\bar{A} + \bar{B} + \bar{C}$  but not  $\overline{A + B + C}$ .

### Implementation of a POS Expression

Implementing a POS expression simply requires ANDing the outputs of two or more OR gates. A sum term is produced by an OR operation, and the product of two or more sum terms is produced by an AND operation. Therefore, a POS expression can be implemented by logic in which the outputs of a number (equal to the number of sum terms in the expression) of OR gates connect to the inputs of an AND gate, as Figure 4-24 shows for the expression  $(A + B)(B + C + D)(A + C)$ . The output  $X$  of the AND gate equals the POS expression.



**FIGURE 4-24** Implementation of the POS expression  $(A + B)(B + C + D)(A + C)$ .

## The Standard POS Form

So far, you have seen POS expressions in which some of the sum terms do not contain all of the variables in the domain of the expression. For example, the expression

$$(A + \bar{B} + C)(A + B + \bar{D})(A + \bar{B} + \bar{C} + D)$$

has a domain made up of the variables  $A$ ,  $B$ ,  $C$ , and  $D$ . Notice that the complete set of variables in the domain is not represented in the first two terms of the expression; that is,  $D$  or  $\bar{D}$  is missing from the first term and  $C$  or  $\bar{C}$  is missing from the second term.

A *standard POS expression* is one in which *all* the variables in the domain appear in each sum term in the expression. For example,

$$(\bar{A} + \bar{B} + \bar{C} + \bar{D})(A + \bar{B} + C + D)(A + B + \bar{C} + D)$$

is a standard POS expression. Any nonstandard POS expression (referred to simply as POS) can be converted to the standard form using Boolean algebra.

### Converting a Sum Term to Standard POS

Each sum term in a POS expression that does not contain all the variables in the domain can be expanded to standard form to include all variables in the domain and their complements. As stated in the following steps, a nonstandard POS expression is converted into standard form using Boolean algebra rule 8 ( $A \cdot \bar{A} = 0$ ) from Table 4–1: A variable multiplied by its complement equals 0.

**Step 1:** Add to each nonstandard product term a term made up of the product of the missing variable and its complement. This results in two sum terms. As you know, you can add 0 to anything without changing its value.

**Step 2:** Apply rule 12 from Table 4–1:  $A + BC = (A + B)(A + C)$

**Step 3:** Repeat Step 1 until all resulting sum terms contain all variables in the domain in either complemented or uncomplemented form.

#### EXAMPLE 4–17

Convert the following Boolean expression into standard POS form:

$$(A + \bar{B} + C)(\bar{B} + C + \bar{D})(A + \bar{B} + \bar{C} + D)$$

#### Solution

The domain of this POS expression is  $A, B, C, D$ . Take one term at a time. The first term,  $A + \bar{B} + C$ , is missing variable  $D$  or  $\bar{D}$ , so add  $D\bar{D}$  and apply rule 12 as follows:

$$A + \bar{B} + C = A + \bar{B} + C + D\bar{D} = (A + \bar{B} + C + D)(A + \bar{B} + C + \bar{D})$$

The second term,  $\bar{B} + C + \bar{D}$ , is missing variable  $A$  or  $\bar{A}$ , so add  $A\bar{A}$  and apply rule 12 as follows:

$$\bar{B} + C + \bar{D} = \bar{B} + C + \bar{D} + A\bar{A} = (A + \bar{B} + C + \bar{D})(\bar{A} + \bar{B} + C + \bar{D})$$

The third term,  $A + \bar{B} + \bar{C} + D$ , is already in standard form. The standard POS form of the original expression is as follows:

$$(A + \bar{B} + C)(\bar{B} + C + \bar{D})(A + \bar{B} + \bar{C} + D) = (A + \bar{B} + C + D)(A + \bar{B} + C + \bar{D})(A + \bar{B} + C + \bar{D})(\bar{A} + \bar{B} + C + \bar{D})(A + \bar{B} + \bar{C} + D)$$

#### Related Problem

Convert the expression  $(A + \bar{B})(B + C)$  to standard POS form.

### Binary Representation of a Standard Sum Term

A standard sum term is equal to 0 for only one combination of variable values. For example, the sum term  $A + \bar{B} + C + \bar{D}$  is 0 when  $A = 0, B = 1, C = 0$ , and  $D = 1$ , as shown below, and is 1 for all other combinations of values for the variables.

$$A + \bar{B} + C + \bar{D} = 0 + \bar{1} + 0 + \bar{1} = 0 + 0 + 0 + 0 = 0$$

In this case, the sum term has a binary value of 0101 (decimal 5). Remember, a sum term is implemented with an OR gate whose output is 0 only if each of its inputs is 0. Inverters are used to produce the complements of the variables as required.

**A POS expression is equal to 0 only if one or more of the sum terms in the expression is equal to 0.**

**EXAMPLE 4-18**

Determine the binary values of the variables for which the following standard POS expression is equal to 0:

$$(A + B + C + D)(A + \bar{B} + \bar{C} + D)(\bar{A} + \bar{B} + \bar{C} + \bar{D})$$

**Solution**

The term  $A + B + C + D$  is equal to 0 when  $A = 0$ ,  $B = 0$ ,  $C = 0$ , and  $D = 0$ .

$$A + B + C + D = 0 + 0 + 0 + 0 = 0$$

The term  $A + \bar{B} + \bar{C} + D$  is equal to 0 when  $A = 0$ ,  $B = 1$ ,  $C = 1$ , and  $D = 0$ .

$$A + \bar{B} + \bar{C} + D = 0 + \bar{1} + \bar{1} + 0 = 0 + 0 + 0 + 0 = 0$$

The term  $\bar{A} + \bar{B} + \bar{C} + \bar{D}$  is equal to 0 when  $A = 1$ ,  $B = 1$ ,  $C = 1$ , and  $D = 1$ .

$$\bar{A} + \bar{B} + \bar{C} + \bar{D} = \bar{1} + \bar{1} + \bar{1} + \bar{1} = 0 + 0 + 0 + 0 = 0$$

The POS expression equals 0 when any of the three sum terms equals 0.

**Related Problem**

Determine the binary values for which the following POS expression is equal to 0:

$$(X + \bar{Y} + Z)(\bar{X} + Y + Z)(X + Y + \bar{Z})(\bar{X} + \bar{Y} + \bar{Z})(X + \bar{Y} + \bar{Z})$$

Is this a standard POS expression?

## Converting Standard SOP to Standard POS

The binary values of the product terms in a given standard SOP expression are not present in the equivalent standard POS expression. Also, the binary values that are not represented in the SOP expression are present in the equivalent POS expression. Therefore, to convert from standard SOP to standard POS, the following steps are taken:

- Step 1:** Evaluate each product term in the SOP expression. That is, determine the binary numbers that represent the product terms.
- Step 2:** Determine all of the binary numbers not included in the evaluation in Step 1.
- Step 3:** Write the equivalent sum term for each binary number from Step 2 and express in POS form.

Using a similar procedure, you can go from POS to SOP.

**EXAMPLE 4-19**

Convert the following SOP expression to an equivalent POS expression:

$$\bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C + ABC$$

**Solution**

The evaluation is as follows:

$$000 + 010 + 011 + 101 + 111$$

Since there are three variables in the domain of this expression, there are a total of eight ( $2^3$ ) possible combinations. The SOP expression contains five of these combinations, so the POS must contain the other three which are 001, 100, and 110. Remember, these are the binary values that make the sum term 0. The equivalent POS expression is

$$(A + B + \bar{C})(\bar{A} + B + C)(\bar{A} + \bar{B} + C)$$

### Related Problem

Verify that the SOP and POS expressions in this example are equivalent by substituting binary values into each.

### SECTION 4-6 CHECKUP

- Identify each of the following expressions as SOP, standard SOP, POS, or standard POS:  
 (a)  $AB + \overline{A}BD + \overline{A}C\overline{D}$       (b)  $(A + \overline{B} + C)(A + B + \overline{C})$   
 (c)  $\overline{A}BC + ABC$       (d)  $(A + \overline{C})(A + B)$
- Convert each SOP expression in Question 1 to standard form.
- Convert each POS expression in Question 1 to standard form.

## 4-7 Boolean Expressions and Truth Tables

All standard Boolean expressions can be easily converted into truth table format using binary values for each term in the expression. The truth table is a common way of presenting, in a concise format, the logical operation of a circuit. Also, standard SOP or POS expressions can be determined from a truth table. You will find truth tables in data sheets and other literature related to the operation of digital circuits.

After completing this section, you should be able to

- Convert a standard SOP expression into truth table format
- Convert a standard POS expression into truth table format
- Derive a standard expression from a truth table
- Properly interpret truth table data

### Converting SOP Expressions to Truth Table Format

Recall from Section 4-6 that an SOP expression is equal to 1 only if at least one of the product terms is equal to 1. A truth table is simply a list of the possible combinations of input variable values and the corresponding output values (1 or 0). For an expression with a domain of two variables, there are four different combinations of those variables ( $2^2 = 4$ ). For an expression with a domain of three variables, there are eight different combinations of those variables ( $2^3 = 8$ ). For an expression with a domain of four variables, there are sixteen different combinations of those variables ( $2^4 = 16$ ), and so on.

The first step in constructing a truth table is to list all possible combinations of binary values of the variables in the expression. Next, convert the SOP expression to standard form if it is not already. Finally, place a 1 in the output column (X) for each binary value that makes the standard SOP expression a 1 and place a 0 for all the remaining binary values. This procedure is illustrated in Example 4-20.

### EXAMPLE 4-20

Develop a truth table for the standard SOP expression  $\overline{A}\overline{B}C + A\overline{B}\overline{C} + ABC$ .

### Solution

There are three variables in the domain, so there are eight possible combinations of binary values of the variables as listed in the left three columns of Table 4-6. The binary values that make the product terms in the expressions equal to 1 are

**TABLE 4-6**

| Inputs |   |   | Output | Product Term      |
|--------|---|---|--------|-------------------|
| A      | B | C | X      |                   |
| 0      | 0 | 0 | 0      |                   |
| 0      | 0 | 1 | 1      | $\bar{A}\bar{B}C$ |
| 0      | 1 | 0 | 0      |                   |
| 0      | 1 | 1 | 0      |                   |
| 1      | 0 | 0 | 1      | $A\bar{B}\bar{C}$ |
| 1      | 0 | 1 | 0      |                   |
| 1      | 1 | 0 | 0      |                   |
| 1      | 1 | 1 | 1      | $ABC$             |

$\bar{A}\bar{B}C$ : 001;  $A\bar{B}\bar{C}$ : 100; and  $ABC$ : 111. For each of these binary values, place a 1 in the output column as shown in the table. For each of the remaining binary combinations, place a 0 in the output column.

### Related Problem

Create a truth table for the standard SOP expression  $\bar{A}\bar{B}C + A\bar{B}\bar{C}$ .

## Converting POS Expressions to Truth Table Format

Recall that a POS expression is equal to 0 only if at least one of the sum terms is equal to 0. To construct a truth table from a POS expression, list all the possible combinations of binary values of the variables just as was done for the SOP expression. Next, convert the POS expression to standard form if it is not already. Finally, place a 0 in the output column (X) for each binary value that makes the expression a 0 and place a 1 for all the remaining binary values. This procedure is illustrated in Example 4-21.

### EXAMPLE 4-21

Determine the truth table for the following standard POS expression:

$$(A + B + C)(A + \bar{B} + C)(A + \bar{B} + \bar{C})(\bar{A} + B + \bar{C})(\bar{A} + \bar{B} + C)$$

### Solution

There are three variables in the domain and the eight possible binary values are listed in the left three columns of Table 4-7. The binary values that make the sum terms in the expression equal to 0 are  $A + B + C$ : 000;  $A + \bar{B} + C$ : 010;  $A + \bar{B} + \bar{C}$ : 011;  $\bar{A} + B + \bar{C}$ : 101; and  $\bar{A} + \bar{B} + C$ : 110. For each of these binary values, place a 0 in the output column as shown in the table. For each of the remaining binary combinations, place a 1 in the output column.

**TABLE 4-7**

| Inputs |   |   | Output | Sum Term                  |
|--------|---|---|--------|---------------------------|
| A      | B | C | X      |                           |
| 0      | 0 | 0 | 0      | $(A + B + C)$             |
| 0      | 0 | 1 | 1      |                           |
| 0      | 1 | 0 | 0      | $(A + \bar{B} + C)$       |
| 0      | 1 | 1 | 0      | $(A + \bar{B} + \bar{C})$ |
| 1      | 0 | 0 | 1      |                           |
| 1      | 0 | 1 | 0      | $(\bar{A} + B + \bar{C})$ |
| 1      | 1 | 0 | 0      | $(\bar{A} + \bar{B} + C)$ |
| 1      | 1 | 1 | 1      |                           |

Notice that the truth table in this example is the same as the one in Example 4–20. This means that the SOP expression in the previous example and the POS expression in this example are equivalent.

### Related Problem

Develop a truth table for the following standard POS expression:

$$(A + \bar{B} + C)(A + B + \bar{C})(\bar{A} + \bar{B} + \bar{C})$$

## Determining Standard Expressions from a Truth Table

To determine the standard SOP expression represented by a truth table, list the binary values of the input variables for which the output is 1. Convert each binary value to the corresponding product term by replacing each 1 with the corresponding variable and each 0 with the corresponding variable complement. For example, the binary value 1010 is converted to a product term as follows:

$$1010 \longrightarrow A\bar{B}C\bar{D}$$

If you substitute, you can see that the product term is 1:

$$A\bar{B}C\bar{D} = 1 \cdot \bar{0} \cdot 1 \cdot \bar{0} = 1 \cdot 1 \cdot 1 \cdot 1 = 1$$

To determine the standard POS expression represented by a truth table, list the binary values for which the output is 0. Convert each binary value to the corresponding sum term by replacing each 1 with the corresponding variable complement and each 0 with the corresponding variable. For example, the binary value 1001 is converted to a sum term as follows:

$$1001 \longrightarrow \bar{A} + B + C + \bar{D}$$

If you substitute, you can see that the sum term is 0:

$$\bar{A} + B + C + \bar{D} = \bar{1} + 0 + 0 + \bar{1} = 0 + 0 + 0 + 0 = 0$$

### EXAMPLE 4–22

From the truth table in Table 4–8, determine the standard SOP expression and the equivalent standard POS expression.

**TABLE 4–8**

| Inputs |   |   | Output |
|--------|---|---|--------|
| A      | B | C | X      |
| 0      | 0 | 0 | 0      |
| 0      | 0 | 1 | 0      |
| 0      | 1 | 0 | 0      |
| 0      | 1 | 1 | 1      |
| 1      | 0 | 0 | 1      |
| 1      | 0 | 1 | 0      |
| 1      | 1 | 0 | 1      |
| 1      | 1 | 1 | 1      |

**Solution**

There are four 1s in the output column and the corresponding binary values are 011, 100, 110, and 111. Convert these binary values to product terms as follows:

$$011 \longrightarrow \overline{A}BC$$

$$100 \longrightarrow A\overline{B}\overline{C}$$

$$110 \longrightarrow AB\overline{C}$$

$$111 \longrightarrow ABC$$

The resulting standard SOP expression for the output  $X$  is

$$X = \overline{A}BC + A\overline{B}\overline{C} + AB\overline{C} + ABC$$

For the POS expression, the output is 0 for binary values 000, 001, 010, and 101. Convert these binary values to sum terms as follows:

$$000 \longrightarrow A + B + C$$

$$001 \longrightarrow A + B + \overline{C}$$

$$010 \longrightarrow A + \overline{B} + C$$

$$101 \longrightarrow \overline{A} + B + \overline{C}$$

The resulting standard POS expression for the output  $X$  is

$$X = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)(\overline{A} + B + \overline{C})$$

**Related Problem**

By substitution of binary values, show that the SOP and the POS expressions derived in this example are equivalent; that is, for any binary value each SOP and POS term should either both be 1 or both be 0, depending on the binary value.

**SECTION 4-7 CHECKUP**

1. If a certain Boolean expression has a domain of five variables, how many binary values will be in its truth table?
2. In a certain truth table, the output is a 1 for the binary value 0110. Convert this binary value to the corresponding product term using variables  $W$ ,  $X$ ,  $Y$ , and  $Z$ .
3. In a certain truth table, the output is a 0 for the binary value 1100. Convert this binary value to the corresponding sum term using variables  $W$ ,  $X$ ,  $Y$ , and  $Z$ .

**4-8 The Karnaugh Map**

A Karnaugh map provides a systematic method for simplifying Boolean expressions and, if properly used, will produce the simplest SOP or POS expression possible, known as the minimum expression. As you have seen, the effectiveness of algebraic simplification depends on your familiarity with all the laws, rules, and theorems of Boolean algebra and on your ability to apply them. The Karnaugh map, on the other hand, provides a “cookbook” method for simplification. Other simplification techniques include the Quine-McCluskey method and the Espresso algorithm.

After completing this section, you should be able to

- ◆ Construct a Karnaugh map for three or four variables
- ◆ Determine the binary value of each cell in a Karnaugh map
- ◆ Determine the standard product term represented by each cell in a Karnaugh map
- ◆ Explain cell adjacency and identify adjacent cells

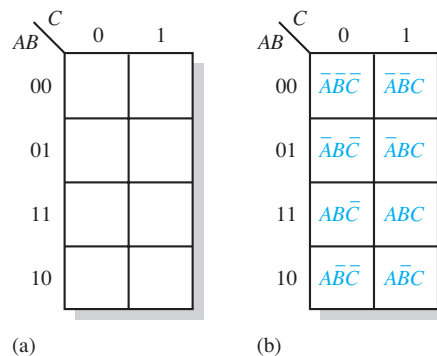
The purpose of a Karnaugh map is to simplify a Boolean expression.

A **Karnaugh map** is similar to a truth table because it presents all of the possible values of input variables and the resulting output for each value. Instead of being organized into columns and rows like a truth table, the Karnaugh map is an array of **cells** in which each cell represents a binary value of the input variables. The cells are arranged in a way so that simplification of a given expression is simply a matter of properly grouping the cells. Karnaugh maps can be used for expressions with two, three, four, and five variables, but we will discuss only 3-variable and 4-variable situations to illustrate the principles. *A discussion of 5-variable Karnaugh maps is available on the website.*

The number of cells in a Karnaugh map, as well as the number of rows in a truth table, is equal to the total number of possible input variable combinations. For three variables, the number of cells is  $2^3 = 8$ . For four variables, the number of cells is  $2^4 = 16$ .

### The 3-Variable Karnaugh Map

The 3-variable Karnaugh map is an array of eight cells, as shown in Figure 4–25(a). In this case,  $A$ ,  $B$ , and  $C$  are used for the variables although other letters could be used. Binary values of  $A$  and  $B$  are along the left side (notice the sequence) and the values of  $C$  are across the top. The value of a given cell is the binary values of  $A$  and  $B$  at the left in the same row combined with the value of  $C$  at the top in the same column. For example, the cell in the upper left corner has a binary value of 000 and the cell in the lower right corner has a binary value of 101. Figure 4–25(b) shows the standard product terms that are represented by each cell in the Karnaugh map.



**FIGURE 4-25** A 3-variable Karnaugh map showing Boolean product terms for each cell.

### The 4-Variable Karnaugh Map

The 4-variable Karnaugh map is an array of sixteen cells, as shown in Figure 4–26(a). Binary values of  $A$  and  $B$  are along the left side and the values of  $C$  and  $D$  are across the top. The value of a given cell is the binary values of  $A$  and  $B$  at the left in the same row combined with the binary values of  $C$  and  $D$  at the top in the same column. For example, the cell in the upper right corner has a binary value of 0010 and the cell in the lower right corner has a binary value of 1010. Figure 4–26(b) shows the standard product terms that are represented by each cell in the 4-variable Karnaugh map.

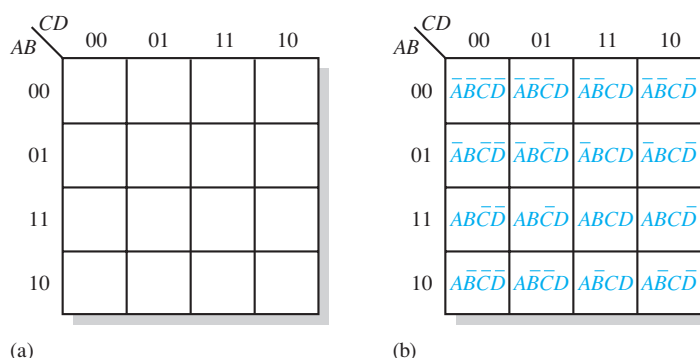
### Cell Adjacency

The cells in a Karnaugh map are arranged so that there is only a single-variable change between adjacent cells. **Adjacency** is defined by a single-variable change. In the 3-variable map the 010 cell is adjacent to the 000 cell, the 011 cell, and the 110 cell. The 010 cell is not adjacent to the 001 cell, the 111 cell, the 100 cell, or the 101 cell.

Physically, each cell is adjacent to the cells that are immediately next to it on any of its four sides. A cell is not adjacent to the cells that diagonally touch any of its corners. Also, the cells in the top row are adjacent to the corresponding cells in the bottom row and

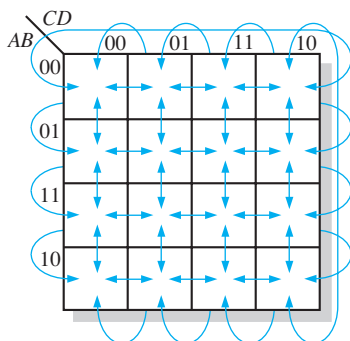
Cells that differ by only one variable are adjacent.

Cells with values that differ by more than one variable are not adjacent.



**FIGURE 4-26** A 4-variable Karnaugh map.

the cells in the outer left column are adjacent to the corresponding cells in the outer right column. This is called “wrap-around” adjacency because you can think of the map as wrapping around from top to bottom to form a cylinder or from left to right to form a cylinder. Figure 4-27 illustrates the cell adjacencies with a 4-variable map, although the same rules for adjacency apply to Karnaugh maps with any number of cells.



**FIGURE 4-27** Adjacent cells on a Karnaugh map are those that differ by only one variable. Arrows point between adjacent cells.

## The Quine-McCluskey Method

Minimizing Boolean functions using Karnaugh maps is practical only for up to four or five variables. Also, the Karnaugh map method does not lend itself to be automated in the form of a computer program.

The Quine-McCluskey method is more practical for logic simplification of functions with more than four or five variables. It also has the advantage of being easily implemented with a computer or programmable calculator.

The Quine-McCluskey method is functionally similar to Karnaugh mapping, but the tabular form makes it more efficient for use in computer algorithms, and it also gives a way to check that the minimal form of a Boolean function has been reached. This method is sometimes referred to as the *tabulation method*. An introduction to the Quine-McCluskey method is provided in Section 4-11.

## Espresso Algorithm

Although the Quine-McCluskey method is well suited to be implemented in a computer program and can handle more variables than the Karnaugh map method, the result is still far from efficient in terms of processing time and memory usage. Adding a variable to the function will roughly double both of these parameters because the truth table length increases exponentially with the number of variables. Functions with a large number of

variables have to be minimized with other methods such as the Espresso logic minimizer, which has become the de facto world standard. *An Espresso algorithm tutorial is available on the website.*

Compared to the other methods, Espresso is essentially more efficient in terms of reducing memory usage and computation time by several orders of magnitude. There is essentially no restrictions to the number of variables, output functions, and product terms of a combinational logic function. In general, tens of variables with tens of output functions can be handled by Espresso.

The Espresso algorithm has been incorporated as a standard logic function minimization step in most logic synthesis tools for programmable logic devices. For implementing a function in multilevel logic, the minimization result is optimized by factorization and mapped onto the available basic logic cells in the target device, such as an FPGA (Field-Programmable Gate Array).

#### SECTION 4-8 CHECKUP

1. In a 3-variable Karnaugh map, what is the binary value for the cell in each of the following locations:
 

|                       |                        |
|-----------------------|------------------------|
| (a) upper left corner | (b) lower right corner |
| (c) lower left corner | (d) upper right corner |
2. What is the standard product term for each cell in Question 1 for variables  $X$ ,  $Y$ , and  $Z$ ?
3. Repeat Question 1 for a 4-variable map.
4. Repeat Question 2 for a 4-variable map using variables  $W$ ,  $X$ ,  $Y$ , and  $Z$ .

## 4-9 Karnaugh Map SOP Minimization

As stated in the last section, the Karnaugh map is used for simplifying Boolean expressions to their minimum form. A minimized SOP expression contains the fewest possible terms with the fewest possible variables per term. Generally, a minimum SOP expression can be implemented with fewer logic gates than a standard expression. In this section, Karnaugh maps with up to four variables are covered.

After completing this section, you should be able to

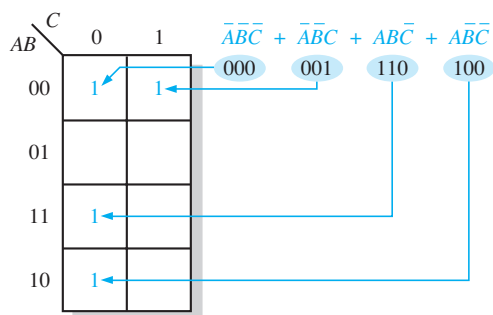
- ◆ Map a standard SOP expression on a Karnaugh map
- ◆ Combine the 1s on the map into maximum groups
- ◆ Determine the minimum product term for each group on the map
- ◆ Combine the minimum product terms to form a minimum SOP expression
- ◆ Convert a truth table into a Karnaugh map for simplification of the represented expression
- ◆ Use “don’t care” conditions on a Karnaugh map

### Mapping a Standard SOP Expression

For an SOP expression in standard form, a 1 is placed on the Karnaugh map for each product term in the expression. Each 1 is placed in a cell corresponding to the value of a product term. For example, for the product term  $A\bar{B}C$ , a 1 goes in the 101 cell on a 3-variable map.

When an SOP expression is completely mapped, there will be a number of 1s on the Karnaugh map equal to the number of product terms in the standard SOP expression. The cells that do not have a 1 are the cells for which the expression is 0. Usually, when working with SOP expressions, the 0s are left off the map. The following steps and the illustration in Figure 4–28 show the mapping process.

- Step 1:** Determine the binary value of each product term in the standard SOP expression. After some practice, you can usually do the evaluation of terms mentally.
- Step 2:** As each product term is evaluated, place a 1 on the Karnaugh map in the cell having the same value as the product term.



**FIGURE 4–28** Example of mapping a standard SOP expression.

#### EXAMPLE 4–23

Map the following standard SOP expression on a Karnaugh map:

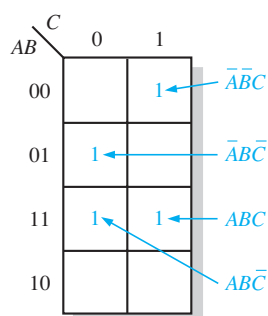
$$\bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

#### Solution

Evaluate the expression as shown below. Place a 1 on the 3-variable Karnaugh map in Figure 4–29 for each standard product term in the expression.

$$\bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$

0 0 1    0 1 0    1 1 0    1 1 1



**FIGURE 4–29**

#### Related Problem

Map the standard SOP expression  $\bar{A}BC + \bar{A}B\bar{C} + A\bar{B}\bar{C}$  on a Karnaugh map.

#### EXAMPLE 4-24

Map the following standard SOP expression on a Karnaugh map:

$$\bar{A}\bar{B}CD + \bar{A}B\bar{C}\bar{D} + AB\bar{C}D + ABCD + A\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}BC\bar{D}$$

#### Solution

Evaluate the expression as shown below. Place a 1 on the 4-variable Karnaugh map in Figure 4-30 for each standard product term in the expression.

$$\bar{A}\bar{B}CD + \bar{A}B\bar{C}\bar{D} + AB\bar{C}D + ABCD + A\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}BC\bar{D}$$

0011    0100    1101    1111    1100    0001    1010

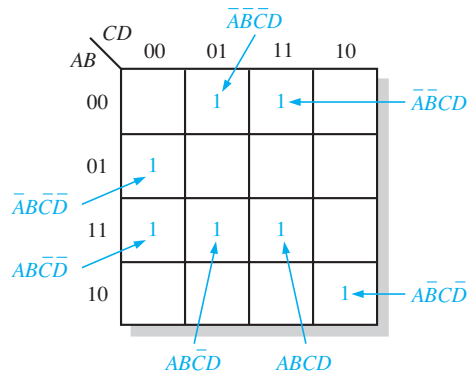


FIGURE 4-30

#### Related Problem

Map the following standard SOP expression on a Karnaugh map:

$$\bar{A}BC\bar{D} + ABC\bar{D} + A\bar{B}\bar{C}\bar{D} + ABCD$$

### Mapping a Nonstandard SOP Expression

A Boolean expression must first be in standard form before you use a Karnaugh map. If an expression is not in standard form, then it must be converted to standard form by the procedure covered in Section 4-6 or by numerical expansion. Since an expression should be evaluated before mapping anyway, numerical expansion is probably the most efficient approach.

#### Numerical Expansion of a Nonstandard Product Term

Recall that a nonstandard product term has one or more missing variables. For example, assume that one of the product terms in a certain 3-variable SOP expression is  $A\bar{B}$ . This term can be expanded numerically to standard form as follows. First, write the binary value of the two variables and attach a 0 for the missing variable  $\bar{C}$ : 100. Next, write the binary value of the two variables and attach a 1 for the missing variable  $C$ : 101. The two resulting binary numbers are the values of the standard SOP terms  $A\bar{B}\bar{C}$  and  $A\bar{B}C$ .

As another example, assume that one of the product terms in a 3-variable expression is  $B$  (remember that a single variable counts as a product term in an SOP expression). This term can be expanded numerically to standard form as follows. Write the binary value of the variable; then attach all possible values for the missing variables  $A$  and  $C$  as follows:

$B$   
010  
011  
110  
111

The four resulting binary numbers are the values of the standard SOP terms  $\overline{A}\overline{B}\overline{C}$ ,  $\overline{A}B\overline{C}$ ,  $A\overline{B}\overline{C}$ , and  $ABC$ .

#### EXAMPLE 4-25

Map the following SOP expression on a Karnaugh map:  $\overline{A} + \overline{A}\overline{B} + \overline{A}B\overline{C}$ .

#### Solution

The SOP expression is obviously not in standard form because each product term does not have three variables. The first term is missing two variables, the second term is missing one variable, and the third term is standard. First expand the terms numerically as follows:

$$\begin{array}{rcl} \overline{A} & + & \overline{A}\overline{B} + \overline{A}B\overline{C} \\ 000 & 100 & 110 \\ 001 & 101 & \\ 010 & & \\ 011 & & \end{array}$$

Map each of the resulting binary values by placing a 1 in the appropriate cell of the 3-variable Karnaugh map in Figure 4-31.

|      |    |     |   |
|------|----|-----|---|
|      |    | $C$ |   |
|      |    | 0   | 1 |
| $AB$ | 00 | 1   | 1 |
|      | 01 | 1   | 1 |
|      | 11 | 1   |   |
|      | 10 | 1   | 1 |

FIGURE 4-31

#### Related Problem

Map the SOP expression  $BC + \overline{A}\overline{C}$  on a Karnaugh map.

#### EXAMPLE 4-26

Map the following SOP expression on a Karnaugh map:

$$\overline{B}\overline{C} + \overline{A}\overline{B} + \overline{A}B\overline{C} + \overline{A}B\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}B\overline{C}D$$

#### Solution

The SOP expression is obviously not in standard form because each product term does not have four variables. The first and second terms are both missing two variables, the third term is missing one variable, and the rest of the terms are standard. First expand the terms by including all combinations of the missing variables numerically as follows:

$$\begin{array}{rclclcl} \overline{B}\overline{C} & + & \overline{A}\overline{B} & + & \overline{A}B\overline{C} & + & \overline{A}B\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}B\overline{C}D \\ 0000 & 1000 & 1100 & 1010 & 0001 & 1011 \\ 0001 & 1001 & 1101 & & & \\ 1000 & 1010 & & & & \\ 1001 & 1011 & & & & \end{array}$$

Map each of the resulting binary values by placing a 1 in the appropriate cell of the 4-variable Karnaugh map in Figure 4–32. Notice that some of the values in the expanded expression are redundant.

|    |    |    |    |    |    |
|----|----|----|----|----|----|
|    |    | CD |    |    |    |
|    |    | 00 | 01 | 11 | 10 |
| AB | 00 | 1  | 1  |    |    |
|    | 01 |    |    |    |    |
|    | 11 | 1  | 1  |    |    |
|    | 10 | 1  | 1  | 1  | 1  |

**FIGURE 4–32**

### Related Problem

Map the expression  $A + \overline{C}D + A\overline{C}D + \overline{A}BC\overline{D}$  on a Karnaugh map.

## Karnaugh Map Simplification of SOP Expressions

The process that results in an expression containing the fewest possible terms with the fewest possible variables is called **minimization**. After an SOP expression has been mapped, a minimum SOP expression is obtained by grouping the 1s and determining the minimum SOP expression from the map.

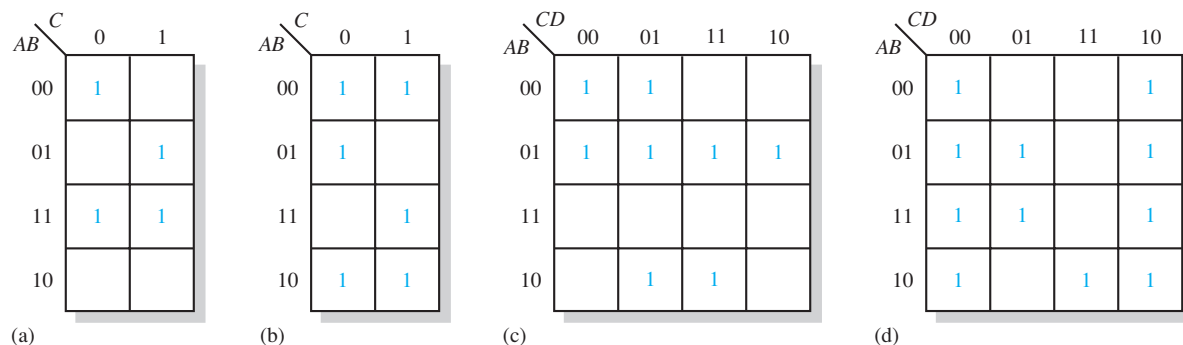
### Grouping the 1s

You can group 1s on the Karnaugh map according to the following rules by enclosing those adjacent cells containing 1s. The goal is to maximize the size of the groups and to minimize the number of groups.

1. A group must contain either 1, 2, 4, 8, or 16 cells, which are all powers of two. In the case of a 3-variable map,  $2^3 = 8$  cells is the maximum group.
2. Each cell in a group must be adjacent to one or more cells in that same group, but all cells in the group do not have to be adjacent to each other.
3. Always include the largest possible number of 1s in a group in accordance with rule 1.
4. Each 1 on the map must be included in at least one group. The 1s already in a group can be included in another group as long as the overlapping groups include noncommon 1s.

### EXAMPLE 4–27

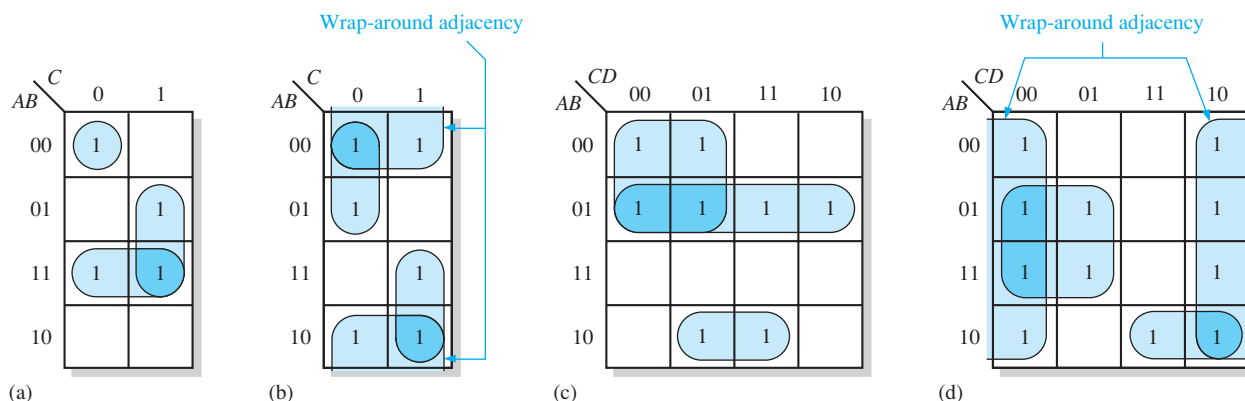
Group the 1s in each of the Karnaugh maps in Figure 4–33.



**FIGURE 4–33**

**Solution**

The groupings are shown in Figure 4–34. In some cases, there may be more than one way to group the 1s to form maximum groupings.

**FIGURE 4–34****Related Problem**

Determine if there are other ways to group the 1s in Figure 4–34 to obtain a minimum number of maximum groupings.

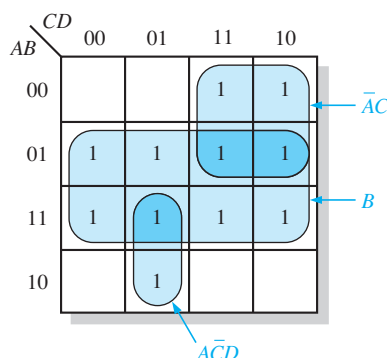
**Determining the Minimum SOP Expression from the Map**

When all the 1s representing the standard product terms in an expression are properly mapped and grouped, the process of determining the resulting minimum SOP expression begins. The following rules are applied to find the minimum product terms and the minimum SOP expression:

1. Group the cells that have 1s. Each group of cells containing 1s creates one product term composed of all variables that occur in only one form (either uncomplemented or complemented) within the group. Variables that occur both uncomplemented and complemented within the group are eliminated. These are called *contradictory variables*.
2. Determine the minimum product term for each group.
  - (a) For a 3-variable map:
    - (1) A 1-cell group yields a 3-variable product term
    - (2) A 2-cell group yields a 2-variable product term
    - (3) A 4-cell group yields a 1-variable term
    - (4) An 8-cell group yields a value of 1 for the expression
  - (b) For a 4-variable map:
    - (1) A 1-cell group yields a 4-variable product term
    - (2) A 2-cell group yields a 3-variable product term
    - (3) A 4-cell group yields a 2-variable product term
    - (4) An 8-cell group yields a 1-variable term
    - (5) A 16-cell group yields a value of 1 for the expression
3. When all the minimum product terms are derived from the Karnaugh map, they are summed to form the minimum SOP expression.

**EXAMPLE 4-28**

Determine the product terms for the Karnaugh map in Figure 4-35 and write the resulting minimum SOP expression.


**FIGURE 4-35**
**Solution**

Eliminate variables that are in a grouping in both complemented and uncomplemented forms. In Figure 4-35, the product term for the 8-cell group is  $B$  because the cells within that group contain both  $A$  and  $\bar{A}$ ,  $C$  and  $\bar{C}$ , and  $D$  and  $\bar{D}$ , which are eliminated. The 4-cell group contains  $B$ ,  $\bar{B}$ ,  $D$ , and  $\bar{D}$ , leaving the variables  $\bar{A}$  and  $C$ , which form the product term  $\bar{A}C$ . The 2-cell group contains  $B$  and  $\bar{B}$ , leaving variables  $A$ ,  $\bar{C}$ , and  $D$  which form the product term  $A\bar{C}D$ . Notice how overlapping is used to maximize the size of the groups. The resulting minimum SOP expression is the sum of these product terms:

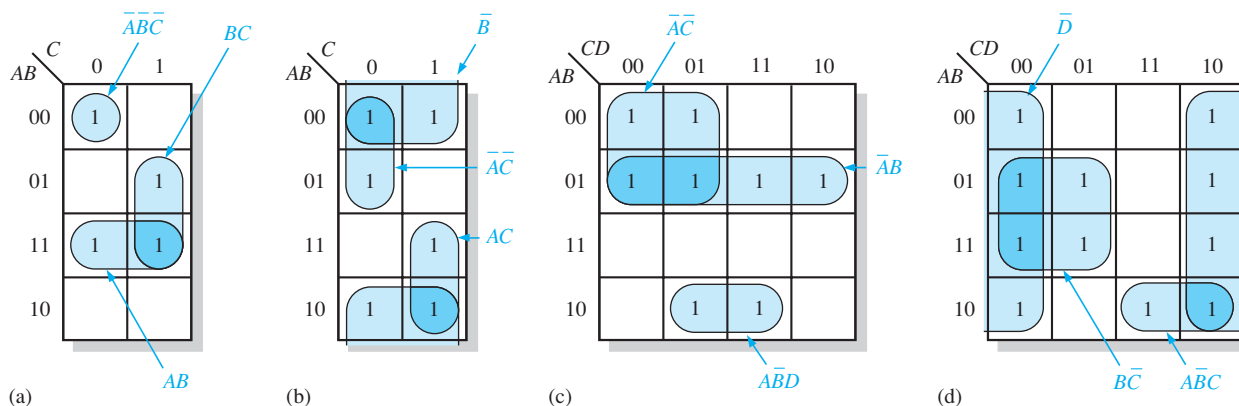
$$B + \bar{A}C + A\bar{C}D$$

**Related Problem**

For the Karnaugh map in Figure 4-35, add a 1 in the lower right cell (1010) and determine the resulting SOP expression.

**EXAMPLE 4-29**

Determine the product terms for each of the Karnaugh maps in Figure 4-36 and write the resulting minimum SOP expression.


**FIGURE 4-36**

**Solution**

The resulting minimum product term for each group is shown in Figure 4–36. The minimum SOP expressions for each of the Karnaugh maps in the figure are

(a)  $AB + BC + \overline{A}\overline{B}\overline{C}$

(b)  $\overline{B} + \overline{A}\overline{C} + AC$

(c)  $\overline{A}B + \overline{A}\overline{C} + \overline{A}BD$

(d)  $\overline{D} + \overline{A}BC + B\overline{C}$

**Related Problem**

For the Karnaugh map in Figure 4–36(d), add a 1 in the 0111 cell and determine the resulting SOP expression.

**EXAMPLE 4-30**

Use a Karnaugh map to minimize the following standard SOP expression:

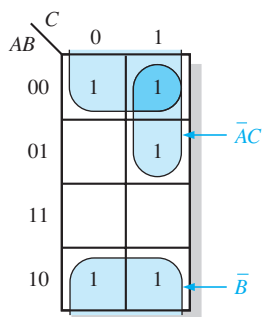
$$\overline{A}BC + \overline{A}BC + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C}$$

**Solution**

The binary values of the expression are

$$101 + 011 + 001 + 000 + 100$$

Map the standard SOP expression and group the cells as shown in Figure 4–37.

**FIGURE 4-37**

Notice the “wrap around” 4-cell group that includes the top row and the bottom row of 1s. The remaining 1 is absorbed in an overlapping group of two cells. The group of four 1s produces a single variable term,  $\overline{B}$ . This is determined by observing that within the group,  $\overline{B}$  is the only variable that does not change from cell to cell. The group of two 1s produces a 2-variable term  $\overline{A}C$ . This is determined by observing that within the group,  $\overline{A}$  and  $C$  do not change from one cell to the next. The product term for each group is shown. The resulting minimum SOP expression is

$$\overline{B} + \overline{A}C$$

Keep in mind that this minimum expression is equivalent to the original standard expression.

**Related Problem**

Use a Karnaugh map to simplify the following standard SOP expression:

$$X\overline{Y}Z + X\overline{Y}\overline{Z} + \overline{X}YZ + \overline{X}\overline{Y}\overline{Z} + X\overline{Y}\overline{Z} + XYZ$$

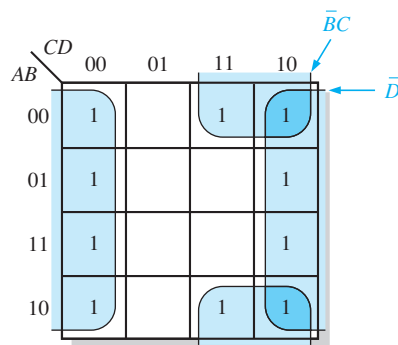
**EXAMPLE 4-31**

Use a Karnaugh map to minimize the following SOP expression:

$$\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + A\overline{B}C\overline{D} + \overline{A}\overline{B}C\overline{D} + \overline{A}B\overline{C}\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D}$$

**Solution**

The first term  $\overline{B}\overline{C}\overline{D}$  must be expanded into  $\overline{A}\overline{B}\overline{C}\overline{D}$  and  $A\overline{B}\overline{C}\overline{D}$  to get the standard SOP expression, which is then mapped; the cells are grouped as shown in Figure 4-38.



**FIGURE 4-38**

Notice that both groups exhibit “wrap around” adjacency. The group of eight is formed because the cells in the outer columns are adjacent. The group of four is formed to pick up the remaining two 1s because the top and bottom cells are adjacent. The product term for each group is shown. The resulting minimum SOP expression is

$$\overline{D} + \overline{B}C$$

Keep in mind that this minimum expression is equivalent to the original standard expression.

**Related Problem**

Use a Karnaugh map to simplify the following SOP expression:

$$\overline{W}\overline{X}\overline{Y}\overline{Z} + W\overline{X}YZ + W\overline{X}\overline{Y}Z + \overline{W}YZ + W\overline{X}\overline{Y}\overline{Z}$$

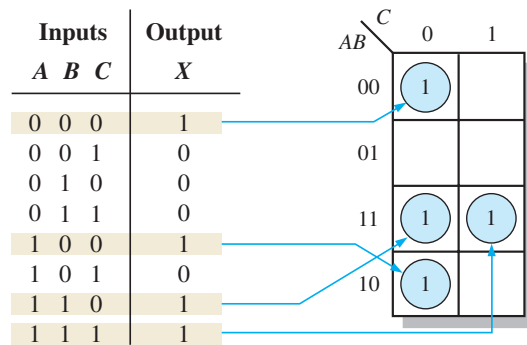
## Mapping Directly from a Truth Table

You have seen how to map a Boolean expression; now you will learn how to go directly from a truth table to a Karnaugh map. Recall that a truth table gives the output of a Boolean expression for all possible input variable combinations. An example of a Boolean expression and its truth table representation is shown in Figure 4-39. Notice in the truth table that the output  $X$  is 1 for four different input variable combinations. The 1s in the output column of the truth table are mapped directly onto a Karnaugh map into the cells corresponding to the values of the associated input variable combinations, as shown in Figure 4-39. In the figure you can see that the Boolean expression, the truth table, and the Karnaugh map are simply different ways to represent a logic function.

## “Don’t Care” Conditions

Sometimes a situation arises in which some input variable combinations are not allowed. For example, recall that in the BCD code covered in Chapter 2, there are six invalid combinations: 1010, 1011, 1100, 1101, 1110, and 1111. Since these unallowed states

$$X = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$



**FIGURE 4-39** Example of mapping directly from a truth table to a Karnaugh map.

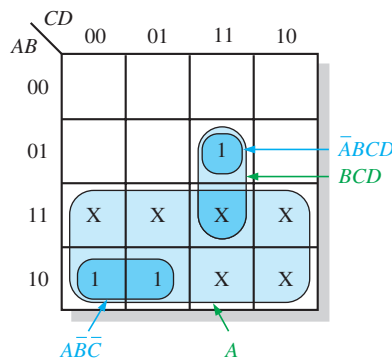
will never occur in an application involving the BCD code, they can be treated as “**don’t care**” terms with respect to their effect on the output. That is, for these “don’t care” terms either a 1 or a 0 may be assigned to the output; it really does not matter since they will never occur.

The “don’t care” terms can be used to advantage on the Karnaugh map. Figure 4-40 shows that for each “don’t care” term, an X is placed in the cell. When grouping the 1s, the Xs can be treated as 1s to make a larger grouping or as 0s if they cannot be used to advantage. The larger a group, the simpler the resulting term will be.

| Inputs |   |   |   | Output |
|--------|---|---|---|--------|
| A      | B | C | D | Y      |
| 0      | 0 | 0 | 0 | 0      |
| 0      | 0 | 0 | 1 | 0      |
| 0      | 0 | 1 | 0 | 0      |
| 0      | 0 | 1 | 1 | 0      |
| 0      | 1 | 0 | 0 | 0      |
| 0      | 1 | 0 | 1 | 0      |
| 0      | 1 | 1 | 0 | 0      |
| 0      | 1 | 1 | 1 | 1      |
| 1      | 0 | 0 | 0 | 1      |
| 1      | 0 | 0 | 1 | 1      |
| 1      | 0 | 1 | 0 | X      |
| 1      | 0 | 1 | 1 | X      |
| 1      | 1 | 0 | 0 | X      |
| 1      | 1 | 0 | 1 | X      |
| 1      | 1 | 1 | 0 | X      |
| 1      | 1 | 1 | 1 | X      |

(a) Truth table

Don’t cares



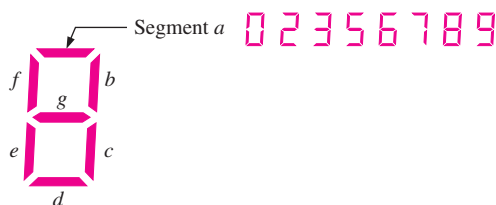
(b) Without “don’t cares”  $Y = \bar{A}\bar{B}\bar{C} + \bar{A}BCD$   
With “don’t cares”  $Y = A + BCD$

**FIGURE 4-40** Example of the use of “don’t care” conditions to simplify an expression.

The truth table in Figure 4-40(a) describes a logic function that has a 1 output only when the BCD code for 7, 8, or 9 is present on the inputs. If the “don’t cares” are used as 1s, the resulting expression for the function is  $A + BCD$ , as indicated in part (b). If the “don’t cares” are not used as 1s, the resulting expression is  $\bar{A}\bar{B}\bar{C} + \bar{A}BCD$ ; so you can see the advantage of using “don’t care” terms to get the simplest expression.

**EXAMPLE 4-32**

In a 7-segment display, each of the seven segments is activated for various digits. For example, segment *a* is activated for the digits 0, 2, 3, 5, 6, 7, 8, and 9, as illustrated in Figure 4-41. Since each digit can be represented by a BCD code, derive an SOP expression for segment *a* using the variables *ABCD* and then minimize the expression using a Karnaugh map.



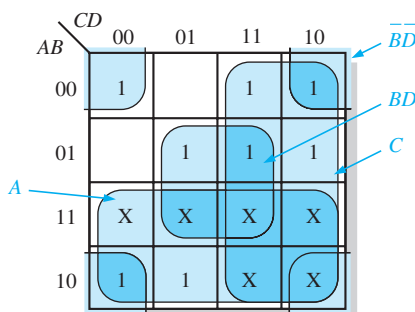
**FIGURE 4-41** 7-segment display.

**Solution**

The expression for segment *a* is

$$a = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}BC\overline{D} + \overline{A}BCD + A\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}D$$

Each term in the expression represents one of the digits in which segment *a* is used. The Karnaugh map minimization is shown in Figure 4-42. X's (don't cares) are entered for those states that do not occur in the BCD code.



**FIGURE 4-42**

From the Karnaugh map, the minimized expression for segment *a* is

$$a = A + C + BD + \overline{B}\overline{D}$$

**Related Problem**

Draw the logic diagram for the segment-*a* logic.

**SECTION 4-9 CHECKUP**

1. Lay out Karnaugh maps for three and four variables.
2. Group the 1s and write the simplified SOP expression for the Karnaugh map in Figure 4-29.
3. Write the original standard SOP expressions for each of the Karnaugh maps in Figure 4-36.

## 4-10 Karnaugh Map POS Minimization

In the last section, you studied the minimization of an SOP expression using a Karnaugh map. In this section, we focus on POS expressions. The approaches are much the same except that with POS expressions, 0s representing the standard sum terms are placed on the Karnaugh map instead of 1s.

After completing this section, you should be able to

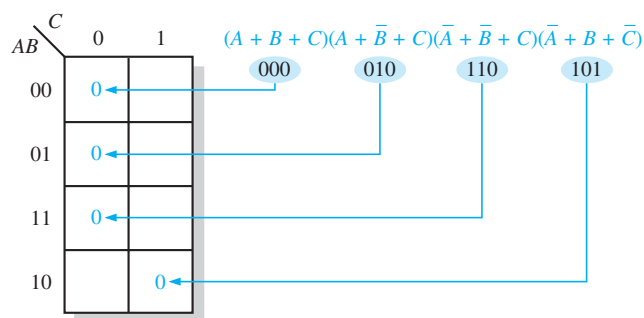
- ♦ Map a standard POS expression on a Karnaugh map
- ♦ Combine the 0s on the map into maximum groups
- ♦ Determine the minimum sum term for each group on the map
- ♦ Combine the minimum sum terms to form a minimum POS expression
- ♦ Use the Karnaugh map to convert between POS and SOP

### Mapping a Standard POS Expression

For a POS expression in standard form, a 0 is placed on the Karnaugh map for each sum term in the expression. Each 0 is placed in a cell corresponding to the value of a sum term. For example, for the sum term  $A + \bar{B} + C$ , a 0 goes in the 010 cell on a 3-variable map.

When a POS expression is completely mapped, there will be a number of 0s on the Karnaugh map equal to the number of sum terms in the standard POS expression. The cells that do not have a 0 are the cells for which the expression is 1. Usually, when working with POS expressions, the 1s are left off. The following steps and the illustration in Figure 4-43 show the mapping process.

- Step 1:** Determine the binary value of each sum term in the standard POS expression. This is the binary value that makes the term equal to 0.
- Step 2:** As each sum term is evaluated, place a 0 on the Karnaugh map in the corresponding cell.



**FIGURE 4-43** Example of mapping a standard POS expression.

### EXAMPLE 4-33

Map the following standard POS expression on a Karnaugh map:

$$(\bar{A} + \bar{B} + C + D)(\bar{A} + B + \bar{C} + \bar{D})(A + B + \bar{C} + D)(\bar{A} + \bar{B} + \bar{C} + \bar{D})(A + B + \bar{C} + \bar{D})$$

#### Solution

Evaluate the expression as shown below and place a 0 on the 4-variable Karnaugh map in Figure 4-44 for each standard sum term in the expression.

$$\begin{array}{cccccc}
 (\bar{A} + \bar{B} + C + D) & (\bar{A} + B + \bar{C} + \bar{D}) & (A + B + \bar{C} + D) & (\bar{A} + \bar{B} + \bar{C} + \bar{D}) & (A + B + \bar{C} + \bar{D}) & \\
 1100 & 1011 & 0010 & 1111 & 0011 & 
 \end{array}$$

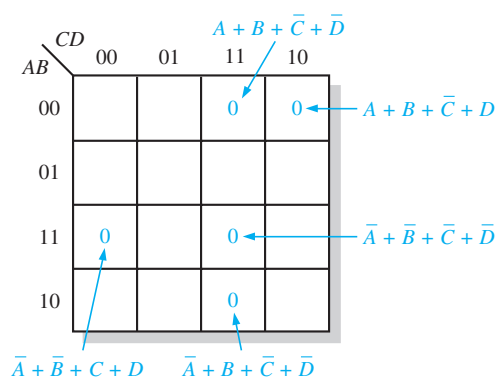


FIGURE 4-44

### Related Problem

Map the following standard POS expression on a Karnaugh map:

$$(A + \bar{B} + \bar{C} + D)(A + B + C + \bar{D})(A + B + C + D)(\bar{A} + B + \bar{C} + D)$$

## Karnaugh Map Simplification of POS Expressions

The process for minimizing a POS expression is basically the same as for an SOP expression except that you group 0s to produce minimum sum terms instead of grouping 1s to produce minimum product terms. The rules for grouping the 0s are the same as those for grouping the 1s that you learned in Section 4-9.

### EXAMPLE 4-34

Use a Karnaugh map to minimize the following standard POS expression:

$$(A + B + C)(A + B + \bar{C})(A + \bar{B} + C)(A + \bar{B} + \bar{C})(\bar{A} + \bar{B} + C)$$

Also, derive the equivalent SOP expression.

### Solution

The combinations of binary values of the expression are

$$(0 + 0 + 0)(0 + 0 + 1)(0 + 1 + 0)(0 + 1 + 1)(1 + 1 + 0)$$

Map the standard POS expression and group the cells as shown in Figure 4-45.

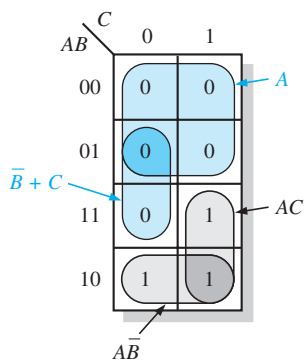


FIGURE 4-45

Notice how the 0 in the 110 cell is included into a 2-cell group by utilizing the 0 in the 4-cell group. The sum term for each blue group is shown in the figure and the resulting minimum POS expression is

$$A(\bar{B} + C)$$

Keep in mind that this minimum POS expression is equivalent to the original standard POS expression.

Grouping the 1s as shown by the gray areas yields an SOP expression that is equivalent to grouping the 0s.

$$AC + A\bar{B} = A(\bar{B} + C)$$

### Related Problem

Use a Karnaugh map to simplify the following standard POS expression:

$$(X + \bar{Y} + Z)(X + \bar{Y} + \bar{Z})(\bar{X} + \bar{Y} + Z)(\bar{X} + Y + Z)$$

### EXAMPLE 4-35

Use a Karnaugh map to minimize the following POS expression:

$$(B + C + D)(A + B + \bar{C} + D)(\bar{A} + B + C + D)(A + \bar{B} + C + D)(\bar{A} + \bar{B} + C + D)$$

### Solution

The first term must be expanded into  $\bar{A} + B + C + D$  and  $A + B + C + D$  to get a standard POS expression, which is then mapped; and the cells are grouped as shown in Figure 4-46. The sum term for each group is shown and the resulting minimum POS expression is

$$(C + D)(A + B + D)(\bar{A} + B + C)$$

Keep in mind that this minimum POS expression is equivalent to the original standard POS expression.

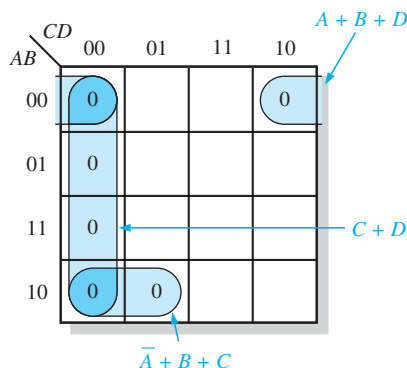


FIGURE 4-46

### Related Problem

Use a Karnaugh map to simplify the following POS expression:

$$(W + \bar{X} + Y + \bar{Z})(W + X + Y + Z)(W + \bar{X} + \bar{Y} + Z)(\bar{W} + \bar{X} + Z)$$

## Converting Between POS and SOP Using the Karnaugh Map

When a POS expression is mapped, it can easily be converted to the equivalent SOP form directly from the Karnaugh map. Also, given a mapped SOP expression, an equivalent POS expression can be derived directly from the map. This provides a good way to compare

both minimum forms of an expression to determine if one of them can be implemented with fewer gates than the other.

For a POS expression, all the cells that do not contain 0s contain 1s, from which the SOP expression is derived. Likewise, for an SOP expression, all the cells that do not contain 1s contain 0s, from which the POS expression is derived. Example 4–36 illustrates this conversion.

#### EXAMPLE 4–36

Using a Karnaugh map, convert the following standard POS expression into a minimum POS expression, a standard SOP expression, and a minimum SOP expression.

$$(\bar{A} + \bar{B} + C + D)(A + \bar{B} + C + D)(A + B + C + \bar{D})(A + B + \bar{C} + \bar{D})(\bar{A} + B + C + \bar{D})(A + B + \bar{C} + D)$$

#### Solution

The 0s for the standard POS expression are mapped and grouped to obtain the minimum POS expression in Figure 4–47(a). In Figure 4–47(b), 1s are added to the cells that do not contain 0s. From each cell containing a 1, a standard product term is obtained as indicated. These product terms form the standard SOP expression. In Figure 4–47(c), the 1s are grouped and a minimum SOP expression is obtained.

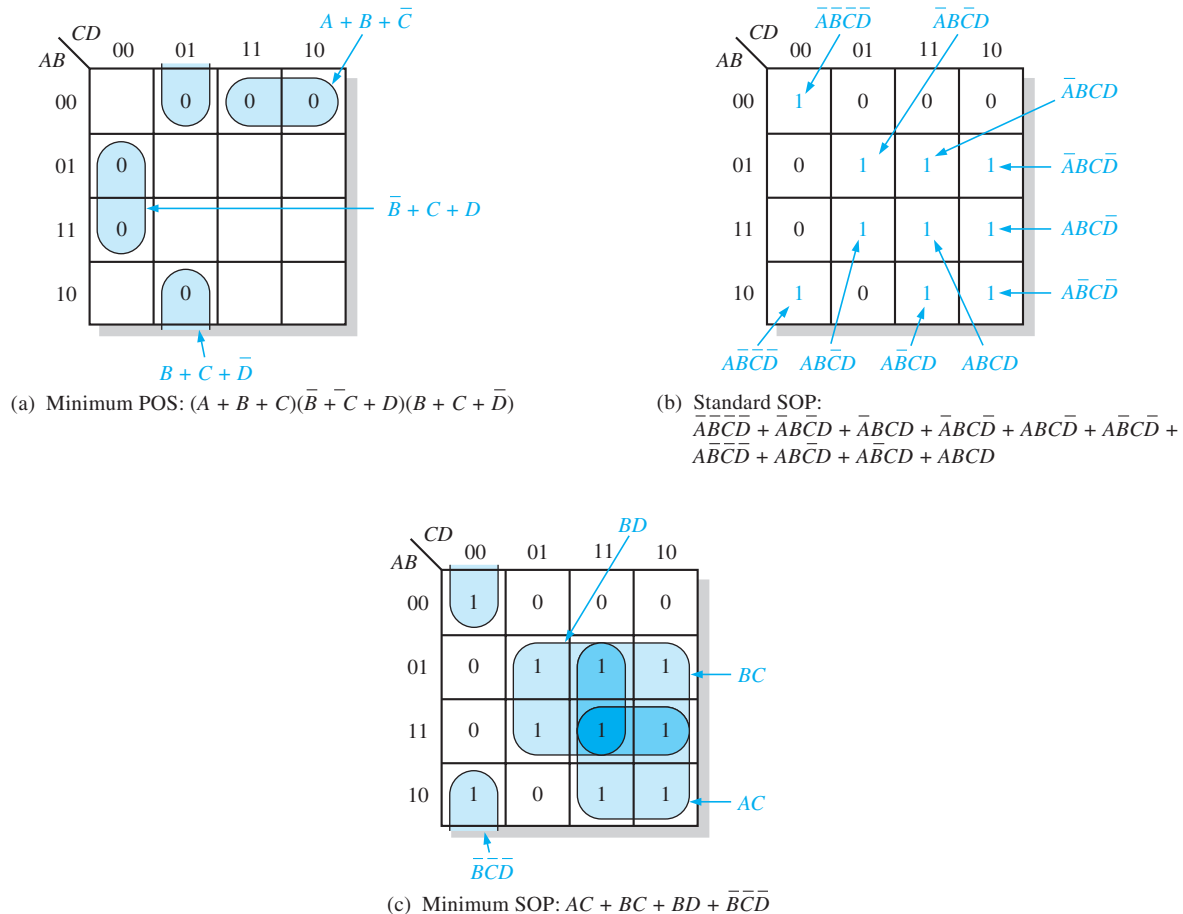


FIGURE 4–47

#### Related Problem

Use a Karnaugh map to convert the following expression to minimum SOP form:

$$(W + \bar{X} + Y + \bar{Z})(\bar{W} + X + \bar{Y} + \bar{Z})(\bar{W} + \bar{X} + \bar{Y} + Z)(\bar{W} + \bar{X} + \bar{Z})$$

**SECTION 4-10 CHECKUP**

1. What is the difference in mapping a POS expression and an SOP expression?
2. What is the standard sum term for a 0 in cell 1011?
3. What is the standard product term for a 1 in cell 0010?

**4-11 The Quine-McCluskey Method**

For Boolean functions up to four variables, the Karnaugh map method is a powerful minimization method. When there are five variables, the Karnaugh map method is difficult to apply and completely impractical beyond five. The Quine-McCluskey method is a formal tabular method for applying the Boolean distributive law to various terms to find the minimum sum of products by eliminating literals that appear in two terms as complements. (For example,  $ABCD + ABC\bar{D} = ABC$ ). A *Quine-McCluskey method tutorial* is available on the website.

After completing this section, you should be able to

- ♦ Describe the Quine-McCluskey method
- ♦ Reduce a Boolean expression using the Quine-McCluskey method

Unlike the Karnaugh mapping method, Quine-McCluskey lends itself to the computerized reduction of Boolean expressions, which is its principal use. For simple expressions, with up to four or perhaps even five variables, the Karnaugh map is easier for most people because it is a graphic method.

To apply the Quine-McCluskey method, first write the function in standard **minterm** (SOP) form. To illustrate, we will use the expression

$$X = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}CD + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}BC\bar{D} + \bar{A}BCD + AB\bar{C}\bar{D} + ABCD$$

and represent it as binary numbers on the truth table shown in Table 4-9. The minterms that appear in the function are listed in the right column.

**TABLE 4-9**

| <i>ABCD</i> | <i>X</i> | Minterm  |
|-------------|----------|----------|
| 0000        | 0        |          |
| 0001        | 1        | $m_1$    |
| 0010        | 0        |          |
| 0011        | 1        | $m_3$    |
| 0100        | 1        | $m_4$    |
| 0101        | 1        | $m_5$    |
| 0110        | 0        |          |
| 0111        | 0        |          |
| 1000        | 0        |          |
| 1001        | 0        |          |
| 1010        | 1        | $m_{10}$ |
| 1011        | 0        |          |
| 1100        | 1        | $m_{12}$ |
| 1101        | 1        | $m_{13}$ |
| 1110        | 0        |          |
| 1111        | 1        | $m_{15}$ |

The second step in applying the Quine-McCluskey method is to arrange the minterms in the original expression in groups according to the number of 1s in each minterm, as shown in Table 4-10. In this example, there are four groups of minterms. (Note that if  $m_0$  had been in the original expression, there would be five groups.)

**TABLE 4-10**

| Number of 1s | Minterm  | ABCD |
|--------------|----------|------|
| 1            | $m_1$    | 0001 |
|              | $m_4$    | 0100 |
| 2            | $m_3$    | 0011 |
|              | $m_5$    | 0101 |
|              | $m_{10}$ | 1010 |
|              | $m_{12}$ | 1100 |
| 3            | $m_{13}$ | 1101 |
| 4            | $m_{15}$ | 1111 |

Third, compare adjacent groups, looking to see if any minterms are the same in every position *except one*. If they are, place a check mark by those two minterms, as shown in Table 4-11. You should check each minterm against all others in the following group, but it is not necessary to check any groups that are not adjacent. In the column labeled *First Level*, you will have a list of the minterm names and the binary equivalent with an x as the placeholder for the literal that differs. In the example, minterm  $m_1$  in Group 1 (0001) is identical to  $m_3$  in Group 2 (0011) except for the *C* position, so place a check mark by these two minterms and enter 00x1 in the column labeled *First Level*. Minterm  $m_4$  (0100) is identical to  $m_5$  (0101) except for the *D* position, so check these two minterms and enter 010x in the last column. If a given term can be used more than once, it should be. In this case, notice that  $m_1$  can be used again with  $m_5$  in the second row with the x now placed in the *B* position.

**TABLE 4-11**

| Number of 1s in Minterm | Minterm  | ABCD   | First Level               |
|-------------------------|----------|--------|---------------------------|
| 1                       | $m_1$    | 0001 ✓ | ( $m_1, m_3$ ) 00x1       |
|                         | $m_4$    | 0100 ✓ | ( $m_1, m_5$ ) 0x01       |
| 2                       | $m_3$    | 0011 ✓ | ( $m_4, m_5$ ) 010x       |
|                         | $m_5$    | 0101 ✓ | ( $m_4, m_{12}$ ) x100    |
|                         | $m_{10}$ | 1010   | ( $m_5, m_{13}$ ) x101    |
|                         | $m_{12}$ | 1100 ✓ | ( $m_{12}, m_{13}$ ) 110x |
| 3                       | $m_{13}$ | 1101 ✓ | ( $m_{13}, m_{15}$ ) 11x1 |
| 4                       | $m_{15}$ | 1111 ✓ |                           |

In Table 4-11, minterm  $m_4$  and minterm  $m_{12}$  are identical except for the *A* position. Both minterms are checked and x100 is entered in the *First Level* column. Follow this procedure for groups 2 and 3. In these groups,  $m_5$  and  $m_{13}$  are combined and so are  $m_{12}$  and  $m_{13}$  (notice that  $m_{12}$  was previously used with  $m_4$  and is used again). For groups 3 and 4, both  $m_{13}$  and  $m_{15}$  are added to the list in the *First Level* column.

In this example, minterm  $m_{10}$  does not have a check mark because no other minterm meets the requirement of being identical except for one position. This term is called an *essential prime implicant*, and it must be included in our final reduced expression.

The terms listed in the *First Level* have been used to form a reduced table (Table 4-12) with one less group than before. The number of 1s remaining in the *First Level* are counted and used to form three new groups.

Terms in the new groups are compared against terms in the adjacent group down. You need to compare these terms only if the x is in the same relative position in adjacent groups; otherwise go on. If the two expressions differ by exactly one position, a check mark is

**TABLE 4-12**

| First Level                        | Number of 1s in First Level | Second Level                      |
|------------------------------------|-----------------------------|-----------------------------------|
| $(m_1, m_3) 00x1$                  | 1                           | $(m_4, m_5, m_{12}, m_{13}) x10x$ |
| $(m_1, m_5) 0x01$                  |                             | $(m_4, m_5, m_{12}, m_{13}) x10x$ |
| $(m_4, m_5) 010x \checkmark$       |                             |                                   |
| $(m_4, m_{12}) x100 \checkmark$    |                             |                                   |
| $(m_5, m_{13}) x101 \checkmark$    | 2                           |                                   |
| $(m_{12}, m_{13}) 110x \checkmark$ |                             |                                   |
| $(m_{13}, m_{15}) 11x1$            | 3                           |                                   |

placed next to both terms as before and all of the minterms are listed in the Second Level list. As before, the one position that has changed is entered as an x in the *Second Level*.

For our example, notice that the third term in Group 1 and the second term in Group 2 meet this requirement, differing only with the *A* literal. The fourth term in Group 1 also can be combined with the first term in Group 2, forming a redundant set of minterms. One of these can be crossed off the list and will not be used in the final expression.

With complicated expressions, the process described can be continued. For our example, we can read the *Second Level* expression as  $\overline{B}\overline{C}$ . The terms that are unchecked will form other terms in the final reduced expression. The first unchecked term is read as  $\overline{A}\overline{B}D$ . The next one is read as  $\overline{A}\overline{C}D$ . The last unchecked term is  $ABD$ . Recall that  $m_{10}$  was an essential prime implicant, so is picked up in the final expression. The reduced expression using the unchecked terms is:

$$X = \overline{B}\overline{C} + \overline{A}\overline{B}D + \overline{A}\overline{C}D + ABD + \overline{A}BC\overline{D}$$

Although this expression is correct, it may not be the minimum possible expression. There is a final check that can eliminate any unnecessary terms. The terms for the expression are written into a prime implicant table, with minterms for each prime implicant checked, as shown in Table 4-13.

**TABLE 4-13**

| Prime Implicants                                      | Minterms     |              |              |              |              |              |              |              |
|-------------------------------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                                                       | $m_1$        | $m_3$        | $m_4$        | $m_5$        | $m_{10}$     | $m_{12}$     | $m_{13}$     | $m_{15}$     |
| $\overline{B}\overline{C} (m_4, m_5, m_{12}, m_{13})$ |              |              | $\checkmark$ | $\checkmark$ |              | $\checkmark$ | $\checkmark$ |              |
| $\overline{A}\overline{B}D (m_1, m_3)$                | $\checkmark$ | $\checkmark$ |              |              |              |              |              |              |
| $\overline{A}\overline{C}D (m_1, m_5)$                | $\checkmark$ |              |              | $\checkmark$ |              |              |              |              |
| $ABD (m_{13}, m_{15})$                                |              |              |              |              |              |              | $\checkmark$ | $\checkmark$ |
| $\overline{A}BC\overline{D} (m_{10})$                 |              |              |              |              | $\checkmark$ |              |              |              |

If a minterm has a single check mark, then the prime implicant is essential and must be included in the final expression. The term  $ABD$  must be included because  $m_{15}$  is only covered by it. Likewise  $m_{10}$  is only covered by  $\overline{A}BC\overline{D}$ , so it must be in the final expression. Notice that the two minterms in  $\overline{A}\overline{C}D$  are covered by the prime implicants in the first two rows, so this term is unnecessary. The final reduced expression is, therefore,

$$X = \overline{B}\overline{C} + \overline{A}\overline{B}D + ABD + \overline{A}BC\overline{D}$$

**SECTION 4-11 CHECKUP**

1. What is a minterm?
2. What is an essential prime implicant?

## 4-12 Boolean Expressions with VHDL

The ability to create simple and compact code is important in a VHDL program. By simplifying a Boolean expression for a given logic function, it is easier to write and debug the VHDL code; in addition, the result is a clearer and more concise program. Many VHDL development software packages contain tools that automatically optimize a program when it is compiled and converted to a downloadable file. However, this does not relieve you from creating program code that is clear and concise. You should not only be concerned with the number of lines of code, but you should also be concerned with the complexity of each line of code. In this section, you will see the difference in VHDL code when simplification methods are applied. Also, three levels of abstraction used in the description of a logic function are examined. *A VHDL tutorial is available on the website.*

After completing this section, you should be able to

- ♦ Write VHDL code to represent a simplified logic expression and compare it to the code for the original expression
- ♦ Relate the advantages of optimized Boolean expressions as applied to a target device
- ♦ Understand how a logic function can be described at three levels of abstraction
- ♦ Relate VHDL approaches to the description of a logic function to the three levels of abstraction

### Boolean Algebra in VHDL Programming

The basic rules of Boolean algebra that you have learned in this chapter should be applied to any applicable VHDL code. Eliminating unnecessary gate logic allows you to create compact code that is easier to understand, especially when someone has to go back later and update or modify the program.

In Example 4-37, DeMorgan's theorems are used to simplify a Boolean expression, and VHDL programs for both the original expression and the simplified expression are compared.

#### EXAMPLE 4-37

First, write a VHDL program for the logic described by the following Boolean expression. Next, apply DeMorgan's theorems and Boolean rules to simplify the expression. Then write a program to reflect the simplified expression.

$$X = \overline{(AC + \overline{BC} + D)} + \overline{BC}$$

#### Solution

The VHDL program for the logic represented by the original expression is



```
entity OriginalLogic is
  port (A, B, C, D: in bit; X: out bit);
end entity OriginalLogic;
architecture Expression1 of OriginalLogic is
begin
  X <= not((A and C) or not(B and not C) or D) or not(not(B and C));
end architecture Expression1;
```

Four inputs and one output are described.

The original logic contains four inputs, 3 AND gates, 2 OR gates, and 3 inverters.

By selectively applying DeMorgan's theorem and the laws of Boolean algebra, you can reduce the Boolean expression to its simplest form.

$$\begin{aligned}
 \overline{(AC + \overline{BC} + D)} + \overline{BC} &= (\overline{AC})(\overline{\overline{BC}})\overline{D} + \overline{BC} && \text{Apply DeMorgan} \\
 &= (\overline{AC})(BC)\overline{D} + \overline{BC} && \text{Cancel double complements} \\
 &= (\overline{A} + \overline{C})\overline{BC}\overline{D} + \overline{BC} && \text{Apply DeMorgan and factor} \\
 &= \overline{A}\overline{BC}\overline{D} + \overline{BC}\overline{D} + \overline{BC} && \text{Distributive law} \\
 &= \overline{BC}\overline{D}(1 + \overline{A}) + \overline{BC} && \text{Factor} \\
 &= \overline{BC}\overline{D} + \overline{BC} && \text{Rule: } 1 + A = 1
 \end{aligned}$$

The VHDL program for the logic represented by the reduced expression is



```

entity ReducedLogic is
  port (B, C, D: in bit; X: out bit);
end entity ReducedLogic;
architecture Expression2 of ReducedLogic is
  begin
    X <= (B and not C and not D) or (B and C);
  end architecture Expression2;
    
```

3 inputs and 1 output are described.

The simplified logic contains three inputs, 3 AND gates, 1 OR gate, and 2 inverters.

As you can see, Boolean simplification is applicable to even simple VHDL programs.

### Related Problem

Write the VHDL architecture statement for the expression  $X = (\overline{A} + B + C)D$  as stated. Apply any applicable Boolean rules and rewrite the VHDL statement.

Example 4-38 demonstrates a more significant reduction in VHDL code complexity, using a Karnaugh map to reduce an expression.

### EXAMPLE 4-38

- Write a VHDL program to describe the following SOP expression.
- Minimize the expression and show how much the VHDL program is simplified.

$$\begin{aligned}
 X = & \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D \\
 & + \overline{A}BC\overline{D} + \overline{A}BCD + A\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}D + A\overline{B}C\overline{D} + A\overline{B}CD \\
 & + AB\overline{C}\overline{D} + AB\overline{C}D + ABC\overline{D} + ABCD
 \end{aligned}$$

### Solution

- The VHDL program for the SOP expression without minimization is large and hard to follow as you can see in the following VHDL code. Code such as this is subject to error. The VHDL program for the original SOP expression is as follows:



```

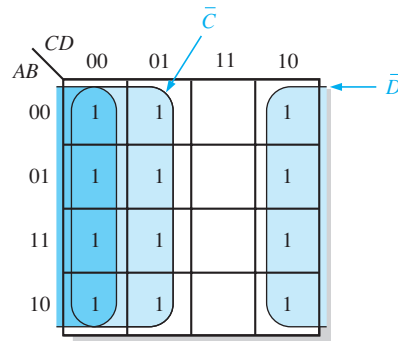
entity OriginalSOP is
  port (A, B, C, D: in bit; X: out bit);
end entity OriginalSOP;
architecture Equation1 of OriginalSOP is
  begin
    X <= (not A and not B and not C and not D) or
          (not A and not B and not C and D) or
          (not A and B and not C and not D) or
          (not A and B and C and not D) or
          (not A and not B and C and not D) or
          (A and not B and not C and not D) or
          (A and not B and C and not D) or
          (A and B and C and not D) or
          (A and B and not C and not D) or
    
```

```

(A and not B and not C and D) or
(not A and B and not C and D) or
(A and B and not C and D);
end architecture Equation1;

```

- (b) Now, use a four-variable Karnaugh map to reduce the original SOP expression to a minimum form. The original SOP expression is mapped in Figure 4–48.



**FIGURE 4–48**

The original SOP Boolean expression that is plotted on the Karnaugh map in Figure 4–48 contains twelve 4-variable terms as indicated by the twelve 1s on the map. Recall that only the variables that do not change within a group remain in the expression for that group. The simplified expression taken from the map is developed next.

Combining the terms from the Karnaugh map, you get the following simplified expression, which is equivalent to the original SOP expression.

$$X = \bar{C} + \bar{D}$$

Using the simplified expression, the VHDL code can be rewritten with fewer terms, making the code more readable and easier to modify. Also, the logic implemented in a target device by the reduced code consumes much less space in the PLD. The VHDL program for the simplified SOP expression is as follows:



```

entity SimplifiedSOP is
  port (A, B, C, D: in bit; X: out bit);
end entity SimplifiedSOP;
architecture Equation2 of SimplifiedSOP is
begin
  X <= not C or not D
end architecture Equation2;

```

### Related Problem

Write a VHDL architecture statement to describe the logic for the expression

$$X = A(BC + \bar{D})$$

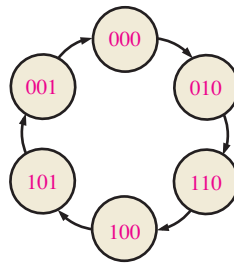
As you have seen, the simplification of Boolean logic is important in the design of any logic function described in VHDL. Target devices have finite capacity and therefore require the creation of compact and efficient program code. Throughout this chapter, you have learned that the simplification of complex Boolean logic can lead to the elimination of unnecessary logic as well as the simplification of VHDL code.

### Levels of Abstraction

A given logic function can be described at three different levels. It can be described by a truth table or a state diagram, by a Boolean expression, or by its logic diagram (schematic).

**Highest level:** The truth table or state diagram

| <i>A</i> | <i>B</i> | <i>C</i> | <i>D</i> | <i>X</i> |
|----------|----------|----------|----------|----------|
| 0        | 0        | 0        | 0        | 0        |
| 0        | 0        | 0        | 1        | 0        |
| ⋮        | ⋮        | ⋮        | ⋮        | ⋮        |
| 1        | 1        | 1        | 1        | 1        |

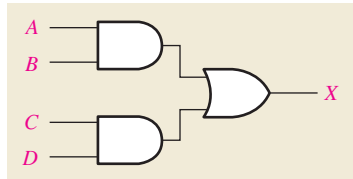


**Middle level:** The Boolean expression, which can be derived from a truth table or schematic

$$X = AB + CD$$

Logic function

**Lowest level:** The logic diagram (schematic)



**FIGURE 4-49** Illustration of the three levels of abstraction for describing a logic function.

The truth table and state diagram are the most abstract ways to describe a logic function. A Boolean expression is the next level of abstraction, and a schematic is the lowest level of abstraction. This concept is illustrated in Figure 4-49 for a simple logic circuit. VHDL provides three approaches for describing functions that correspond to the three levels of abstraction.

- The data flow approach is analogous to describing a logic function with a Boolean expression. The data flow approach specifies each of the logic gates and how the data flows through them. This approach was applied in Examples 4-37 and 4-38.
- The structural approach is analogous to using a logic diagram or schematic to describe a logic function. It specifies the gates and how they are connected, rather than how signals (data) flow through them. The structural approach is used to develop VHDL code for describing logic circuits in Chapter 5.
- The behavioral approach is analogous to describing a logic function using a state diagram or truth table. However, this approach is the most complex; it is usually restricted to logic functions whose operations are time dependent and normally require some type of memory.

#### SECTION 4-12 CHECKUP

1. What are the advantages of Boolean logic simplification in terms of writing a VHDL program?
2. How does Boolean logic simplification benefit a VHDL program in terms of the target device?
3. Name the three levels of abstraction for a combinational logic function and state the corresponding VHDL approaches for describing a logic function.



## Applied Logic

### Seven-Segment Display

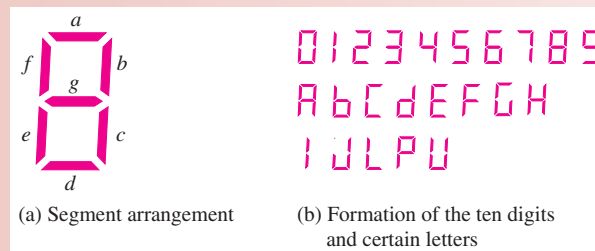
Seven-segment displays are used in many types of products that you see every day. A 7-segment display was used in the tablet-bottling system that was introduced in Chapter 1. The display in the bottling system is driven by logic circuits that decode a binary coded decimal (BCD) number and activate the appropriate digits on the display. BCD-to-7-segment decoder/drivers are readily available as single IC packages for activating the ten decimal digits.

In addition to the numbers from 0 to 9, the 7-segment display can show certain letters. For the tablet-bottling system, a requirement has been added to display the letters A, b, C, d, and E on a separate common-anode 7-segment display that uses a hexadecimal keypad for both the numerical inputs and the letters. These letters will be used to identify the type of vitamin tablet that is being bottled at any given time. In this application, the decoding logic for displaying the five letters is developed.

#### The 7-Segment Display

Two types of 7-segment displays are the LED and the LCD. Each of the seven segments in an LED display uses a light-emitting diode to produce a colored light when there is current through it and can be seen in the dark. An LCD or liquid-crystal display operates by polarizing light so that when a segment is not activated by a voltage, it reflects incident light and appears invisible against its background; however, when a segment is activated, it does not reflect light and appears black. LCD displays cannot be seen in the dark.

The seven segments in both LED and LCD displays are arranged as shown in Figure 4–50 and labeled *a*, *b*, *c*, *d*, *e*, *f*, and *g* as indicated in part (a). Selected segments are activated to create each of the ten decimal digits as well as certain letters of the alphabet, as shown in part (b). The letter *b* is shown as lowercase because a capital B would be the same as the digit 8. Similarly, for *d*, a capital letter would appear as a 0.



**FIGURE 4-50** Seven-segment display.

#### Exercise

1. List the segments used to form the digit 2.
2. List the segments used to form the digit 5.
3. List the segments used to form the letter A.
4. List the segments used to form the letter E.
5. Is there any one segment that is common to all digits?
6. Is there any one segment that is common to all letters?

## Display Logic

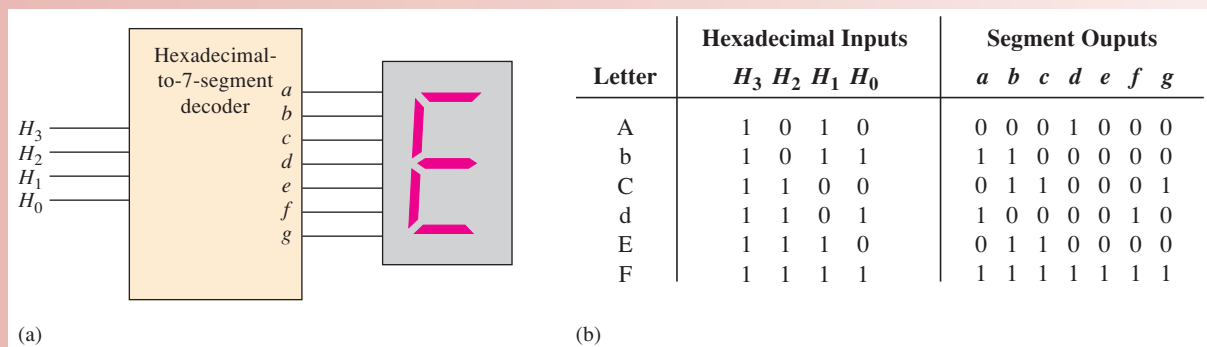
The segments in a 7-segment display can be used in the formation of various letters as shown in Figure 4–50(b). Each segment must be activated by its own decoding circuit that detects the code for any of the letters in which that segment is used. Because a common-anode display is used, the segments are turned *on* with a LOW (0) logic level and turned *off* with a HIGH (1) logic level. The active segments are shown for each of the letters required for the tablet-bottling system in Table 4–14. Even though the active level is LOW (lighting the LED), the logic expressions are developed exactly the same way as discussed in this chapter, by mapping the desired output (1, 0, or X) for every possible input, grouping the 1s on the map, and reading the SOP expression from the map. In effect, the reduced logic expression is the logic for keeping a given segment OFF. At first, this may sound confusing, but it is simple in practice and it avoids an output current capability issue with bipolar (TTL) logic (discussed in Chapter 15 on the website).

**TABLE 4-14**

Active segments for each of the five letters used in the system display.

| Letter | Segments Activated      |
|--------|-------------------------|
| A      | <i>a, b, c, e, f, g</i> |
| b      | <i>c, d, e, f, g</i>    |
| C      | <i>a, d, e, f</i>       |
| d      | <i>b, c, d, e, g</i>    |
| E      | <i>a, d, e, f, g</i>    |

A block diagram of a 7-segment logic and display for generating the five letters is shown in Figure 4–51(a), and the truth table is shown in part (b). The logic has four hexadecimal inputs and seven outputs, one for each segment. Because the letter F is not used as an input, we will show it on the truth table with all outputs set to 1 (OFF).



**FIGURE 4-51** Hexadecimal-to-7-segment decoder for letters A through E, used in the system.

## Karnaugh Maps and the Invalid BCD Code Detector

To develop the simplified logic for each segment, the truth table information in Figure 4–51 is mapped onto Karnaugh maps. Recall that the BCD numbers will not be shown on the letter display. For this reason, an entry that represents a BCD number will be entered as an “X” (“don’t care”) on the K-maps. This makes the logic much simpler but would put some strange outputs on the display unless steps are taken to eliminate that possibility. Because all of the letters are *invalid* BCD characters, the display is activated only when an invalid BCD code is entered into the keypad, thus allowing only letters to be displayed.

### Expressions for the Segment Logic

Using the table in 4–51(b), a standard SOP expression can be written for each segment and then minimized using a K-map. The desired outputs from the truth table are entered in the appropriate cells representing the hex inputs. To obtain the minimum SOP expressions for the display logic, the 1s and Xs are grouped.

**Segment *a*** Segment *a* is used for the letters A, C, and E. For the letter A, the hexadecimal code is 1010 or, in terms of variables,  $H_3\bar{H}_2H_1\bar{H}_0$ . For the letter C, the hexadecimal code is 1100 or  $H_3H_2\bar{H}_1\bar{H}_0$ . For the letter E, the code is 1110 or  $H_3H_2H_1\bar{H}_0$ . The complete standard SOP expression for segment *a* is

$$a = H_3\bar{H}_2H_1\bar{H}_0 + H_3H_2\bar{H}_1\bar{H}_0 + H_3H_2H_1\bar{H}_0$$

Because a LOW is the active output state for each segment logic circuit, a 0 is entered on the Karnaugh map in each cell that represents the code for the letters in which the segment is *on*. The simplification of the expression for segment *a* is shown in Figure 4–52(a) after grouping the 1s and Xs.

**Segment *b*** Segment *b* is used for the letters A and d. The complete standard SOP expression for segment *b* is

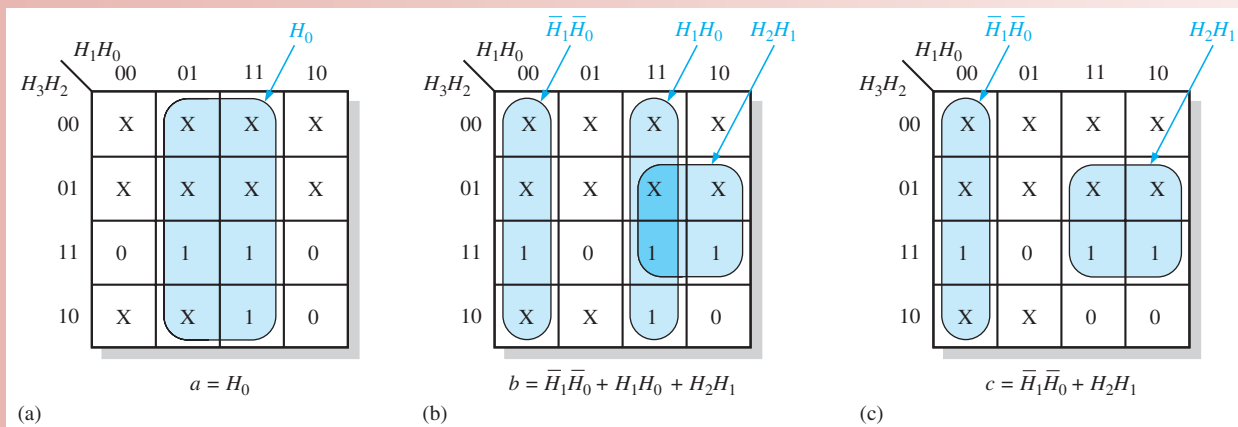
$$b = H_3\bar{H}_2H_1\bar{H}_0 + H_3H_2\bar{H}_1H_0$$

The simplification of the expression for segment *b* is shown in Figure 4–52(b).

**Segment *c*** Segment *c* is used for the letters A, b, and d. The complete standard SOP expression for segment *c* is

$$c = H_3\bar{H}_2H_1\bar{H}_0 + H_3\bar{H}_2H_1H_0 + H_3H_2\bar{H}_1H_0$$

The simplification of the expression for segment *c* is shown in Figure 4–52(c).



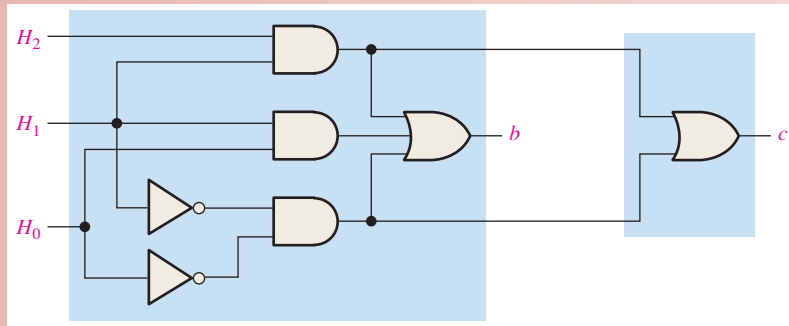
**FIGURE 4–52** Minimization of the expressions for segments *a*, *b*, and *c*.

### Exercise

7. Develop the minimum expression for segment *d*.
8. Develop the minimum expression for segment *e*.
9. Develop the minimum expression for segment *f*.
10. Develop the minimum expression for segment *g*.

### The Logic Circuits

From the minimum expressions, the logic circuits for each segment can be implemented. For segment *a*, connect the  $H_0$  input directly (no gate) to the *a* segment on the display. The segment *b* and segment *c* logic are shown in Figure 4–53 using AND or OR gates. Notice that two of the terms ( $H_2H_1$  and  $\bar{H}_1\bar{H}_0$ ) appear in the expressions for both *b* and *c* logic so two of the AND gates can be used in both, as indicated.



**FIGURE 4-53** Segment-*b* and segment-*c* logic circuits.

### Exercise

11. Show the logic for segment *d*.
12. Show the logic for segment *e*.
13. Show the logic for segment *f*.
14. Show the logic for segment *g*.



### Describing the Decoding Logic with VHDL

The 7-segment decoding logic can be described using VHDL for implementation in a programmable logic device (PLD). The logic expressions for segments *a*, *b*, and *c* of the display are as follows:

$$\begin{aligned} a &= H_0 \\ b &= \overline{H_1}\overline{H_0} + H_1H_0 + H_2H_1 \\ c &= \overline{H_1}\overline{H_0} + H_2H_1 \end{aligned}$$

- ◆ The VHDL code for segment *a* is

```
entity SEGLOGIC is
    port (H0: in bit; SEGa: out bit);
end entity SEGLOGIC;
architecture LogicFunction of SEGLOGIC is
begin
    SEGa <= H0;
end architecture LogicFunction;
```

- ◆ The VHDL code for segment *b* is

```
entity SEGLOGIC is
    port (H0, H1, H2: in bit; SEGb: out bit);
end entity SEGLOGIC;
architecture LogicFunction of SEGLOGIC is
begin
    SEGb <= (not H1 and not H0) or (H1 and H0) or (H2 and H1);
end architecture LogicFunction;
```

- ◆ The VHDL code for segment *c* is

```
entity SEGLOGIC is
    port (H0, H1, H2: in bit; SEGc: out bit);
end entity SEGLOGIC;
architecture LogicFunction of SEGLOGIC is
begin
    SEGc <= (not H1 and not H0) or (H2 and H1);
end architecture LogicFunction;
```

### Exercise

15. Write the VHDL code for segments *d*, *e*, *f*, and *g*.

### Simulation

The decoder simulation using Multisim is shown in Figure 4–54 with the letter E selected. Subcircuits are used for the segment logic to be developed as activities or in the lab. The purpose of simulation is to verify proper operation of the circuit.

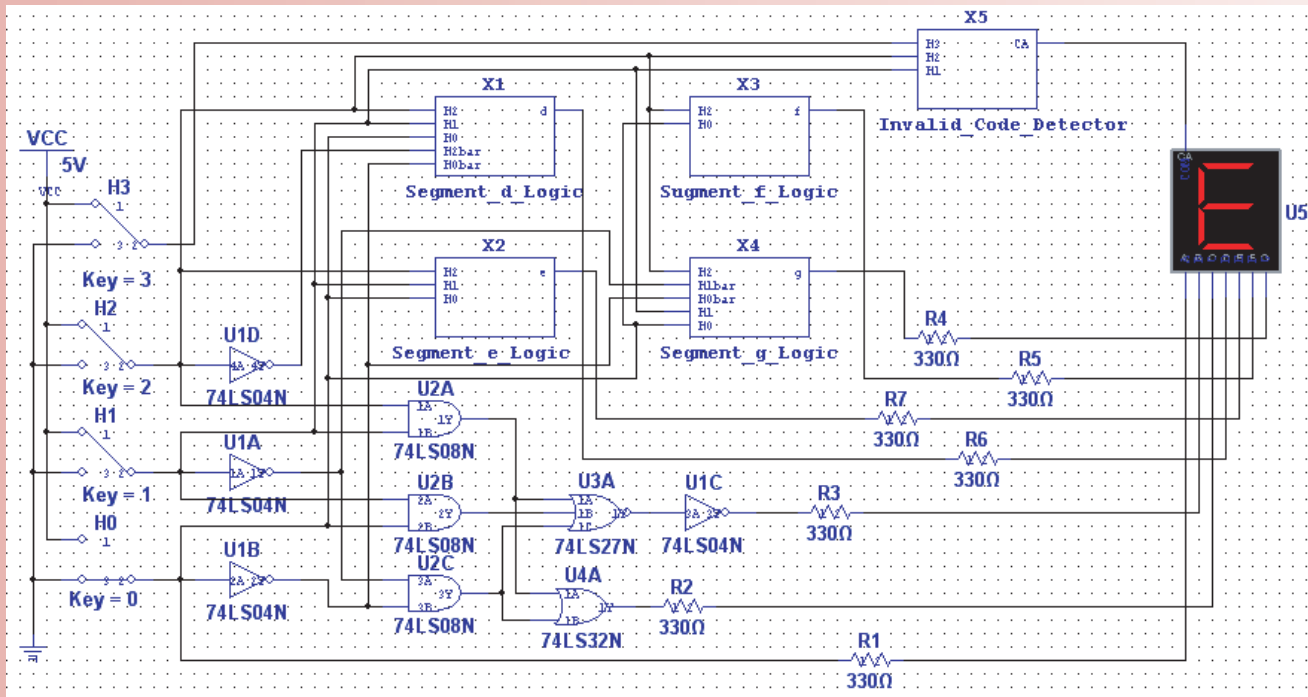


FIGURE 4–54 Multisim circuit screen for decoder and display.



Open file AL04 in the Applied Logic folder on the website. Run the simulation of the decoder and display using your Multisim software. Observe the operation for the specified letters.

### Putting Your Knowledge to Work

How would you modify the decoder for a common-cathode 7-segment display?

## SUMMARY

- Gate symbols and Boolean expressions for the outputs of an inverter and 2-input gates are shown in Figure 4–55.



FIGURE 4–55

## SUMMARY

- Commutative laws:  $A + B = B + A$   
 $AB = BA$
- Associative laws:  $A + (B + C) = (A + B) + C$   
 $A(BC) = (AB)C$
- Distributive law:  $A(B + C) = AB + AC$
- Boolean rules:
 

|                      |                               |
|----------------------|-------------------------------|
| 1. $A + 0 = A$       | 7. $A \cdot A = A$            |
| 2. $A + 1 = 1$       | 8. $A \cdot \bar{A} = 0$      |
| 3. $A \cdot 0 = 0$   | 9. $\bar{\bar{A}} = A$        |
| 4. $A \cdot 1 = A$   | 10. $A + AB = A$              |
| 5. $A + A = A$       | 11. $A + \bar{A}B = A + B$    |
| 6. $A + \bar{A} = 1$ | 12. $(A + B)(A + C) = A + BC$ |
- DeMorgan's theorems:
  - The complement of a product is equal to the sum of the complements of the terms in the product.  
$$\overline{XY} = \bar{X} + \bar{Y}$$
  - The complement of a sum is equal to the product of the complements of the terms in the sum.  
$$\overline{X + Y} = \bar{X}\bar{Y}$$
- Karnaugh maps for 3 variables have 8 cells and for 4 variables have 16 cells.
- Quinn-McCluskey is a method for simplification of Boolean expressions.
- The three levels of abstraction in VHDL are data flow, structural, and behavioral.

## KEY TERMS

Key terms and other bold terms in the chapter are defined in the end-of-book glossary.

**Complement** The inverse or opposite of a number. In Boolean algebra, the inverse function, expressed with a bar over a variable. The complement of a 1 is 0, and vice versa.

**“Don’t care”** A combination of input literals that cannot occur and can be used as a 1 or a 0 on a Karnaugh map for simplification.

**Karnaugh map** An arrangement of cells representing the combinations of literals in a Boolean expression and used for a systematic simplification of the expression.

**Minimization** The process that results in an SOP or POS Boolean expression that contains the fewest possible literals per term.

**Product-of-sums (POS)** A form of Boolean expression that is basically the ANDing of ORed terms.

**Product term** The Boolean product of two or more literals equivalent to an AND operation.

**Sum-of-products (SOP)** A form of Boolean expression that is basically the ORing of ANDed terms.

**Sum term** The Boolean sum of two or more literals equivalent to an OR operation.

**Variable** A symbol used to represent an action, a condition, or data that can have a value of 1 or 0, usually designated by an italic letter or word.

## TRUE/FALSE QUIZ

Answers are at the end of the chapter.

- Variable, complement, and literal are all terms used in Boolean algebra.
- Addition in Boolean algebra is equivalent to the NOR function. **equivalent to OR function**
- Multiplication in Boolean algebra is equivalent to the AND function.
- The commutative law, associative law, and distributive law are all laws in Boolean algebra.
- The complement of 0 is 0 itself. **complement of 0 is 1**
- When a Boolean variable is multiplied by its complement, the result is the variable. **the result is 0**  
**when Boolean variable is multiplied by itself, the result is that variable.**

**SECOND LAW STATEMENT:** The complement of a sum of variables is equal to the product of the complements of the variables.

✓ “The complement of a product of variables is equal to the sum of the complements of each variable” is a statement of DeMorgan’s theorem.

8. SOP means sum-of-products. **POS: product-of-sum**

9. Karnaugh maps can be used to simplify Boolean expressions.

cannot have exactly 6 cells

10. A 3-variable Karnaugh map has six cells. **8 cells**

11. VHDL is a type of hardware definition language.

12. A VHDL program consists of an entity and an architecture.

**Entity:** This is the external view of the circuit. It declares the circuit’s name and its ports (the inputs and outputs). It tells the outside world what the circuit is.

**Architecture:** This is the internal view of the circuit. It describes the actual behavior or structure of the logic, how the inputs relate to the outputs. It tells the outside world how the circuit works.

## SELF-TEST

Answers are at the end of the chapter.

1. A variable is a symbol in Boolean algebra used to represent

- (a) data (b) a condition  
(c) an action (d) answers (a), (b), and (c)

2. The Boolean expression  $A + B + C$  is

- (a) a sum term (b) a literal term **A literal is a variable or the complement of a variable.**  
(c) an inverse term (d) a product term

3. The Boolean expression  $\overline{ABCD}$  is

- (a) a sum term (b) a literal term  
(c) an inverse term (d) a product term

4. The domain of the expression  $\overline{ABCD} + \overline{AB} + \overline{CD} + B$  is

- (a) A and D (b) B only  
(c) A, B, C, and D (d) none of these

5. According to the associative law of addition,

- (a)  $A + B = B + A$  (b)  $A = A + A$   
(c)  $(A + B) + C = A + (B + C)$  (d)  $A + 0 = A$

6. According to commutative law of multiplication,

- (a)  $AB = BA$  (b)  $A = AA$   
(c)  $(AB)C = A(BC)$  (d)  $A0 = A$

7. According to the distributive law,

- (a)  $A(B + C) = AB + AC$  (b)  $A(BC) = ABC$   
(c)  $A(A + 1) = A$  (d)  $A + AB = A$

8. Which one of the following is *not* a valid rule of Boolean algebra?

- (a)  $A + 1 = 1$  (b)  $A = \overline{A}$   
(c)  $AA = A$  (d)  $A + 0 = A$

9. Which of the following rules states that if one input of an AND gate is always 1, the output is equal to the other input?

- (a)  $A + 1 = 1$  (b)  $A + A = A$   
(c)  $A \cdot A = A$  (d)  $A \cdot 1 = A$

10. According to DeMorgan’s theorems, the complement of a product of variables is equal to

- (a) the complement of the sum  $(A+B)'$  (b) the sum of the complements  $A'+B'$   
(c) the product of the complements (d) answers (a), (b), and (c)

11. The Boolean expression  $X = (A + B)(C + D)$  represents  
(a) two ORs ANDed together (b) two ANDs ORed together  **$X = (AB) + (CD)$**   
(c) A 4-input AND gate (d) a 4-input OR gate  **$X = A+B+C+D$**

12. An example of a sum-of-products expression is

- (a)  $A + B(C + D)$  (b)  $\overline{AB} + \overline{AC} + \overline{ABC}$   
(c)  $(\overline{A} + B + C)(A + \overline{B} + C)$  (d) both answers (a) and (b)

13. An example of a product-of-sums expression is

- (a)  $A(B + C) + \overline{AC}$  (b)  $(A + B)(\overline{A} + B + \overline{C})$   
(c)  $\overline{A} + \overline{B} + BC$  (d) both answers (a) and (b)

14. An example of a standard SOP expression is

- (a)  $\overline{AB} + \overline{ABC} + \overline{ABD}$  (b)  $\overline{ABC} + \overline{ACD}$   
(c)  $\overline{AB} + \overline{AB} + \overline{AB}$  (d)  $\overline{ABCD} + \overline{AB} + \overline{A}$

A standard SOP expression is one in which all the variables in the domain appear in each product term in the expression.

product  
term means  
SOP

15. A 4-variable Karnaugh map has no. of cells =  $2^{\text{(number of variables)}}$   
 (a) four cells (2-variable) (b) eight cells (3-variable)  
 (c) sixteen cells (d) thirty-two cells (5-variable)
16. In a 4-variable Karnaugh map, a 2-variable product term is produced by (for POS: ans is d)  
 (a) a 2-cell group of 1s (b) an 8-cell group of 1s  
 (c) a 4-cell group of 1s (d) a 4-cell group of 0s cell group size =  $2^{\text{(total variables - number of variables in term)}}$
17. The Quine-McCluskey method can be used to  
 (a) replace the Karnaugh map method (b) simplify expressions with 5 or more variables  
 (c) both (a) and (b) (d) none of the above
18. VHDL is a type of VHDL stands for VHSIC (Very High Speed Integrated Circuit) Hardware Description Language.  
 (a) programmable logic (b) hardware description language It is a language used to describe the structure and behavior of electronic circuits, such as digital logic and microprocessors, before they are physically manufactured.  
 (c) programmable array (d) logical mathematics
19. In VHDL, a port is  
 (a) a type of entity (b) a type of architecture  
 (c) an input or output (d) a type of variable
20. Using VHDL, a logic circuit's inputs and outputs are described in the  
 (a) architecture (b) component  
 (c) entity (d) data flow (A port defines the interface of the digital circuit (or block). It specifies the signals that will carry information into or out of the design block, functioning as the input and output pins of the virtual chip.)

## PROBLEMS

Answers to odd-numbered problems are at the end of the book.

### Section 4-1 Boolean Operations and Expressions

- Using Boolean notation, write an expression that is a 0 only when all of its variables ( $A$ ,  $B$ ,  $C$ , and  $D$ ) are 0s.  $X = A+B+C+D$
- Write an expression that is a 1 when one or more of its variables ( $A$ ,  $B$ ,  $C$ ,  $D$ , and  $E$ ) are 0s.  $X = A'+B'+C'+D'+E'$  OR  $X = (ABCDE)'$
- Write an expression that is a 0 when one or more of its variables ( $A$ ,  $B$ , and  $C$ ) are 0s.  $X = ABC$
- Evaluate the following operations:  
 (a)  $0 + 0 + 0 + 0$  (b)  $0 + 0 + 0 + 1$  (c)  $1 + 1 + 1 + 1$   
 (d)  $1 \cdot 1 + 0 \cdot 0 + 1$  (e)  $1 \cdot 0 \cdot 1 \cdot 0$  (f)  $1 \cdot 0 + 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 1$
- Find the values of the variables that make each product term 1 and each sum term 0.  
 (a)  $ABC$  (b)  $A + B + C$  (c)  $\overline{A}\overline{B}C$  (d)  $\overline{A} + \overline{B} + C$   
 (e)  $A + \overline{B} + \overline{C}$  (f)  $\overline{A} + \overline{B} + \overline{C}$
- Find the value of  $X$  for all possible values of the variables.  
 (a)  $X = A + B + C$  (b)  $X = (A + B)C$  (c)  $X = (A + B)(\overline{B} + \overline{C})$   
 (d)  $X = (A + B) + (\overline{AB} + \overline{BC})$  (e)  $X = (\overline{A} + \overline{B})(A + B)$

### Section 4-2 Laws and Rules of Boolean Algebra

- Identify the law of Boolean algebra upon which each of the following equalities is based:  
 (a)  $A + AB + ABC + \overline{ABCD} = \overline{ABCD} + ABC + AB + A$  commutative  
 (b)  $A + \overline{AB} + ABC + \overline{ABCD} = \overline{DCBA} + CBA + \overline{BA} + A$  commutative  
 (c)  $AB(CD + \overline{CD} + EF + \overline{EF}) = ABCD + \overline{ABCD} + AB\overline{EF} + AB\overline{EF}$  distributive
- Identify the Boolean rule(s) on which each of the following equalities is based:  
 (a)  $\overline{\overline{AB} + \overline{CD}} + \overline{EF} = AB + CD + \overline{EF}$  (b)  $A\overline{AB} + AB\overline{C} + AB\overline{B} = AB\overline{C}$   
 (c)  $A(BC + \overline{BC}) + AC = A(BC) + AC$  (d)  $AB(C + \overline{C}) + AC = AB + AC$   
 (e)  $A\overline{B} + A\overline{B}C = A\overline{B}$  (f)  $ABC + \overline{AB} + \overline{ABCD} = ABC + \overline{AB} + D$

### Section 4-3 DeMorgan's Theorems

- Apply DeMorgan's theorems to each expression:  
 (a)  $A + \overline{B}$  (b)  $\overline{AB}$  (c)  $\overline{A + B + C}$  (d)  $\overline{ABC}$   
 (e)  $\overline{A(B + C)}$  (f)  $\overline{AB} + \overline{CD}$  (g)  $\overline{AB + CD}$  (h)  $\overline{(A + B)(C + D)}$

standard  
priority order  
for evaluating  
Boolean  
expressions:  
1) Parenthesis  
2) NOT  
3) AND  
4) OR

10. Apply DeMorgan's theorems to each expression:

- (a)  $\overline{AB(C + \overline{D})}$  (b)  $\overline{AB(CD + EF)}$   
 (c)  $\overline{(A + \overline{B} + C + \overline{D}) + ABCD}$  (d)  $\overline{(A + B + C + D)(\overline{A}\overline{B}\overline{C}\overline{D})}$   
 (e)  $\overline{AB(CD + \overline{EF})(\overline{AB} + \overline{CD})}$

11. Apply DeMorgan's theorems to the following:

- (a)  $\overline{(ABC)(\overline{EFG}) + (\overline{HIJ})(KLM)}$  (b)  $\overline{(A + \overline{BC} + CD) + \overline{BC}}$   
 (c)  $\overline{(A + B)(C + D)(\overline{E} + \overline{F})(\overline{G} + \overline{H})}$

### Section 4-4 Boolean Analysis of Logic Circuits

12. Write the Boolean expression for each of the logic gates in Figure 4-56.

- a)  $X = A.B$   
 b)  $X = A'$   
 c)  $X = A + B$   
 d)  $X = A + B + C$

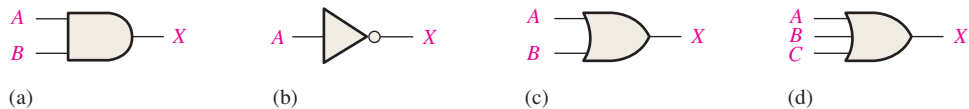


FIGURE 4-56

13. Write the Boolean expression for each of the logic circuits in Figure 4-57.

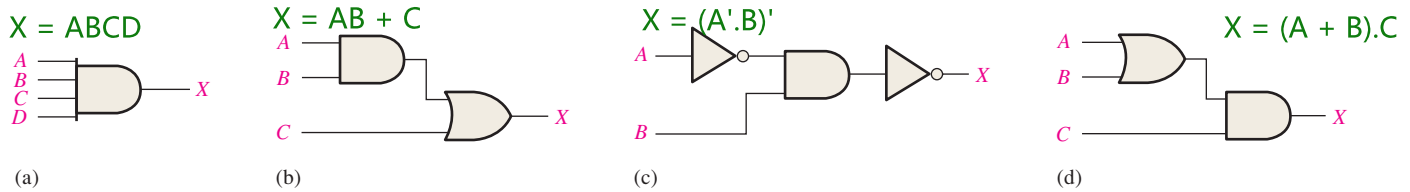


FIGURE 4-57

14. Draw the logic circuit represented by each of the following expressions:

- (a)  $A + B + C + D$  (b)  $ABCD$   
 (c)  $A + BC$  (d)  $ABC + D$

15. Draw the logic circuit represented by each expression:

- (a)  $AB + \overline{AB}$  (b)  $ABCD$   
 (c)  $A + BC$  (d)  $ABC + D$

16. (a) Draw a logic circuit for the case where the output, ENABLE, is HIGH only if the inputs, ASSERT and READY, are both LOW.

(b) Draw a logic circuit for the case where the output, HOLD, is HIGH only if the input, LOAD, is LOW and the input, READY, is HIGH.

17. Develop the truth table for each of the circuits in Figure 4-58.

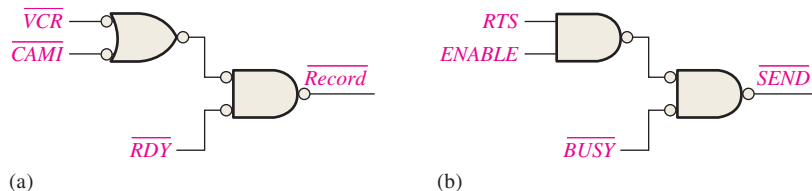


FIGURE 4-58

18. Construct a truth table for each of the following Boolean expressions:

- (a)  $A + B + C$  (b)  $ABC$  (c)  $AB + BC + CA$   
 (d)  $(A + B)(B + C)(C + A)$  (e)  $\overline{AB} + \overline{BC} + \overline{CA}$

### Section 4-5 Logic Simplification Using Boolean Algebra

19. Using Boolean algebra techniques, simplify the following expressions as much as possible:

- (a)  $A(A + B)$  (b)  $A(\overline{A} + AB)$  (c)  $BC + \overline{BC}$   
 (d)  $A(A + \overline{AB})$  (e)  $\overline{ABC} + \overline{ABC} + \overline{ABC}$

20. Using Boolean algebra, simplify the following expressions:

- (a)  $(\bar{A} + B)(A + C)$       (b)  $A\bar{B} + A\bar{B}C + A\bar{B}CD + A\bar{B}CDE$   
 (c)  $BC + \overline{BCD} + B$       (d)  $(B + \bar{B})(BC + B\bar{C})$   
 (e)  $BC + (\bar{B} + \bar{C})D + BC$

21. Using Boolean algebra, simplify the following expressions:

- (a)  $CE + C(E + F) + \bar{E}(E + G)$       (b)  $\bar{B}\bar{C}D + (\bar{B} + \bar{C} + \bar{D}) + \bar{B}\bar{C}\bar{D}E$   
 (c)  $(C + CD)(C + \bar{C}D)(C + E)$       (d)  $BCDE + BC(\bar{D}E) + (\bar{B}C)DE$   
 (e)  $BCD[BC + \bar{D}(CD + BD)]$

22. Determine which of the logic circuits in Figure 4–59 are equivalent.

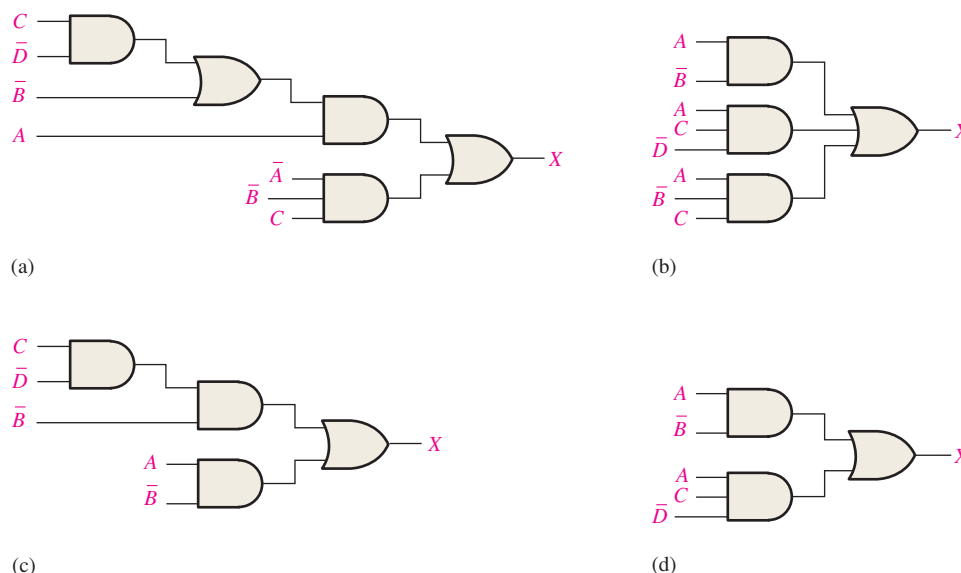


FIGURE 4–59

### Section 4–6 Standard Forms of Boolean Expressions

23. Convert the following expressions to sum-of-product (SOP) forms:

- (a)  $(C + D)(A + \bar{D})$       (b)  $A(\bar{A}\bar{D} + C)$       (c)  $(A + C)(CD + AC)$

24. Convert the following expressions to sum-of-product (SOP) forms:

- (a)  $BC + DE(\bar{B}C + DE)$       (b)  $BC(\bar{C}\bar{D} + CE)$       (c)  $B + C[BD + (C + \bar{D})E]$

25. Define the domain of each SOP expression in Problem 23 and convert the expression to standard SOP form.

26. Convert each SOP expression in Problem 24 to standard SOP form.

27. Determine the binary value of each term in the standard SOP expressions from Problem 25.

28. Determine the binary value of each term in the standard SOP expressions from Problem 26.

29. Convert each standard SOP expression in Problem 25 to standard POS form.

30. Convert each standard SOP expression in Problem 26 to standard POS form.

### Section 4–7 Boolean Expressions and Truth Tables

31. Develop a truth table for each of the following standard SOP expressions:

- (a)  $ABC + \bar{A}\bar{B}C + ABC$       (b)  $\bar{X}\bar{Y}\bar{Z} + \bar{X}Y\bar{Z} + X\bar{Y}\bar{Z} + \bar{X}YZ + XY\bar{Z}$

32. Develop a truth table for each of the following standard SOP expressions:

- (a)  $A\bar{B}\bar{C}\bar{D} + ABC\bar{D} + \bar{A}\bar{B}CD + \bar{A}\bar{B}\bar{C}\bar{D}$   
 (b)  $WXYZ + \bar{W}\bar{X}\bar{Y}Z + W\bar{X}\bar{Y}\bar{Z} + \bar{W}\bar{X}YZ + WX\bar{Y}\bar{Z}$

33. Develop a truth table for each of the SOP expressions:

- (a)  $\bar{A}B + A\bar{B}C + \bar{A}\bar{C} + \bar{A}BC$       (b)  $\bar{X} + Y\bar{Z} + WZ + \bar{X}\bar{Y}Z$

34. Develop a truth table for each of the standard POS expressions:
- (a)  $(\bar{A} + \bar{B} + \bar{C})(A + B + C)(A + B + \bar{C})$
- (b)  $(A + \bar{B} + C + \bar{D})(\bar{A} + B + \bar{C} + D)(A + B + \bar{C} + \bar{D})(\bar{A} + \bar{B} + C + D)$
35. Develop a truth table for each of the standard POS expressions:
- (a)  $(A + B)(A + C)(A + B + C)$
- (b)  $(A + \bar{B})(A + \bar{B} + \bar{C})(B + C + \bar{D})(\bar{A} + B + \bar{C} + D)$
36. For each truth table in Table 4–15, derive a standard SOP and a standard POS expression.

**TABLE 4–15**

| $ABC$ |   | $ABC$ |   | $ABCD$ | $X$ | $ABCD$ | $X$ |
|-------|---|-------|---|--------|-----|--------|-----|
| 000   | 0 | 000   | 0 | 0000   | 1   | 0000   | 0   |
| 001   | 1 | 001   | 0 | 0001   | 1   | 0001   | 0   |
| 010   | 0 | 010   | 0 | 0010   | 0   | 0010   | 1   |
| 011   | 0 | 011   | 0 | 0011   | 1   | 0011   | 0   |
| 100   | 1 | 100   | 0 | 0100   | 0   | 0100   | 1   |
| 101   | 1 | 101   | 1 | 0101   | 1   | 0101   | 1   |
| 110   | 0 | 110   | 1 | 0110   | 1   | 0110   | 0   |
| 111   | 1 | 111   | 1 | 0111   | 0   | 0111   | 1   |
|       |   |       |   | 1000   | 0   | 1000   | 0   |
|       |   |       |   | 1001   | 1   | 1001   | 0   |
|       |   |       |   | 1010   | 0   | 1010   | 0   |
|       |   |       |   | 1011   | 0   | 1011   | 1   |
|       |   |       |   | 1100   | 1   | 1100   | 1   |
|       |   |       |   | 1101   | 0   | 1101   | 0   |
|       |   |       |   | 1110   | 0   | 1110   | 0   |
|       |   |       |   | 1111   | 0   | 1111   | 1   |

### Section 4–8 The Karnaugh Map

37. Draw a 3-variable Karnaugh map and label each cell according to its binary value.
38. Draw a 4-variable Karnaugh map and label each cell according to its binary value.
39. Write the standard product term for each cell in a 3-variable Karnaugh map.

### Section 4–9 Karnaugh Map SOP Minimization

40. Use a Karnaugh map to find the minimum SOP form for each expression:
- (a)  $\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}BC$  (b)  $AC(\bar{B} + C)$
- (c)  $\bar{A}(BC + \bar{B}\bar{C}) + A(BC + \bar{B}\bar{C})$  (d)  $\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}BC + ABC$
41. Use a Karnaugh map to simplify each expression to a minimum SOP form:
- (a)  $\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}BC + ABC$  (b)  $AC[\bar{B} + B(B + \bar{C})]$
- (c)  $DEF + \bar{D}\bar{E}\bar{F} + \bar{D}E\bar{F}$
42. Expand each expression to a standard SOP form:
- (a)  $AB + \bar{A}\bar{B}C + ABC$  (b)  $A + BC$
- (c)  $\bar{A}\bar{B}\bar{C}D + A\bar{C}\bar{D} + B\bar{C}D + \bar{A}BC\bar{D}$  (d)  $\bar{A}\bar{B} + \bar{A}\bar{B}\bar{C}D + CD + \bar{B}\bar{C}D + ABCD$
43. Minimize each expression in Problem 42 with a Karnaugh map.
44. Use a Karnaugh map to reduce each expression to a minimum SOP form:
- (a)  $A + \bar{B}\bar{C} + CD$
- (b)  $\bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + ABCD + ABC\bar{D}$
- (c)  $\bar{A}B(\bar{C}\bar{D} + \bar{C}D) + AB(\bar{C}\bar{D} + \bar{C}D) + \bar{A}\bar{B}\bar{C}D$
- (d)  $(\bar{A}\bar{B} + \bar{A}\bar{B})(CD + \bar{C}\bar{D})$
- (e)  $\bar{A}\bar{B} + \bar{A}\bar{B} + \bar{C}\bar{D} + \bar{C}\bar{D}$

45. Reduce the function specified in truth Table 4–16 to its minimum SOP form by using a Karnaugh map.
46. Use the Karnaugh map method to implement the minimum SOP expression for the logic function specified in truth Table 4–17.
47. Solve Problem 46 for a situation in which the last six binary combinations are not allowed.

**TABLE 4–16**

| Inputs |   |   | Output |
|--------|---|---|--------|
| A      | B | C | X      |
| 0      | 0 | 0 | 1      |
| 0      | 0 | 1 | 1      |
| 0      | 1 | 0 | 0      |
| 0      | 1 | 1 | 1      |
| 1      | 0 | 0 | 1      |
| 1      | 0 | 1 | 1      |
| 1      | 1 | 0 | 0      |
| 1      | 1 | 1 | 1      |

**TABLE 4–17**

| Inputs |   |   |   | Output |
|--------|---|---|---|--------|
| A      | B | C | D | X      |
| 0      | 0 | 0 | 0 | 0      |
| 0      | 0 | 0 | 1 | 1      |
| 0      | 0 | 1 | 0 | 1      |
| 0      | 0 | 1 | 1 | 0      |
| 0      | 1 | 0 | 0 | 0      |
| 0      | 1 | 0 | 1 | 0      |
| 0      | 1 | 1 | 0 | 1      |
| 0      | 1 | 1 | 1 | 1      |
| 1      | 0 | 0 | 0 | 1      |
| 1      | 0 | 0 | 1 | 0      |
| 1      | 0 | 1 | 0 | 1      |
| 1      | 0 | 1 | 1 | 0      |
| 1      | 1 | 0 | 0 | 1      |
| 1      | 1 | 0 | 1 | 1      |
| 1      | 1 | 1 | 0 | 0      |
| 1      | 1 | 1 | 1 | 1      |

### Section 4–10 Karnaugh Map POS Minimization

48. Use a Karnaugh map to find the minimum POS for each expression:
- $(A + B + C)(\bar{A} + \bar{B} + \bar{C})(A + \bar{B} + C)$
  - $(X + \bar{Y})(\bar{X} + Z)(X + \bar{Y} + \bar{Z})(\bar{X} + \bar{Y} + Z)$
  - $A(B + \bar{C})(\bar{A} + C)(A + \bar{B} + C)(\bar{A} + B + \bar{C})$
49. Use a Karnaugh map to simplify each expression to minimum POS form:
- $(A + \bar{B} + C + \bar{D})(\bar{A} + B + \bar{C} + D)(\bar{A} + \bar{B} + \bar{C} + \bar{D})$
  - $(X + \bar{Y})(W + \bar{Z})(\bar{X} + \bar{Y} + \bar{Z})(W + X + Y + Z)$
50. For the function specified in Table 4–16, determine the minimum POS expression using a Karnaugh map.
51. Determine the minimum POS expression for the function in Table 4–17.
52. Convert each of the following POS expressions to minimum SOP expressions using a Karnaugh map:
- $(A + \bar{B})(A + \bar{C})(\bar{A} + \bar{B} + C)$
  - $(\bar{A} + B)(\bar{A} + \bar{B} + \bar{C})(B + \bar{C} + D)(A + \bar{B} + C + \bar{D})$

### Section 4–11 The Quine-McCluskey Method

53. List the minterms in the expression
- $$X = ABC + \bar{A}\bar{B}C + A\bar{B}\bar{C} + \bar{A}BC + \bar{A}BC$$
54. List the minterms in the expression
- $$X = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}B\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D} + \bar{A}B\bar{C}D + \bar{A}B\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D}$$
55. Create a table for the number of 1s in the minterms for the expression in Problem 54 (similar to Table 4–10).
56. Create a table of first level minterms for the expression in Problem 54 (similar to Table 4–11).

57. Create a table of second level minterms for the expression in Problem 54 (similar to Table 4–12).
58. Create a table of prime implicants for the expression in Problem 54 (similar to Table 4–13).
59. Determine the final reduced expression for the expression in Problem 54.

### Section 4–12 Boolean Expressions with VHDL

60. Write a VHDL program for the logic circuit in Figure 4–60.

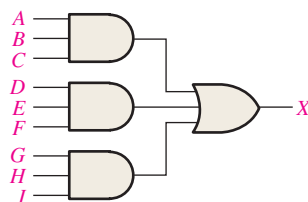


FIGURE 4–60

61. Write a program in VHDL for the expression

$$Y = \overline{A}BC + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + \overline{A}BC$$

### Applied Logic

62. If you are required to choose a type of digital display for low light conditions, will you select LED or LCD 7-segment displays? Why?
63. Explain the purpose of the invalid code detector.
64. For segment *c*, how many fewer gates and inverters does it take to implement the minimum SOP expression than the standard SOP expression?
65. Repeat Problem 64 for the logic for segments *d* through *g*.

### Special Design Problems

66. The logic for segments *b* and *c* in Figure 4–53 produces LOW outputs to activate the segments. If a type of 7-segment display is used that requires a HIGH to activate a segment, modify the logic accordingly.
67. Redesign the logic for segment *a* in the Applied Logic to include the letter F in the display.
68. Repeat Problem 67 for segments *b* through *g*.
69. Design the invalid code detector.



### Multisim Troubleshooting Practice

70. Open file P04-70. For the specified fault, predict the effect on the circuit. Then introduce the fault and verify whether your prediction is correct.
71. Open file P04-71. For the specified fault, predict the effect on the circuit. Then introduce the fault and verify whether your prediction is correct.
72. Open file P04-72. For the observed behavior indicated, predict the fault in the circuit. Then introduce the suspected fault and verify whether your prediction is correct.

## ANSWERS

### SECTION CHECKUPS

#### Section 4–1 Boolean Operations and Expressions

1.  $\overline{\overline{A}} = \overline{0} = 1$
2.  $A = 1, B = 1, C = 0; \overline{A} + \overline{B} + C = \overline{1} + \overline{1} + 0 = 0 + 0 + 0 = 0$
3.  $A = 1, B = 0, C = 1; \overline{A}BC = 1 \cdot \overline{0} \cdot 1 = 1 \cdot 1 \cdot 1 =$

#### Section 4–2 Laws and Rules of Boolean Algebra

1.  $A + (B + C + D) = (A + B + C) + D$
2.  $A(B + C + D) = AB + AC + AD$

**Section 4-3 DeMorgan's Theorems**

1. (a)  $\overline{ABC} + \overline{(D + E)} = \overline{A} + \overline{B} + \overline{C} + \overline{DE}$  (b)  $\overline{(A + B)C} = \overline{A} \overline{B} + \overline{C}$   
 (c)  $\overline{A + B + C + DE} = \overline{A} \overline{B} \overline{C} + \overline{D} + \overline{E}$

**Section 4-4 Boolean Analysis of Logic Circuits**

1.  $(C + D)B + A$   
 2. Abbreviated truth table: The expression is a 1 when  $A$  is 1 or when  $B$  and  $C$  are 1s or when  $B$  and  $D$  are 1s. The expression is 0 for all other variable combinations.

**Section 4-5 Logic Simplification Using Boolean Algebra**

1. (a)  $A + AB + \overline{A} \overline{B} C = A$  (b)  $(\overline{A} + B)C + ABC = C(\overline{A} + B)$   
 (c)  $\overline{A} \overline{B} C(BD + CDE) + A \overline{C} = A(\overline{C} + \overline{B} DE)$   
 2. (a) *Original*: 2 AND gates, 1 OR gate, 1 inverter; *Simplified*: No gates (straight connection)  
 (b) *Original*: 2 OR gates, 2 AND gates, 1 inverter; *Simplified*: 1 OR gate, 1 AND gate, 1 inverter  
 (c) *Original*: 5 AND gates, 2 OR gates, 2 inverters; *Simplified*: 2 AND gates, 1 OR gate, 2 inverters

**Section 4-6 Standard Forms of Boolean Expressions**

1. (a) SOP (b) standard POS (c) standard SOP (d) POS  
 2. (a)  $\overline{A} \overline{B} \overline{C} \overline{D} + \overline{A} \overline{B} C \overline{D} + \overline{A} B \overline{C} \overline{D} + \overline{A} B C \overline{D} + \overline{A} \overline{B} C D + \overline{A} B C D + A \overline{B} C \overline{D} + A \overline{B} C D$   
 (c) Already standard  
 3. (b) Already standard  
 (d)  $(A + \overline{B} + \overline{C})(A + \overline{B} + C)(A + B + \overline{C})(A + B + C)$

**Section 4-7 Boolean Expressions and Truth Tables**

1.  $2^5 = 32$  2.  $0110 \longrightarrow \overline{W} X Y \overline{Z}$  3.  $1100 \longrightarrow \overline{W} + \overline{X} + Y + Z$

**Section 4-8 The Karnaugh Map**

1. (a) upper left cell: 000 (b) lower right cell: 101  
 (c) lower left cell: 100 (d) upper right cell: 001  
 2. (a) upper left cell:  $\overline{X} \overline{Y} \overline{Z}$  (b) lower right cell:  $X \overline{Y} Z$   
 (c) lower left cell:  $X \overline{Y} \overline{Z}$  (d) upper right cell:  $\overline{X} Y Z$   
 3. (a) upper left cell: 0000 (b) lower right cell: 1010  
 (c) lower left cell: 1000 (d) upper right cell: 0010  
 4. (a) upper left cell:  $\overline{W} \overline{X} \overline{Y} \overline{Z}$  (b) lower right cell:  $W \overline{X} Y \overline{Z}$   
 (c) lower left cell:  $W \overline{X} \overline{Y} \overline{Z}$  (d) upper right cell:  $\overline{W} X Y \overline{Z}$

**Section 4-9 Karnaugh Map SOP Minimization**

1. 8-cell map for 3 variables; 16-cell map for 4 variables  
 2.  $AB + \overline{B} \overline{C} + \overline{A} \overline{B} C$   
 3. (a)  $\overline{A} \overline{B} \overline{C} + \overline{A} \overline{B} C + \overline{A} B \overline{C} + \overline{A} B C$   
 (b)  $\overline{A} \overline{B} \overline{C} + \overline{A} \overline{B} C + \overline{A} B \overline{C} + \overline{A} B C + A \overline{B} \overline{C} + A \overline{B} C$   
 (c)  $\overline{A} \overline{B} \overline{C} \overline{D} + \overline{A} \overline{B} \overline{C} D + \overline{A} \overline{B} C \overline{D} + \overline{A} \overline{B} C D + \overline{A} B \overline{C} \overline{D} + \overline{A} B \overline{C} D + \overline{A} B C \overline{D} + \overline{A} B C D$   
 (d)  $\overline{A} \overline{B} \overline{C} \overline{D} + \overline{A} \overline{B} \overline{C} D + \overline{A} \overline{B} C \overline{D} + \overline{A} \overline{B} C D + \overline{A} B \overline{C} \overline{D} + \overline{A} B \overline{C} D + \overline{A} B C \overline{D} + \overline{A} B C D + A \overline{B} \overline{C} \overline{D} + A \overline{B} \overline{C} D + A \overline{B} C \overline{D} + A \overline{B} C D + A B \overline{C} \overline{D} + A B \overline{C} D + A B C \overline{D} + A B C D$

**Section 4-10 Karnaugh Map POS Minimization**

1. In mapping a POS expression, 0s are placed in cells whose value makes the standard sum term zero; and in mapping an SOP expression 1s are placed in cells having the same values as the product terms.



- 2. 0 in the 1011 cell:  $\overline{A} + B + \overline{C} + \overline{D}$
- 3. 1 in the 0010 cell:  $\overline{A}\overline{B}C\overline{D}$

Section 4-11 The Quine-McCluskey Method

- 1. A minterm is a product term in which each variable appears once, either complemented or uncomplemented.
- 2. An essential prime implicant is a product term that cannot be further simplified by combining with other terms.

Section 4-12 Boolean Expressions with VHDL

- 1. Simplification can make a VHDL program shorter, easier to read, and easier to modify.
- 2. Code simplification results in less space used in a target device, thus allowing capacity for more complex circuits.
- 3. Truth table: Behavioral  
Boolean expression: Data flow  
Logic diagram: Structural

RELATED PROBLEMS FOR EXAMPLES

- 4-1  $\overline{A} + B = 0$  when  $A = 1$  and  $B = 0$ .
- 4-2  $\overline{A}\overline{B} = 1$  when  $A = 0$  and  $B = 0$ .
- 4-3  $XYZ$
- 4-4  $W + X + Y + Z$
- 4-5  $AB\overline{C}\overline{D}\overline{E}$
- 4-6  $(A + \overline{B} + \overline{C}D)\overline{E}$
- 4-7  $\overline{ABCD} = \overline{A} + \overline{B} + \overline{C} + \overline{D}$
- 4-8 Results should be same as example.
- 4-9  $A\overline{B}$
- 4-10  $CD$
- 4-11  $AB\overline{C} + \overline{A}C + \overline{A}\overline{B}$
- 4-12  $\overline{A} + \overline{B} + \overline{C}$
- 4-13 Results should be same as example.
- 4-14  $\overline{A}B\overline{C} + AB + A\overline{C} + A\overline{B} + \overline{B}\overline{C}$
- 4-15  $W\overline{X}YZ + W\overline{X}Y\overline{Z} + W\overline{X}YZ + \overline{W}\overline{X}Y\overline{Z} + W\overline{X}YZ + W\overline{X}Y\overline{Z}$
- 4-16 011, 101, 110, 010, 111. Yes
- 4-17  $(A + \overline{B} + C)(A + \overline{B} + \overline{C})(A + B + C)(\overline{A} + B + C)$
- 4-18 010, 100, 001, 111, 011. Yes
- 4-19 SOP and POS expressions are equivalent.
- 4-20 See Table 4-18.
- 4-21 See Table 4-19.

| TABLE 4-18 |   |   |   |
|------------|---|---|---|
| A          | B | C | X |
| 0          | 0 | 0 | 0 |
| 0          | 0 | 1 | 0 |
| 0          | 1 | 0 | 1 |
| 0          | 1 | 1 | 0 |
| 1          | 0 | 0 | 0 |
| 1          | 0 | 1 | 1 |
| 1          | 1 | 0 | 0 |
| 1          | 1 | 1 | 0 |

| TABLE 4-19 |   |   |   |
|------------|---|---|---|
| A          | B | C | X |
| 0          | 0 | 0 | 1 |
| 0          | 0 | 1 | 0 |
| 0          | 1 | 0 | 0 |
| 0          | 1 | 1 | 1 |
| 1          | 0 | 0 | 1 |
| 1          | 0 | 1 | 1 |
| 1          | 1 | 0 | 1 |
| 1          | 1 | 1 | 0 |

4-22 The SOP and POS expressions are equivalent.

4-23 See Figure 4-61.

4-24 See Figure 4-62.

| AB \ C | 0 | 1 |
|--------|---|---|
| 00     |   |   |
| 01     |   | 1 |
| 11     |   |   |
| 10     | 1 | 1 |

FIGURE 4-61

| AB \ CD | 00 | 01 | 11 | 10 |
|---------|----|----|----|----|
| 00      |    |    |    |    |
| 01      |    |    |    | 1  |
| 11      | 1  |    | 1  | 1  |
| 10      |    |    |    |    |

FIGURE 4-62

4-25 See Figure 4-63.

4-26 See Figure 4-64.

| AB \ C | 0 | 1 |
|--------|---|---|
| 00     | 1 |   |
| 01     | 1 | 1 |
| 11     |   | 1 |
| 10     |   |   |

FIGURE 4-63

| AB \ CD | 00 | 01 | 11 | 10 |
|---------|----|----|----|----|
| 00      |    | 1  |    |    |
| 01      |    | 1  |    | 1  |
| 11      | 1  | 1  | 1  | 1  |
| 10      | 1  | 1  | 1  | 1  |

FIGURE 4-64

4-27 No other ways

4-28  $X = B + \bar{A}C + A\bar{C}D + \bar{C}\bar{D}$

4-29  $X = \bar{D} + \bar{A}BC + B\bar{C} + \bar{A}B$

4-30  $Q = X + Y$

4-31  $Q = \bar{X}\bar{Y}\bar{Z} + W\bar{X}Z + \bar{W}YZ$

4-32 See Figure 4-65.

4-33 See Figure 4-66.

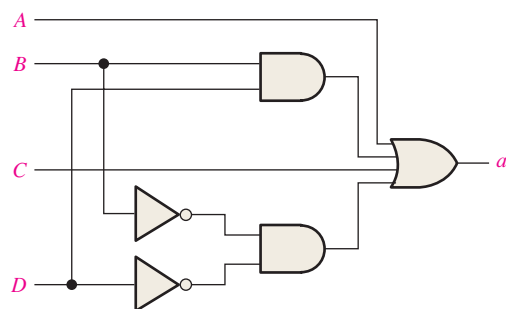


FIGURE 4-65

| AB \ CD | 00 | 01 | 11 | 10 |
|---------|----|----|----|----|
| 00      | 0  | 0  |    |    |
| 01      |    |    |    | 0  |
| 11      |    |    |    |    |
| 10      |    |    |    | 0  |

FIGURE 4-66

**4-34**  $(X + \bar{Y})(X + \bar{Z})(\bar{X} + Y + Z)$

**4-35**  $(\bar{X} + \bar{Y} + Z)(\bar{W} + \bar{X} + Z)(W + X + Y + Z)(W + \bar{X} + Y + \bar{Z})$

**4-36**  $\bar{Y}\bar{Z} + \bar{X}\bar{Z} + \bar{W}Y + \bar{X}\bar{Y}Z$

**4-37** architecture RelProb\_1 of Example4\_37 is

begin

X <= (not A or B or C) and D;

end architecture RelProb\_1;

architecture RelProb\_2 of Example4\_37 is

begin

X <= (not A and D or B and D or C and D);

end architecture RelProb\_2;

**4-38** architecture RelProb of Example4\_38 is

begin

X <= not(A and ((B and C) or not D))

end architecture RelProb;

### TRUE/FALSE QUIZ

- |      |      |      |       |       |       |
|------|------|------|-------|-------|-------|
| 1. T | 2. F | 3. T | 4. T  | 5. F  | 6. F  |
| 7. T | 8. T | 9. T | 10. F | 11. F | 12. T |

### SELF-TEST

- |         |         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|---------|
| 1. (d)  | 2. (a)  | 3. (d)  | 4. (c)  | 5. (c)  | 6. (a)  | 7. (a)  |
| 8. (b)  | 9. (d)  | 10. (b) | 11. (a) | 12. (b) | 13. (b) | 14. (c) |
| 15. (c) | 16. (c) | 17. (c) | 18. (b) | 19. (c) | 20. (c) |         |